Thakur Educational Trust's (Regd.)
THAKUR COLLEGE OF SCIENCE & COMMERCE
Empowered Autonomous College Permanently Affiliated to University of Mumbai
(NAAC Accredited with Grade "A" (3rd Cycle) & ISO 21001:2018 Certified)
Best College Award by University of Mumbai for the Year 2018-2019

DEPARTMENT OF INFORMATION TECHNOLOGY

A PROJECT REPORT ON

Browser Based Discrete – Event Simulator

"JustCmul8: A No-Code, Browser-Based Discrete Event Simulation Platform"

SUBMITTED BY

Ms. Khushi Singh – 261713

Mr. Mohit Gupta – 261715

UNDER THE GUIDANCE OF

Ms. Jinal Gujar

Submitted in partial fulfilment of the requirements for qualifying

BACHELOR OF SCIENCE (INFORMATION TECHNOLOGY)

Semester – V Examination

THAKUR COLLEGE OF SCIENCE AND COMMERCE

(Empowered Autonomous College, Permanently Affiliated to University of Mumbai)

THAKUR VILLAGE, KANDIVALI (EAST), MUMBAI – 400 101, MAHARASHTRA

ACADEMIC YEAR 2026 – 2027

Thakur Educational Trust's (Regd.)
THAKUR COLLEGE OF SCIENCE & COMMERCE
Empowered Autonomous College Permanently Affiliated to University of Mumbai
(NAAC Accredited with Grade "A" (3rd Cycle) & ISO 21001:2018 Certified)
Best College Award by University of Mumbai for the Year 2018-2019

CERTIFICATE

This is to certify that the project titled "**JustCmul8 - Browser Based Discrete Event Simulator**" undertaken at the "Thakur College of Science and Commerce" by Khushi Singh - 261713 and Mohit Gupta - 261715, in partial fulfillment of B.Sc. IT degree Examination, has not been submitted for any other examination and does not form part of any other course undergone by the candidates.

It is further certified that the students have completed all required phases of the project.

External Examiner	Internal Examiner
Project Guide	Head of Department

Thakur College of Science and Commerce,
Thakur Village, Kandivali(E),
Mumbai-400101

CANDIDATES' DECLARATION

We declare that the project described in this report, "**Browser Based Discrete – Event Simulator**", is our own work. It was done during the academic year 2026 – 2027 under the guidance of Ms. Jinal Gujar and is submitted to the Department of Information Technology, Thakur College of Science and Commerce, in partial fulfilment of the Bachelor of Science in Information Technology (Semester–V) degree.

The analysis, design, source code, integration of the simulation engine, testing and write-up were all done by the two of us. Where we drew on published literature, official technical documentation or open-source libraries, we have said so in the text and listed the source under References. None of this material has been submitted, wholly or partly, to another university or institute for any degree or diploma.

We accept that any lapse in academic integrity, whenever it is found, can lead to disciplinary action under the rules of the college and the University of Mumbai.

Place: Mumbai

Date: _____

Ms. Khushi Singh	**Mr. Mohit Gupta**
Roll No. 261713	Roll No. 261715

ACKNOWLEDGEMENT

Many people helped this project along and we would like to thank them here.

Our first thanks go to our guide, **Ms. Jinal Gujar** of the Department of Information Technology, who kept us on course through the whole development of JustCmul8. She insisted that the simulation engine be checked before any feature was stacked on it. We had planned it the other way round and following her advice spared us a lot of re-work. She also read our early drafts and the shape of this report came out of her comments.

Prof. Dr. Santosh Kumar Singh, Head of the Department of Information Technology, made it possible for students to take on a project of this size and his suggestions in the review sessions were useful. We are grateful to him.

We thank the Principal and management of Thakur College of Science and Commerce for the laboratory, network and machines we used. Testing a WebAssembly Python runtime and trying real-time collaboration on several computers at once, needed the department laboratory.

The faculty of the department taught us data structures, database systems, software engineering and web technologies and we used that material directly: priority queues, normalisation and the life-cycle models all appear in the work below. The laboratory assistants also deserve thanks for sorting out machine access and configuration during long testing sessions.

Ms. Khushi Singh – 261713

Mr. Mohit Gupta – 261715

ABSTRACT

A few are the number of people who use DES, a powerful demonstration tool, in order to see how a queue, a production line or a delivery network will behave before beginning to spend money to change it. For example, the commercial packages, like AnyLogic, Arena and Simul8, cost thousands each year for a licence. The so-far main open-source alternative is the Python library SimPy, but before a first run of a model, a user has to write and debug their own code. Almost all of these tools have to be downloaded and installed, an issue in shared labs and managed machines. Drawing tools like these, such as Visio and Lucid chart, work vice versa: they're easy to use, but they don't execute their diagrams, nor tell you what will happen if one extra counter is opened.

JustCmul8 is our attempt towards filling that gap. No downloading or installation required and nodes are dragged to a canvas, connected and each node is completed with normal form fills and the model is run. It is designed in such a way that the model is never approximated in JavaScript. The graph is consolidated to a true Python SimPy script, executed by a CPython interpreter that was ported to WebAssembly (Pyodide) and passed in as a Web Worker, so there's no freeze in the interface. These numbers are accordingly built from the same hardware that would be implemented in a terminal.

The front end is a single-page application implemented in Next.js 16, React 19 and TypeScript and the canvas is based on React Flow and Zustand state management library. Supabase manages sign-in, storage and real-time channels and PostgreSQL Row-Level Security does projects at the database level – without having to rely on client code. The twelve working modules contain fifteen node types and span six domains: human queues, vehicle traffic and the liquid and material flow, manufacturing, logistics and network signalling. The analytics layer calculates utilisation, waiting times, throughput, queue lengths, consistency with Little's law and a composite system health score after a run and it identifies the name of the bottleneck node. A Monte Carlo facility is a facility that takes a run several times on independent replications and provides 95 per cent intervals instead of just one. These are surrounded by an animated digital-twin playback, a cost and return-on-investment panel, a Python code inspector with Jupyter export and other executive PDF reports and revokable share links. A Google Gemini assistant accessed via a model router that observes model health and defaults automatically suggests a message in the form of a candidate graph and makes suggestions to improve this model's performance.

INDEX

1. INTRODUCTION.....1
1.1 Background and Motivation 1
1.2 Objective and Scope of the Project.....5
1.3 Problem Definition.....8
1.4 Limitations 10
2. Requirement Analysis.....11
2.1 User Requirements / SRS / Functional Requirements 11
2.2 Feasibility Study 16
2.3 Details of Hardware and Software Used.....19
2.4 Proposed System and Advantages of the Proposed System 22
3. PROJECT PLANNING.....24
3.1 List of Activities / Tasks.....24
3.2 Timeline for Each Task.....27
3.3 Gantt Chart and Activity Network Diagram.....29
3.4 Resource Planning 33
3.5 Proposed Software Development Model 35
3.6 Project Management Tool: Jira.....37
4. SOFTWARE MODULES 38
4.1 Module 1: Platform and Data Layer 39
4.2 Module 2: Visual Graph Editor.....41
4.3 Module 3: Node Configuration Panel.....44
4.4 Module 4: Simulation Engine 45
4.5 Module 5: AI Assistant.....46
4.6 Module 6: AI System Optimiser.....47
4.7 Module 7: KPI Dashboard and Results.....48
4.8 Module 8: Monte Carlo and Scenario Comparison 50
4.9 Module 9: Digital Twin Animated Playback.....51
4.10 Module 10: Simulation Template Gallery 52
4.11 Module 11: Executive Report and Share 53
4.12 Module 12: Real-Time Collaboration.....54
4.13 Hardware Modules and Considerations.....55
4.14 Individual Work Justification 56
5. SYSTEM ANALYSIS AND DESIGN.....57
5.1 Context Diagram.....57

5.2 DFD, ERD, Class Diagram and State Transition Diagram....59
5.3 Use Case Diagram....64
5.4 Architecture Design....68
5.5 Activity, Component and Collaboration Diagrams....70
5.6 Table Design....74
5.7 Architecture / Deployment Diagram....78

6. SYSTEM IMPLEMENTATION AND METHODOLOGY....80
6.1 Development Methodology....80
6.2 Technologies Used....81
6.3 Algorithms and Techniques Used....82
6.4 Module-wise Implementation....85
6.5 Database Implementation....86
6.6 User Interface Design....87
6.7 Important Code Snippets....91

7. SYSTEM TESTING....94
7.1 Testing Strategy....94
7.2 Testable Units....96
7.3 Test Case Design with Reference to the Project....97
7.4 Test Approach....102
7.5 Test Results and Validation....103

8. RESULTS AND DISCUSSION....105
8.1 System Output....105
8.2 Result Analysis....114
8.3 Performance Evaluation....117

9. COST AND BENEFIT ANALYSIS AND SOFTWARE PARAMETER ESTIMATION....119
9.1 Effort Estimation....119
9.2 KSLOC Analysis....122
9.3 Function Point Analysis....124
9.4 Comparison of Estimates and Cost–Benefit Analysis....127

10. CONCLUSION AND FUTURE SCOPE....132
10.1 Achievements of the Project....132
10.2 Conclusion....134
10.3 Future Scope....135

REFERENCES AND BIBLIOGRAPHY....137

LIST OF FIGURES

Figure 1.1: Functional Modules of JustCmul8 grouped by area 6
Figure 2.1: Execution Pipeline of the Proposed System 22
Figure 3.1: Weekly Gantt Chart of the JustCmul8 Development Schedule (July W1, 2026 – March W4, 2027) 29
Figure 3.2: Activity Network Diagram of the JustCmul8 Development Lifecycle 31
Figure 3.3: Iterative and Incremental Development Model adopted for JustCmul8 35
Figure 4.1: Module Structure of JustCmul8 38
Figure 5.1: Context Diagram (DFD Level 0) for JustCmul8 57
Figure 5.2: Level-1 Data Flow Diagram for JustCmul8 59
Figure 5.4: Entity Relationship Diagram for the JustCmul8 Database 60
Figure 5.5: Class Diagram of the JustCmul8 Object Model 61
Figure 5.6: State Transition Diagram for a Node During Simulation Execution 62
Figure 5.7: Use Case Diagram for JustCmul8 64
Figure 5.8: Architecture Design of JustCmul8 68
Figure 5.9: Activity Diagram of the End-to-End User Workflow 70
Figure 5.10: Component Diagram of the JustCmul8 Implementation 71
Figure 5.11: Collaboration Diagram for the Simulation Run Interaction 72
Figure 5.12: Deployment Architecture of JustCmul8 78
Figure 6.1: Landing Page 87
Figure 6.2: Registration and Login Screens 88
Figure 6.3: Project Dashboard 88
Figure 6.4: Workspace with the Visual Graph Editor 89
Figure 6.5: Node Configuration Panel 89
Figure 6.6: Template Gallery 90
Figure 7.1: Layered Testing Strategy for JustCmul8 94
Figure 8.1: Executive Summary Tab of the Results Dashboard 105
Figure 8.2: Flow and Balance Check (Little's Law Verification) 106
Figure 8.3: Traffic Heatmap 106
Figure 8.4: Line Buildup Timeline 107
Figure 8.5: Station Counters 107
Figure 8.6: Person / Item Journey (Entity Tracer) 108
Figure 8.7: Cost and Return-on-Investment Panel 108
Figure 8.8: Monte Carlo Analysis with 95% Confidence Bands 109
Figure 8.9: Digital Twin Animated Playback 109
Figure 8.10: AI Assistant Generating a Model from a Description 110
Figure 8.11: AI System Optimiser Recommendation 110
Figure 8.12: Python Code Inspector 111
Figure 8.13: Real-Time Presence and Live Cursors 111
Figure 8.14: Export, Share and Executive PDF Report 112
Figure 8.15: Read-Only Shared View 112

Figure 8.16: Profile and Settings..... 113
Figure 8.17: Simulated Results against Queueing Theory (M/M/1 and M/M/3 benchmarks)..... 114
Figure 8.18: Deviation of Simulated Metrics from Theory against Accepted Tolerance 115

LIST OF TABLES

Table 1.1: Comparison of Discrete Event and Continuous Simulation 2
Table 1.2: Probability Distributions Implemented in JustCmul8 2
Table 2.1: User Classes and Access Levels 11
Table 2.2: Functional Requirements 13
Table 2.3: Non-Functional Requirements 14
Table 2.4: Indicative Cost Comparison with Commercial Alternatives 17
Table 2.5: Development Hardware and Software Configuration 19
Table 2.6: Minimum and Recommended Client Requirements 20
Table 2.7: Technology Stack with Justification 20
Table 2.8: Comparison of the Proposed System with Existing Approaches 22
Table 3.1: List of Activities and Tasks with Dependencies 25
Table 3.2: Planned Timeline of Each Task (July W1 – December W4, 2026) 27
Table 3.3: Project Milestones and Completion Criteria 28
Table 3.4: Human Resources and Responsibilities 33
Table 3.5: Hardware, Software and Service Resources 33
Table 3.6: Planned Effort Allocation by Phase 34
Table 3.7: Phases of the Adopted Life-Cycle Model 36
Table 3.8: Why the Iterative and Incremental Model Was Preferred 36
Table 4.1: Module Overview and Requirement Coverage 38
Table 4.2: The Fifteen Node Types Supported by the Simulation Engine 41
Table 4.3: The Six Simulation Domains and Their Vocabularies 42
Table 4.4: Key Performance Indicators and Their Computation 48
Table 4.5: Hardware Roles in the Deployed System 55
Table 4.6: Division of Work between Team Members 56
Table 5.1: Use Case Specification: Run Simulation 65
Table 5.2: Use Case Specification: Generate Model via AI 66
Table 5.3: Use Case Specification: Share Project Read-Only 66
Table 5.4: Use Case Specification: Collaborate in Real Time 67
Table 5.5: Table: auth.users (managed by Supabase Auth) 74
Table 5.6: Table: public.profiles 74
Table 5.7: Table: public.projects 74
Table 5.8: Table: public.simulation_runs 75
Table 5.9: Table: public.chat_history 75
Table 5.10: Table: public.project_shares 76
Table 5.11: Row-Level Security Policies 76
Table 6.1: Complexity of Graph Operations on the Adjacency List 82
Table 6.2: Module-wise Implementation Summary 85
Table 7.1: Testable Units and the Techniques Applied to Them 96
Table 7.2: Engine Verification Test Cases and Results 97
Table 7.3: Invariants Asserted for the Complex Model (Benchmark B6) 98

Table 7.4: Functional Test Case Design 99
Table 7.5: Edge-Case Probes and Observed Behaviour 103
Table 7.6: Requirements Traceability 104
Table 8.1: Worked Example: M/M/1 Queue at ρ = 0.8 (λ = 0.8, μ = 1.0, 36,000 s, 5 replications) 115
Table 8.2: Results for the Multi-Stage Hospital-Flow Model (7,200 s, 3 seeds) 115
Table 8.3: Engine Throughput Measured by the Verification Harness 117
Table 8.4: Browser Performance Measurements (to be recorded) 117
Table 9.1: COCOMO Coefficients by Development Mode 119
Table 9.2: COCOMO Estimation Results (basic model, written from scratch) 119
Table 9.3: COCOMO Estimate Adjusted for Agentic Development 120
Table 9.4: Source Lines of Code by Layer 122
Table 9.5: Measured Project Size 122
Table 9.6: Standard IFPUG Complexity Weights 124
Table 9.7: Function Point Count by Component Type 124
Table 9.8: General System Characteristics and Degrees of Influence 125
Table 9.9: FPA-Based Cost Estimate by Rate Tier 126
Table 9.10: Comparison of Estimates against Planned Effort 127
Table 9.11: Role-Based Personnel Cost (Real IT Sector) 128
Table 9.12: Infrastructure and Operational Costs (Monthly and Annual) 128
Table 9.13: Total Project Cost, Real IT Sector against Student Project 129
Table 9.14: Summary of Cost Estimates from All Methods 129
Table 9.15: Benefits of the System 130

LIST OF ABBREVIATIONS

Term	Meaning	Term	Meaning
AI	Artificial Intelligence	NFR	Non-Functional Requirement
API	Application Programming Interface	OAuth	Open Authorization
COCOMO	Constructive Cost Model	PK / FK	Primary Key / Foreign Key
CRUD	Create, Read, Update, Delete	PM	Person-Months
CSV	Comma-Separated Values	PRNG	Pseudo-Random Number Generator
DES	Discrete Event Simulation	Pyodide	CPython distribution for WebAssembly
DFD	Data Flow Diagram	RBAC	Role-Based Access Control
ERD	Entity Relationship Diagram	RLS	Row-Level Security
FIFO / LIFO	First / Last In, First Out	SDLC	Software Development Life Cycle
FP / FPA	Function Point / Analysis	SimPy	Simulation in Python (DES library)
FR	Functional Requirement	SLOC / KSLOC	(Thousand) Source Lines of Code
HCI	Human–Computer Interaction	SPA	Single-Page Application
JSON / JSONB	JavaScript Object Notation / Binary	SRS	Software Requirements Specification
KPI	Key Performance Indicator	TLS	Transport Layer Security
LLM	Large Language Model	UAT	User Acceptance Testing
M/M/1, M/M/c	Markovian queues with 1 or c servers	UFP / VAF	Unadjusted FP / Value Adjustment Factor
ML	Machine Learning	UML	Unified Modeling Language
WASM	WebAssembly	UUID	Universally Unique Identifier

1. INTRODUCTION

What if we change this is a question that sooner or later arises on the lips of a bank branch, a factory floor, a hospital ward and a courier company. Testing one counter, or a new order for workstations, or a completely re-routed fleet, is an expensive experiment – particularly if you go wrong. Discrete Event Simulation (DES) provides the ability to get answers from a computer before any of the money goes. What has always been problematic is that it required a $100 license, programming and having to be installed locally. JustCmul8 was founded to eliminate them all in one go. The chapter describes the rationale for its construction, the goal in mind, the issues that underlie it and its limits.

1.1 Background and Motivation

1.1.1 Simulation as Decision Making Tool

Real operations subject to randomness and competition for common resources. It is not always evenly-distributed that customers arrive. The machine isn't the same length everywhere. Often two entities want the same server at the same time. All averages cover up this: if people come in 'clumps' to a counter which requires 4 minutes per transaction an average, the counter can accumulate a long queue of customers. A simulation generates a number of service times and arrival times, all random and then observes what happens to the entire (random) bunch that has been fed into the system; the person making the decision sees the likely outcome – and how variable it is likely to be. It is applied in production, medicine, logistics, transport and telecommunications as well as public services. The great thing about it is that an experiment that may wreak havoc, pose risk or take a ton of money on site, is now inexpensive and repeatable in the pool. A 3rd triage nurse can be tested under no one being inconvenienced at a hospital.

1.1.2 Theoretical Background

Several established topics from computer science, operations research and software engineering went into the project. The following ones have been used directly in the design and the code. Two forms of simulation are mentioned: Discrete Event Simulation and Continuous Simulation. A DES models a system as an occurrence of events in separate moments of simulated time. 2 events is no different than 1, so the clock doesn't tick between 2 events, but just jumps from event to event. If a customer arrives, another machine has

completed a job and a packet is delivered to a router, they are all events. In Continuous Simulation, on the other hand, variables with smooth changes are simulated and summed by small, fixed-sized steps. The DES does not touch the time intervals where no events occur; thus, a whole working day can be emulated in a fraction of a second.

Table 1.1: Comparison of Discrete Event and Continuous Simulation

Aspect	Discrete Event Simulation	Continuous Simulation
Time advance	Jumps to the next scheduled event	Advances in small fixed increments
State change	Instantaneous, at event instants only	Smooth and continuous throughout
Mathematical basis	Probability and queueing theory	Differential and difference equations
Typical application	Queues, production lines, supply chains	Thermal, hydraulic and control systems
Computational cost	Proportional to the number of events	Proportional to the simulated duration
Role in JustCmul8	Primary execution paradigm	Approximated by container-level sampling

Probability distributions. The model is parameterized using the values from probability distributions of arrival gap and service time to make the parameter's values change as they do in the real life. There are five samplers, all of which have different uses.

Table 1.2: Probability Distributions Implemented in JustCmul8

Distribution	Parameters	Characteristics	Typical use
Exponential	mean ($1/\lambda$)	Memoryless; short intervals frequent, long ones rare	Inter-arrival times of independent arrivals
Uniform	min, max (mean ± 50%)	All values in the range equally likely	A duration known only to lie between two bounds
Normal	mean, std. deviation	Symmetric about the mean; truncated at zero	Machine cycle times with small random error
Poisson	λ (mean rate)	Arrival counts in a fixed interval; exponential gaps	Arrivals per period
Deterministic	fixed value	No variability; repeats exactly	Conveyor transfers and scheduled operations

Queueing theory: The formula for simple queues is known to the simulator and can thus be compared with the formula on which it is based. For an M/M/1 queue, the utilisation is $\rho = \lambda/\mu$, the mean wait in the queue is $Wq = \rho/(\mu - \lambda)$ and the mean amount of people waiting is $Lq = \rho^2/(1 - \rho)$. For those quantities, Erlang-C is used in a multi-server queues. Little's law, $L = \lambda W$, is a mathematical fact that relates the average number of entities handled by a system to the average time that any particular entity spends in the system and is true for almost every stable queue. The results are used twice: for verification of the engine during development and during run time to test if each simulation is self-consistent or not.

Monte Carlo estimation: Each different instance of the random process in which a draw may be made is a stochastic run. Monte Carlo analysis is performed many times using different streams of random data and assumes the outcome is a sample. The 95 per cent confidence interval of a metric is the sample mean plus or minus 1.96 standard errors, where the standard error is the sample standard deviation divided by the square root of the number of replications.

Supporting concepts: Software Development Life Cycle (SDLC) divides the job into Analysis, Design, Implementation, Testing and Deployment. The architecture is multi-layered, separating the presentation from the state, separation of the domain logic from the state, separation of the execution from the storage, these layers can be tested independently from one another. So graph traversal is verifying if a model is OK – which is a directed graph. The relational database theory ensures user data security through primary and foreign keys and referential integrity as well as row level security. Namely, ideas around human–computer interaction enable the tool to be easy to use for all people (regardless of whether or not they are programmers): Ideas of naming the objects themselves with plain labels, the little mental load and familiar drag and drop etc.

1.1.3 Motivation

This was done with three reasons being in mind. Students, small businesses and most colleges can't afford to buy commercial simulators where they could test the decisions – and they would benefit the most. I tried using the best free engine, SimPy, but it is a code library, meaning that if the user isn't familiar with the queues, then he can't use it for generator functions. WebAssembly is now running at close to native speed on browsers and for the first time a real Python simulation engine can run on the user's device, without any server, no install. These combined provided us the brief – Make a platform that is free, doesn't require any minimum

operating requirements such as a body of simulator software, rather the words of the user's own profession and with a verifiable engine underneath. How people work today is what inspired the creation of the AI assistant and live collaboration, to describe what they want in normal language and add a workspace and collaborate with colleagues.

1.2 Objective and Scope of the Project

1.2.1 Objective

The primary goal of JustCmul8 is to develop and share a free, no-code DES that can be used by students, researchers, start-ups and companies to create, run, analyse and share simulation models within a Web browser as if they were software that does not require an installation, learning a programming language or paying a license fee.

Users are provided with a drag-and-drop modelling environment that is built on top of a real-time Python-based SimPy engine, running via WebAssembly as a typical web application. A machine-learning-based assistant converts a description in the English language to a model that is then implemented as a computer program, while an interactive dashboard provides the results and a computer-generated animation demonstrates how the system operates, using a digital twin.

The specific objectives are these.

- **Eliminate the need for cost and installation barrier** – no licence fee and there is nothing to install other than an up-to-date browser.
- **Ensure simulation is an accurate simulation:** use realistic Python SimPy engine, rather than a simulation in visual form; check the simulation against analytical queueing theory. Here, remove the coding prerequisite: provide a graphical user interface that a non-coders individual can use.
- **Record uncertainty:** provide confidence intervals, not one number on the off-hand.
- **Find the bottleneck** automatically, verify with Little's law, quantify overall system health and recommend changes, along with corresponding projected benefits and costs.
- **Facilitate collaboration and sharing:** live presence and shared cursors in a share workspace, read-only shared cursor links that can be canceled.
- **Support Growth:** Design the system to be extensible to create new domains, node types and engines in accordance with the registry without re-engineering the base system.

1.2.2 Scope of the Project

JustCmul8 highlights the design and development of browser based application for building, running, analysing and sharing DES models. These are divided into four groups of 12 functioning modules, that in total cover the entire system from account registration to exporting the final result.

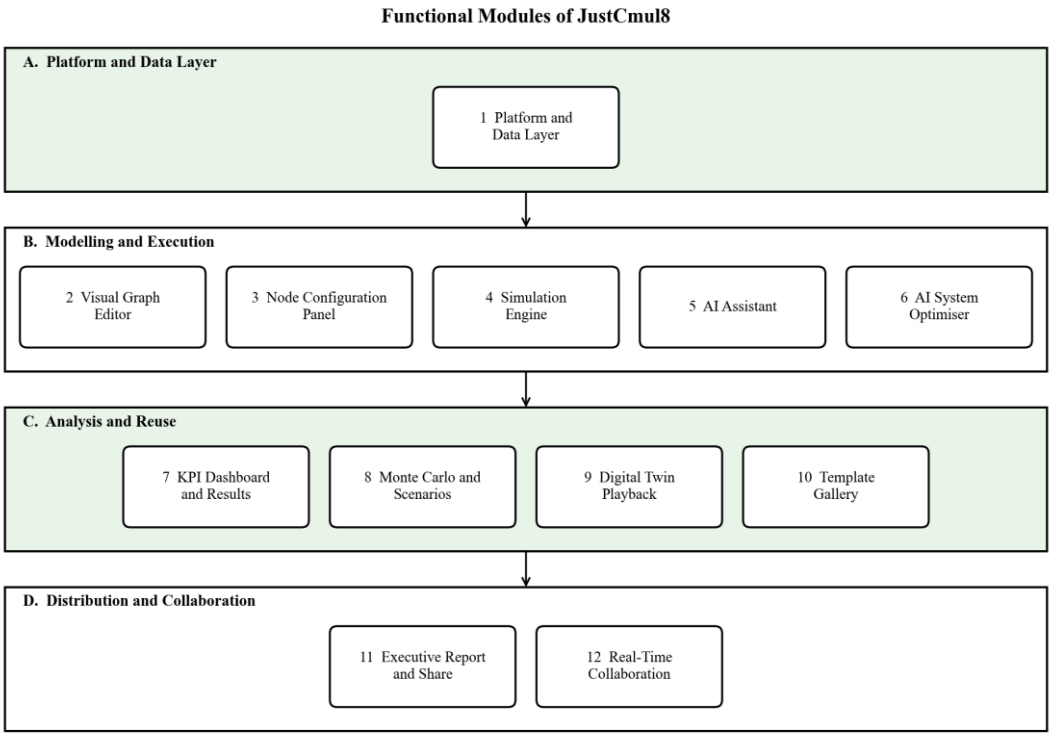


Figure 1.1: Functional Modules of JustCmul8 grouped by area

- **Platform and data layer** (Module 1): Register and log in with email or via Google/GitHub OAuth, creation, renaming and deletion of projects, handling of sessions with automatic saving to the cloud, profile and preference management.
- **The visual graph editor and the node configuration panel** (Modules 2 and 3), the **SimPy engine, hosted in Pyodide** (Modules 2 to 5), the AI assistant that can create a model from natural language and the AI optimiser that runs on a finished model and suggests a fix (Modules 2 to 6).
- **Analysis and reuse:** KPI dashboard with bottleneck detection and cost analysis, Monte Carlo and scenario comparison, animated digital-twin playback and template gallery (Modules 7 – 10).
- **Distribution and collaboration:** graph export as JSON, results as CSV, free public share links and real-time presence with collaborative editing (Modules 11 and 12) and executive PDF reports..

Supported: human queue systems, vehicle traffic, liquid and material flow, manufacturing processes, logistics and warehousing, network and signal flow. All six are implemented in one simulation engine and a library of 15 node types can be configured for a wide variety of scenarios.

1.2.3 Out of Scope

Certain abilities have not been included by design in this release in order for the project to be manageable in the academic programme. We list them to help avoid the wrong conclusion that this is just a short list we've forgotten about. Some of these are more frequently referenced in the roadmap of chapter 10.

- **Native mobile applications:** interface responds well and functions in mobile versions of browsers, though no dedicated Android or iOS app.
- **Enterprise services:** Single sign-on, LDAP integration, organisation-level administration and subscription billing.
- **Learning-management features:** the platform is not courseware – it is considered a simulation tool.
- **Offline and self-hosted deployment:** the application itself is deployed as a cloud application that is accessible through a web application. Industrial hardware integration – no direct tie to IoT device, PLC, sensors, robots.
- **Optimisation:** Person enters a change to be optimised and the AI optimiser tells the person what change to make; No AI searching the parameter space. In line with a browser-first approach, there isn't a packaged Windows, macOS or Linux build of the desktop version.

1.3 Problem Definition

For manufacturing, health, logistics, transport and services, DES is one of the best known ways of enhancing manufacturing, with very few people using it outside of large organisations. It's practical, rather than theoretical, reasons and they are divided in 4 groups.

1.3.1 The Cost of Commercial Simulation Software

AnyLogic, Arena and Simul8 are all thousands of dollars per seat per year. A department of 60 students enrolled in an operation research course, or the restaurant owner deciding whether or not to install a second service counter, might have an answer that costs more than the question. Consequently those that might afford a hit can have it, while those that cannot do not have it.

1.3.2 The Technical Complexity of Free Alternatives

There are also free frameworks, such as SimPy, which are programming libraries and not yet fully developed applications. A user is required to install Python, learn how to use a virtual environment, understand what the request for accessing a resource must contain and understand how to correct errors that occur after sending a request. For part of the simulation ideas, part of the Python idioms, needs to be picked up simultaneously—just a lot to ask for someone who is wondering whether to open another counter.

1.3.3 The Limits of Desktop Distribution

There is simulation software that requires installation, configuration and maintenance. People have no right to install anything on others' Laptop or on a Chromebook in a shared college laboratory, or on a company-managed laptop. This is no matter how sophisticated and expensive the tool is or how skilled the person is, a tool that cannot be opened is useless.

1.3.4 The Inadequacy of Static Diagrams and Spreadsheets

Many people will have made two try's and each try's will have failed for the same reason. The maps made with Visio and Lucidchart are descriptive and dry. The only thing not shown in the diagram is the number of the items displayed by the inspector by mid afternoon and this is not attached to a box called “inspection: 5 min”. However, a sheet of paper doesn't compute and there is no natural way of representing an event queue, or the set of entities that are competing for a resource, or a server that is still processing the next entity entering the resource queue. Neither puts less pressure on the other; and that’s the pressure that makes queues LineUp.

1.3.5 Statement of the Problem

Taken together, these barriers make a technique that may reliably answer routine questions about operations within an organisation that holds a licence and has trained staff, extremely inefficient if not impossible.

1.4 Limitations

All designs are compromises. The following are the restrictions of this release. Each of these stems from a conscious architectural decision made and each is related to an entry in the roadmap of Section 10.3. Client side resources bound model size: the model size and length of a simulation are limited by the memory that the browser can free up for the simulation to be run on the client's device. It is not a speed restriction, but rather a memory restriction.

Elapsed time grows linearly with the number of replications, because Monte Carlo replications are run sequentially (e.g., one at a time, using a single worker); their execution times are independent of each other.

Fist run runtime download: The Pyodide runtime is not downloaded the first time Pyodesk is run and is stored when downloaded afterwards and thus simulation runs in a session take longer to begin. All need connectivity for the cloud: saving, collaboration and AI help. When “loading” a model that is already loaded in, it can still be run independently, but no memory is stored. Project-scoped: Presence and graph operations are sent over a private channel agreed to by the project's row-level security. Editor and viewer permissions through role-based assignments are coming. • In need of human judgment: generated graphs and recommendations are checked and presented for approval; there is no way to ensure they accurately model the described process. One simulation paradigm: the engine is SimPy based, behaviour continuous or agent base, approximating (the container node samples the level on the boundaries of the events). • No native mobile means that the interface is responsive, however it's still cumbersome to place nodes with precision on a touchscreen.

2. Requirement Analysis

The outlines delineated within this chapter are the "how" of the system and the "who. It starts with the Software Requirements Specification, identifies the types of user and outlines the types of functional and non-functional software requirements, item by item in the following chapters, providing an identifier by which each of them can be referred to in the following chapters. It then considers the level of feasibility, the hardware and software implemented to achieve this and then proposes a system along with its advantages.

2.1 User Requirements / SRS / Functional Requirements

2.1.1 Purpose and Overall Description

Every assessment task serves a purpose and has a general description. JustCmul8 is a desktop, tablet, laptop or phone simulation platform, which is accessed via an ordinary browser. It consists of four levels: a User-Interface layer, an Application and Domain-logic layer, a Client-side Execution layer and a Managed Database layer. There is nothing to be installed outside of the browser. The heaviest load is on the client that hosts the interface, the application state and all of the simulation work. Only stores data and authentication, pass-through for real-time and forwards requests to the AI service.

2.1.2 User Classes and Characteristics

Table 2.1: User Classes and Access Levels

User class	Responsibilities	Access level	Skill required
Registered User (Project Owner)	Creates projects, builds and configures models, runs simulations, uses AI, exports and shares results	Full control of owned projects	None (no-code)
Collaborator	Joins a project’s live workspace to view presence, cursors and shared edits	Project-scoped, authorised by row-level security	None
Guest / Public Viewer	Opens a public share link to view a model read-only	Read only; no account required	None

As a result of No Code, none of the classes requires any programming skills. Guest role read-only so that a model can be given for commenting without any chance that something will be accidentally changed.

2.1.3 Operating Environment

- **Supported browsers:** a modern browser (Google Chrome, Mozilla Firefox, Microsoft Edge or Safari) with WebAssembly support on Windows, macOS, Linux android or iOS. This interface adjusts to tablets, desktops and smartphones.
- A supabase-hosted Next server runtime runs the pages and a minimal number of API routes, providing support for production-grade PostgreSQL, authentication, real-time channels and storage.
- **Slowdown due to simulative reloading:** SimPy runs in the browser in a Web Worker making minimal use of the server's CPU.

2.1.4 Design and Implementation Constraints

- All functionality needed to run in the browser (no installation required on the client-side).
- The security aspect of the database; all were evaluated on a row level, meaning when a user accesses the database they'll only have access to their own or project(s) that are expressly granted to them.
- The interface needed to be easy and fast to use, as its users had little or no experience in programming.
- The architecture needed to be modular and domain and node type driven, accepting new domains and node types without redesigning the bottom layer.
- The most important thing was that secrets (specifically the AI service key) didn't need to be included in the client bundle and didn't need to be placed on the server.

2.1.5 Functional Requirements

The functional requirements are presented in modules. They are repeated in the test cases in Chapter 7.

Table 2.2: Functional Requirements

ID	Module	Requirement
FR-01	Platform	Users register and log in via email, password or practically the same via Google or GitHub OAuth and they can log out securely.
FR-02	Platform	The step of signing in is carried forward from one visit to the next and projects are checked first to see if they are protected.
FR-03	Platform	If a user forgets the password, he/she is sent an e-mail with a link and can opt for a new password.
FR-04	Platform	Users who are signed in are able to create, open, rename and delete simulation projects, selecting one of the six chosen domains for each project.
FR-05	Platform	Project graph state will automatically be written to cloud storage – there is no need to press a Save button.
FR-06	Platform	A dashboard that displays the user's projects, with domain and last modified date, plus profile/s, preferences and account management such as deleting the account.
FR-07	Graph Editor	Fifteen node types are supported on the canvas which can be placed and dragged, connected and deleted.
FR-08	Graph Editor	The nodes follow the edges when moved, the canvas can be zoomed, panned and presented to the viewer for a best fit in the window.
FR-09	Graph Editor	When the graph is not amenable to running, the names of the graph and the offending node are displayed before a run.
FR-10	Node Configuration	Only the parameters suitable for that node are displayed in a side panel, which is used to edit the parameters for that selected node (arrival rate, capacity, service-time distribution, routing and costs).
FR-11	Node Configuration	The values of one or more parameters are tested prior to using values by the engine with the node type.
FR-12	Simulation Engine	There's a Python SimPy script created based on the graph, which is run on the client (which is implemented with Pyodide). The server performs no simulation work.
FR-13	Simulation Engine	The user assigns simulated duration and its associated time unit before a run in order to: seconds, minutes, hours and days.
FR-14	Simulation Engine	A run is executed inside a Web Worker, thus avoiding blocking the UI.
FR-15	Simulation Engine	The user can view progress and status of the run and a stream of log events while the run is in progress, can stop the run and can adjust to various playback speeds from 0.5× to 10×.
FR-16	AI Assistant	Natural language based scenario is transformed to a candidate graph of nodes and edges.
FR-17	AI Assistant	A graph generated is first checked for its validity, then the user can view it and implement it. In addition, a chat history is maintained for each project which will check out the other AI model in the event that the initial one fails.

ID	Module	Requirement
FR-18	AI Optimiser	Once a run is complete, the optimiser identifies the problem and proposes a fix to be applied to the run in one operation
FR-19	KPI Dashboard	Upon a run finish, the utilisation, average and percentile waiting times, throughput and queue lengths at each node will be assigned.
FR-20	KPI Dashboard	The dashboard indicates the critical node of the wind tunnel, displays Little's law results and displays a composite health score of the system from 0 to 100.
FR-21	KPI Dashboard	The resource, holding and waiting-penalty cost parameters are used to construct a Cost and ROI analysis.
FR-22	Monte Carlo	A model can be used for 10, 20, 50 or 100 replications and reported with a 95% confidence interval for that replications - with a baseline compared with a modified scenario.
FR-23	Digital Twin	The entities are lit up, moving and are 2-D in a playback with play and pause buttons, step and speed controls.
FR-24	Template Gallery	Starter templates include all 6 domains and will have each create their own project under whoever's name they request to be created in.
FR-25	Export and Share	The graph is exported as JSON and results as CSV, the created Python script is written in a Jupyter notebook and an executive report is written as a PDF.
FR-26	Export and Share	Read only public share link can be created and revoked. Either doesn't grant access to edit, or, if the link has been revoked or is invalid, the person clearly notices this.
FR-27	Collaboration	Users can see each other, including cursors, within the same project; the operations to be performed on the graph are streamed via an authenticated private channel.

2.1.6 Non-Functional Requirements

Table 2.3: Non-Functional Requirements

ID	Category	Requirement
NFR-01	Performance	Moving nodes around on the canvas should happen immediately and there shouldn't be any lag when the node is moved and connected to the canvas nor a lag when the dashboard displays a finished run.
NFR-02	Responsiveness	Running simulation should never stop the interface; runs are queued up and run in a background worker thread.
NFR-03	Correctness	For reference queues, the simulation should be utilized and simulation waiting time and queue length should closely follow the analytical queuing theory, within a specified tolerance.
NFR-04	Security	Only salted hashes like these are stored as domain passwords and all traffic is TLS; secret keys will never be sent to the client.
NFR-05	Access Control	ROW-LEVEL SECURITY is visible for each of the tables and cannot be circumvented by a client modified to get around the authorisation.

ID	Category	Requirement
NFR-06	Usability	The user interface should be operational by a first time, non-technical user and is based around the notion of domain language, not simulation jargon.
NFR-07	Reliability	Project State will automatically stay between sessions and between devices.
NFR-08	Portability	It will function unchanged across the majority of current web browsers for Windows, macOS, Linux android and iOS which support WebAssembly.
NFR-09	Maintainability	The logic of a simulation remains internal to the worker and domain / node definitions remain in a single repository.
NFR-10	Data Integrity	The database is a foreign-key constraint database that cascades delete.
NFR-11	Availability	The availableifted app has a goal of 99% availability for both the front end and the managed back end.

2.1.7 User Personas and Operational Workflow

The interface was designed utilizing three personas. They progress in different ways around the system.

- **The student** is introduced to the concept of queueing theory in a lecture and wants to observe the M/M/1 behaviour rather than derive it. This individual opens a template, edits the arrival rate to be different, views the result and then compares dashboards. Here, the important thing is that, a useful first result is present within a minute of the arrival.
- **The lecturer** shows in class how congestion can occur. The model is illustrated with an animated playback and by providing a share link, students can open and examine copies without having an account later on their own devices.
- **The small-business** owner is considering installing another service counter. This person is able to give the AI assistant the description of the shop in plain language, accepts the generated graph, performs a Monte Carlo and reads the confidence intervals and reads the cost panel before they spend any money on the shop.

All three have a similar workflow: log in, create a project or open an existing project, build the graph – either manually or with the assistance, then set the node parameters, validate the structure, run the graph, read the analytics, repeat this step, optionally, with different scenario and then export or share the resulting graph. This is done in Chapter 5, when we represent this path as an activity diagram.

2.2 Feasibility Study

Prior to the establishment of the design recommendation, we explored the project on five levels. Technical Feasibility is the most important requirement; if the technical part failed to work, so would the whole architecture.

2.2.1 Technical Feasibility

Next.js, React, TypeScript, Supabase, React Flow, Pyodide and SimPy are all popular, well-documented and highly supported. This reduces risks of development and facilitates long term maintenance costs.

The main challenge was to get:

a) a python discrete event simulation running in a browser,

b) that it performs quickly enough.

The primary questions were how to get:

a) a python discrete event simulation running in a browser

b) how fast exactly.

They both meet Pyodide's head-on, by compiling the CPython interpreter into WebAssembly code, meaning that SimPy is a Python pod installed on the client. With a prototype spike, we proved to be that as everything else relied on that. Out of that "peaker" we were left with two constraints which we designed with.

The Pyodide runtime is a few megabytes and is downloaded once and stored. WebAssembly is also run inside the browser's limit on how big its memory can be, so large models are a practical limit when used in a Web Worker; also, using Web Assembly ensures that a large model is not hindering the response of the user. Also, although not quite so much, there was a concern about Canvas rendering, as hundreds of nodes in the DOM can cause problems to render quickly.

We have created the editor with React Flow, which is designed for node based interfaces and the animated playback (when a lot of entities move together) with Pixi.js which draws given animated objects through WebGL.

2.2.2 Economic Feasibility

This project is low cost, as for the most part most of the components in the project are open source, or only available as a cloud-based service with very generous free tiers. None of the

Section 1.3 commercial packages is without licence fee, unlike most of the others listed above, which are completely free.

The speed and cost of simulations on the client are important. The server doesn't have to provide the computing, the user's browser does; so the number and size of simulations has no effect on the cost of the server.

The simulator can handle 1000 of them running a 1 hour simulation at the same time, without causing any extra load on the server as compared to 1000 people reading a regular page. The only recurring costs are the managed database tier and the metered AI quota and the latter is negligible as an AI request is only sent when the user requests for generation/diagnosis.

Table 2.4: Indicative Cost Comparison with Commercial Alternatives

Item	Commercial DES tool	JustCmul8
Software licence	Typically thousands of USD per seat per year	None; open-source stack
Installation and IT effort	Required on every machine	None; runs in the browser
Simulation compute	Local workstation or licensed server	User’s browser; no server cost
Backend hosting	Not applicable or self-managed	Managed free tier during the academic phase
AI assistance	Generally unavailable	Gemini API, metered per explicit request
Training	Formal instruction usually necessary	Not required; no-code interface

Commercial figures are only indicative as vendor price lists are subject to change. The difference is the ‘point’ of the table.

2.2.3 Operational Feasibility

It's simple to implement the platform. The drag and drop interface, combined with the AI assistant means that a person with no programming or simulation knowledge can create a model, there is no software to install, maintain or lose. The users of the project are from somewhat different groups - students, teachers, researchers and small-business owners - who are already familiar with modern web applications, so formal training and a technical support desk are not necessary. There are some user interface options which aid further. The vocabulary of the domain that is selected for the palettes is used – hence the word “Teller” for a person modelling a bank, or the word “Line Buildup Timeline” for a result tab.

2.2.4 Schedule Feasibility

A plan for development is in accordance with the academic time frame. The development has been broken up into modules, which are created in an order depending on their prerequisites, with the fundamental features built initially and everything else (support features like AI assistance, collaboration and so on) added afterwards. Testing, debugging and integration are not done after the development, but concurrently. The biggest impact decision was to have the scheduling of the simulation engine proceed concurrently with the work on the backend and to have that work begun in the second week of July, still early enough to react to it. The full schedule of the twenty-four weeks are shown in chapter 3.

2.2.5 Legal and Ethical Feasibility

The legal and ethical feasibility of the project.Legal and ethical feasibility of project. No licensing obligations are remaining as all dependencies are used under open-source licences or are used by free-listing tiers. PostgreSQL Row-Level Security restricts each user to access his/her own projects and passwords are never stored in a recoverable format. When users delete an account it does go through and delete their data. A public share link can be cancelled at any time, after which you can find an explanation on the link instead of the project. AI-generated charts are never just sent off to a person to effect any change to a project, but instead given guidance as to how to proceed.

2.3 Details of Hardware and Software Used

Shows details of Used Hardware & Software There are two different configurations for this and it's easy to mix them up. The one I'm using to build the platform is pretty strenuous since an editor and development server are on the same machine as the browser. The other is what an end user needs which is much less and it's this second one that will determine the extent to which the platform can be adopted.

2.3.1 Development Environment

Table 2.5: Development Hardware and Software Configuration

Component	Specification	Purpose
Processor	Intel Core i5 / AMD Ryzen 5 or higher	Runs the development server, browser and Pyodide runtime concurrently
Memory	8 – 16 GB RAM	Editor, browser with several tabs and local build processes together
Storage	256 GB SSD or larger	Sources, Node.js dependency tree and build artefacts
Display	1920 × 1080 or higher	Canvas, configuration panel and dashboard side by side
Operating system	Windows 11	Primary development platform
Network	Stable broadband connection	Package installation, Supabase and AI API access
Editor	Visual Studio Code with TypeScript and ESLint	Primary development environment
Version control	Git with a GitHub remote	Source history, branching and collaborative development
API testing	Postman	Verification of API routes
Runtime	Node.js LTS with npm	Dependency management and the Next.js build pipeline
Verification	Python 3 with SimPy	Native harness used to verify the engine against queueing theory
Browser tooling	Chrome DevTools	Profiling, memory inspection and performance measurement

2.3.2 Target Client Requirements

In a web application (on client side), no special hardware is required. Any decent desktop, laptop, tablet or smartphone that has a latter version of a browser will work. One consequence of this is that the architecture: Since the engine is not done on the server, the Web Worker path

is provided to keep the user interface usable in the case of very large models that will be running slower on a machine with less memory.

Table 2.6: Minimum and Recommended Client Requirements

Requirement	Minimum	Recommended
Browser	Current Chrome, Edge, Firefox or Safari with WebAssembly	Latest evergreen release of Chrome or Edge
Memory	4 GB RAM	8 GB RAM or more for large models
Processor	Dual-core	Quad-core or better
Display	1366 × 768	1920 × 1080 or higher
Network	Stable connection for loading and persistence	Broadband for real-time collaboration
Local installation	None	None

2.3.3 Software and Technology Stack

Table 2.7: Technology Stack with Justification

Layer	Technology	Reason for selection
Frontend framework	Next.js 16 (App Router)	File-based routing, server rendering and API routes in one framework
UI library	React 19	Component model suited to a complex, highly interactive editor
Language	TypeScript	Static typing across the code base, catching parameter mismatches at compile time
Styling	Tailwind CSS v4	Utility-first styling giving consistent spacing and a coherent theme
Graph editor	React Flow (@xyflow/react)	Production-grade node-based canvas with handles and edge routing
Graph layout	Dagre	Automatic layered layout for generated and template graphs
Simulation engine	Python SimPy via Pyodide (WebAssembly)	A real DES engine executing client-side
State management	Zustand	Lightweight store with slice-based separation
Charts	Recharts	Declarative, composable charts for the KPI dashboard
2D animation	Pixi.js	WebGL rendering for the digital-twin playback
AI service	Google Gemini (via a model router)	Structured generation of graphs and diagnoses from natural language

Layer	Technology	Reason for selection
Database and auth	Supabase (PostgreSQL, Auth, Realtime)	Managed Postgres with Row-Level Security, OAuth and WebSocket channels
Reporting	jsPDF, html-to-image	Client-side generation of executive PDF reports
Animation and scroll	Framer Motion, Lenis	Transition choreography and smooth scrolling on the landing page
Icons	Lucide React	Consistent icon set covering all node types

2.4 Proposed System and Advantages of the Proposed System

2.4.1 The Proposed System

The system we propose is JustCmul8 – a simulation platform based on discrete events to exist in the browser. The user is able to drag nodes onto a canvas in the protocol, to complete the node parameters on forms, to press Run and read the answer on the plain language. The graph is translated behind the button into a real SimPy program that is compiled to WebAssembly and then executed in a Web Worker, by CPython. Out of it comes a layer of analytics where performance metrics (power metrics, verification, ranking of constraint, cost) are calculated, approved and sorted, then recommendations, costs and an animated replay of the constraint.

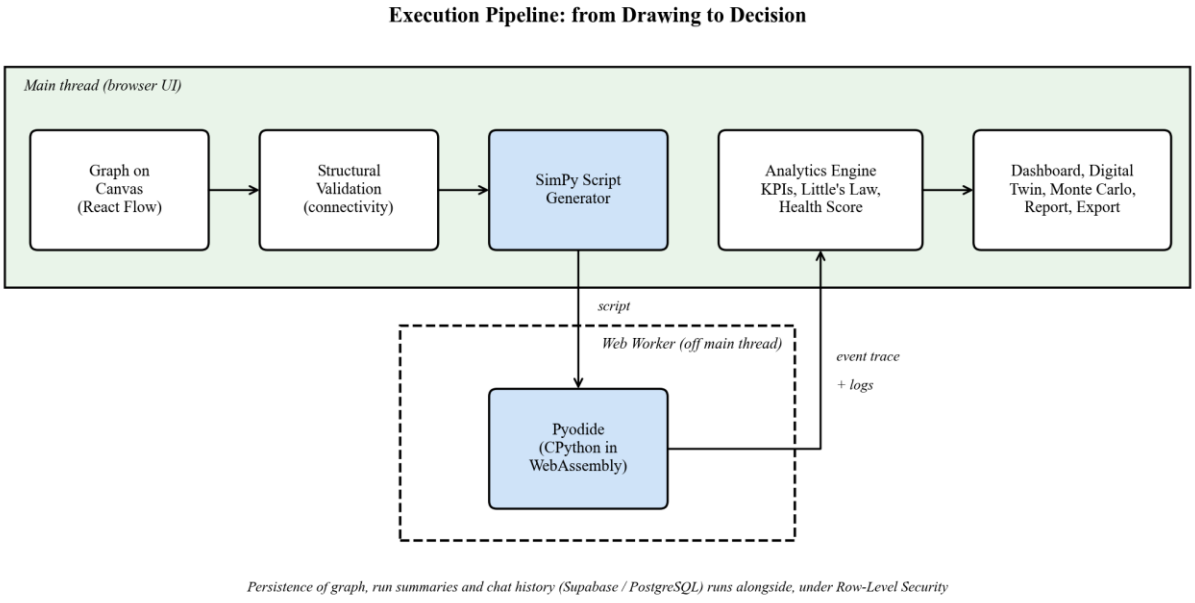


Figure 2.1: Execution Pipeline of the Proposed System

2.4.2 Comparison with Existing Approaches

Table 2.8: Comparison of the Proposed System with Existing Approaches

Capability	Commercial DES tools	SimPy library	Diagram tools / spreadsheets	JustCmul8
Free to use	No	Yes	Partly	Yes
No installation	No	No	Yes	Yes
No programming needed	Partly	No	Yes	Yes
Executes real DES	Yes	Yes	No	Yes (SimPy)
Runs in the browser	Rarely	No	Yes	Yes
AI-assisted modelling	Rarely	No	No	Yes

Capability	Commercial DES tools	SimPy library	Diagram tools / spreadsheets	JustCmul8
Confidence intervals (Monte Carlo)	Yes	Manual coding	No	Yes, one click
Live collaboration and sharing	Limited	No	Yes	Yes
Readable generated code	No	Yes	No	Yes (inspector and notebook)

2.4.3 Advantages of the Proposed System

The proposed system has the following advantages:

- **Free, no setup required:** it's free but only requires a "url" link from any laptop, lab machine and/or Chromebook. Reliable results – numbers are supplied by SimPy and not an imitation and engine validated with queueing theory.
- **No-code modelling:** domain vocabulary, form based configuration and a template gallery makes it possible for a first-user to get to a result in a minute.
- **Workflow assisted with AI:** simple text to a runnable model and a completed run to a concrete, costed recommendation.
- **Topics include:** Monte Carlo confidence intervals allowing for uncertainty in the answer; showing an apparent improvement is larger than the noise.
- **Transparent and repeatable:** Code generated in Python is readable in code inspector and output can be published as a jupyter notebook for any to be able to re-run a result independently.
- **Easy-to-understand output:** bottleneck flags, a system health score, a flow-balance check, a traffic heatmap, an entity tracer and an animated digital twin transform the numbers into pictures.
- **Security enforced by the database:** Access controls determined by the database and share links are opaque, read-only and revocable.
- **Useful to scale:** computation runs on the client side: it is not costly to the host depending on the usage rate.
- **Easy to extend:** Each domain's vocabulary, palette, layout and templates is defined in one registry and addition of new domains and node types does not affect the engine.

3. PROJECT PLANNING

Nothing big gets built in one go. We broke JustCmul8 into small steps that we could actually finish and this chapter shows how. It covers the list of activities, how long each one took, the Gantt chart and the activity network, the people and tools we needed, the life cycle model we picked and how we tracked the work in Jira.

3.1 List of Activities / Tasks

We laid the activities out in the usual SDLC order: initiation and analysis, design, development, testing, documentation and deployment. There are 27 activities in total. For each one we wrote down how long it would take and what it depends on. The plan runs for twenty-four weeks, from 1 July 2026 to the last week of December 2026.

Writing the dependencies down helped us in two ways. First, we didn't start a module before the things it needs were ready. Second, we could see which tasks don't depend on each other and run them side by side, which cut the total time without cutting corners. The biggest example is the simulation engine. It started in the second week of July and ran alongside the backend work. Running Python SimPy inside a browser through Pyodide and WebAssembly was the riskiest part of the whole project and the AI assistant, KPI dashboard, Monte Carlo and digital twin all depended on it working. Starting early gave us time to fix compatibility problems, check the numbers against known queueing results and settle the interface that the other modules would use.

A quick note on module numbers. The synopsis we submitted at the start listed eleven modules: Authentication, Project Management, Visual Graph Editor, Node Configuration Panel, Simulation Engine, AI Assistant, KPI Dashboard, Template Gallery, Export and Share, User Profile and Settings and Real-Time Collaboration. The activity list below follows that plan. While building, we reorganised the system into twelve modules, which you will see in Chapter 4.

Authentication, project management and profile turned into one Platform and Data Layer module. Three features outgrew their original place and became modules of their own: the AI System Optimiser, Monte Carlo and Scenario Comparison and the Digital Twin Playback. So the activity list shows the plan as we made it and Chapter 4 shows what we actually delivered.

Table 3.1: List of Activities and Tasks with Dependencies

Sr.	Task / Activity	Phase	Duration	Depends on
1	Project Initiation	Initiation / Analysis	2 weeks	None
2	Requirement Gathering	Initiation / Analysis	3 weeks	None
3	Literature Survey	Initiation / Analysis	2 weeks	Project Initiation
4	System Analysis	Initiation / Analysis	2 weeks	Requirement Gathering
5	UI/UX Design	Design	3 weeks	System Analysis
6	Database Design	Design	3 weeks	System Analysis
7	Module: Authentication	Development	3 weeks	Database Design
8	Module: Project Management	Development	3 weeks	Authentication
9	API Development	Development	5 weeks	Database Design
10	Backend Development	Development	8 weeks	API Development
11	Module: Visual Graph Editor	Development	4 weeks	UI/UX Design, Project Management
12	Module: Node Configuration Panel	Development	3 weeks	Visual Graph Editor
13	Frontend Development	Development	9 weeks	UI/UX Design
14	Module: Simulation Engine	Development	8 weeks	Early parallel track; runs ahead of backend
15	Module: AI Assistant	Development	4 weeks	Simulation Engine
16	Integration	Development	5 weeks	Modules 1 – 6 of the plan
17	Module: KPI Dashboard	Development	3 weeks	Simulation Engine
18	Module: Template Gallery	Development	3 weeks	Visual Graph Editor
19	Module: Export and Share	Development	3 weeks	KPI Dashboard
20	Module: User Profile and Settings	Development	3 weeks	Authentication
21	Module: Real-Time Collaboration	Development	4 weeks	Visual Graph Editor
22	Unit Testing	Testing	5 weeks	Modules in progress
23	Integration Testing	Testing	3 weeks	Integration complete
24	Bug Fixing	Testing	3 weeks	Integration Testing
25	User Acceptance Testing (UAT)	Testing	2 weeks	Bug Fixing
26	Documentation	Documentation	23 weeks	Continuous, alongside every module
27	Final Review and Deployment	Deployment	4 weeks	User Acceptance Testing

Here is how the phases worked out.

Initiation and Analysis. Before writing any code we decided the objectives, the target users and the requirements. We also looked at the commercial and open-source tools from Section 1.3 and put our findings into a specification.

Design. The database schema is covered in Chapter 5. In this phase we also decided the workspace layout, how the node palette is organised and how the configuration panel behaves.

Development. This was the biggest phase and it ran in three parallel streams. The backend and API stream handled authentication, project management and the data layer. The simulation stream handled the engine, the AI assistant, the KPI dashboard and the templates. The frontend stream handled the graph editor, the configuration panel, export, profile and collaboration.

Testing. Unit testing started before the modules were finished, so bugs in the early modules got fixed while later ones were still being written. After integration testing came bug fixing and then user acceptance testing.

Documentation. We wrote it during 23 of the 24 weeks so that it kept pace with the work.

Deployment. A final review, then upload to the hosting platform.

3.2 Timeline for Each Task

Table 3.2 gives the planned start and end week of each activity. W1 means the first week of that month.

Table 3.2: Planned Timeline of Each Task (July W1 – December W4, 2026)

Sr.	Task / Activity	Duration	Start	End
1	Project Initiation	2 weeks	Jul W1	Jul W2
2	Requirement Gathering	3 weeks	Jul W1	Jul W3
3	Literature Survey	2 weeks	Jul W2	Jul W3
4	System Analysis	2 weeks	Jul W3	Jul W4
5	UI/UX Design	3 weeks	Jul W4	Aug W2
6	Database Design	3 weeks	Jul W4	Aug W2
7	Module: Authentication	3 weeks	Aug W1	Aug W3
8	Module: Project Management	3 weeks	Aug W2	Aug W4
9	API Development	5 weeks	Aug W1	Sep W1
10	Backend Development	8 weeks	Aug W1	Sep W4
11	Module: Visual Graph Editor	4 weeks	Aug W2	Sep W1
12	Module: Node Configuration Panel	3 weeks	Aug W3	Sep W1
13	Frontend Development	9 weeks	Aug W2	Oct W2
14	Module: Simulation Engine	8 weeks	Jul W2	Sep W1
15	Module: AI Assistant	4 weeks	Sep W2	Oct W1
16	Integration	5 weeks	Sep W3	Oct W3
17	Module: KPI Dashboard	3 weeks	Sep W3	Oct W1
18	Module: Template Gallery	3 weeks	Sep W4	Oct W2
19	Module: Export and Share	3 weeks	Oct W1	Oct W3
20	Module: User Profile and Settings	3 weeks	Oct W1	Oct W3
21	Module: Real-Time Collaboration	4 weeks	Oct W2	Nov W1
22	Unit Testing	5 weeks	Sep W4	Oct W4
23	Integration Testing	3 weeks	Oct W4	Nov W2
24	Bug Fixing	3 weeks	Nov W1	Nov W3
25	User Acceptance Testing (UAT)	2 weeks	Nov W3	Nov W4
26	Documentation	23 weeks	Jul W1	Dec W3
27a	Final Review	2 weeks	Dec W1	Dec W2
27b	Deployment	2 weeks	Dec W3	Dec W4

Project Milestones

Table 3.3: Project Milestones and Completion Criteria

Milestone	Target	Completion criterion
M1: Requirements baselined	Jul W3	Functional and non-functional requirements approved by the guide
M2: Literature survey complete	Jul W3	Comparable tools evaluated and the technology stack fixed
M3: Design frozen	Aug W2	Interface layout and database schema approved
M4: Engine proof of concept	Sep W1	A SimPy model executing in the browser with output verified against theory
M5: Core modelling path working	Sep W1	A graph can be built, configured and executed end to end
M6: Analytics operational	Oct W1	KPI dashboard rendering utilisation, wait time and throughput
M7: Feature complete	Nov W1	All scheduled modules implemented and integrated
M8: Testing signed off	Nov W4	Unit, integration and acceptance testing passed
M9: Deployed	Dec W4	Application live on the hosting platform and report submitted

M4 is the milestone that matters most. We placed it early on purpose, because it removes the biggest technical risk and every milestone after it assumes that Python can run in the browser.

3.3 Gantt Chart and Activity Network Diagram

3.3.1 Gantt Chart

A Gantt chart shows every activity as a horizontal bar on a calendar. You can read a task's length straight off the bar and see which tasks overlap. Software teams use it to plan work, hand out tasks and track progress. It is useful because it cuts a project into pieces with realistic time frames and shows clashes early enough to do something about them.

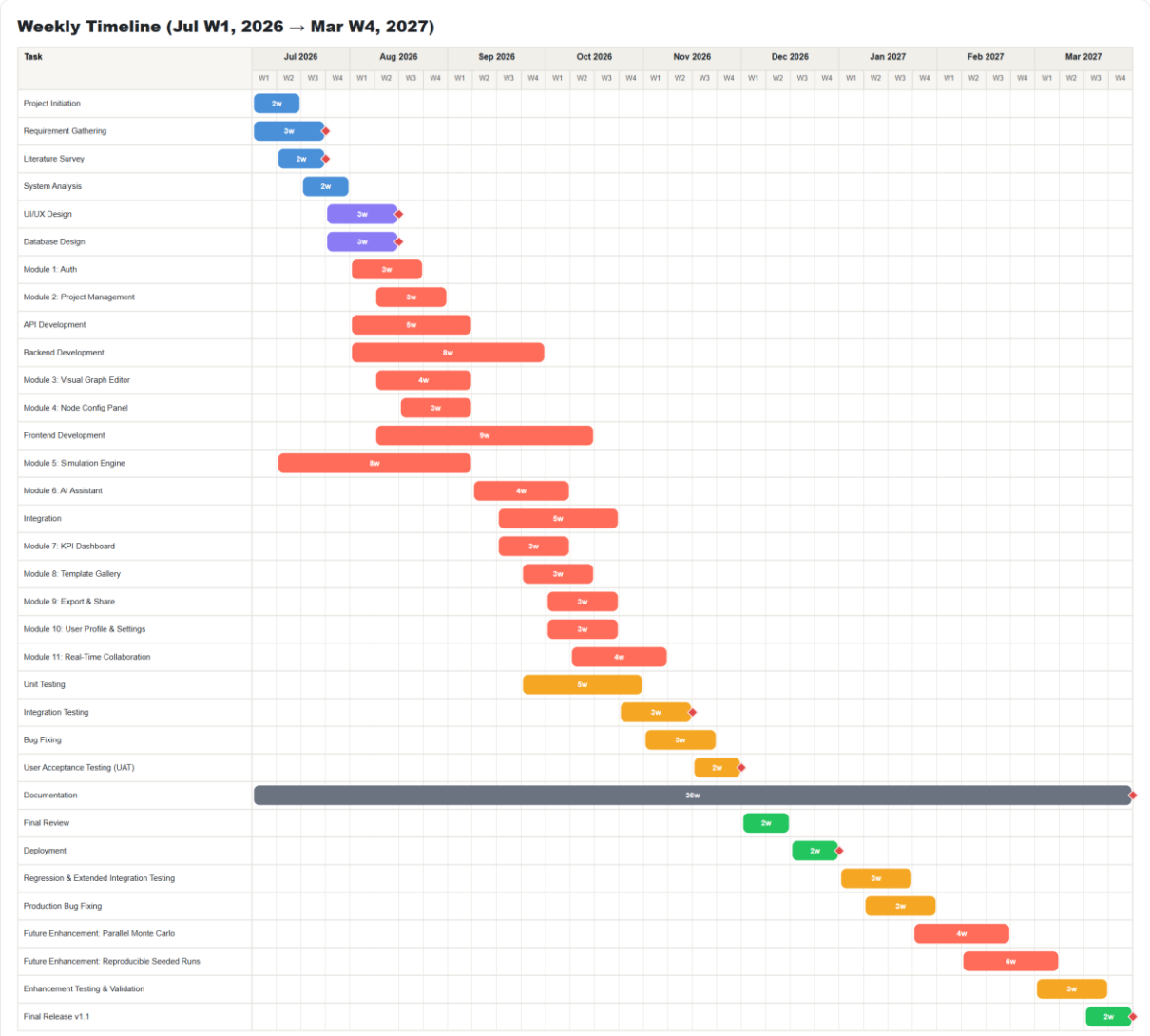


Figure 3.1: Weekly Gantt Chart of the JustCmul8 Development Schedule (July W1, 2026 – March W4, 2027)

The bars are grouped by phase and each phase has its own shade. The first four bars are initiation and analysis, which fill up July. Next come interface design and database design, about three weeks each, starting in late July. Most of the chart is development. The two

foundation modules come first, then the API and backend, then the visual graph editor and node configuration panel, which is where the user does most of the work.

The simulation engine bar needs a closer look. It starts in the second week of July and overlaps with the backend, for the reason given in Section 3.1. The AI module begins once the engine has settled and the remaining modules land through September and October, each after its prerequisite. The testing bars overlap the development bars on purpose, since we tested modules as they came together. Documentation is one long bar across nearly the whole period and the two bars after it are the final review and deployment in December, which close the plan as originally scoped. The chart then carries a three-month extension into 2027: a regression and bug-fixing buffer in January, followed by two future-scope items from Section 10.3 (parallel Monte Carlo and reproducible seeded runs) in January and February, a short validation pass in March and a final v1.1 release in the last week of March.

3.3.2 Activity Network Diagram

An activity network diagram shows the flow of work through a project: the order of the activities, the decision points and the parallel paths. A Gantt chart tells you when a task starts and ends, while the network tells you which task comes after which, what can run in parallel and where a decision is needed. We drew ours in UML notation using activity nodes, control flow, decision nodes, fork and join bars and start and end markers.

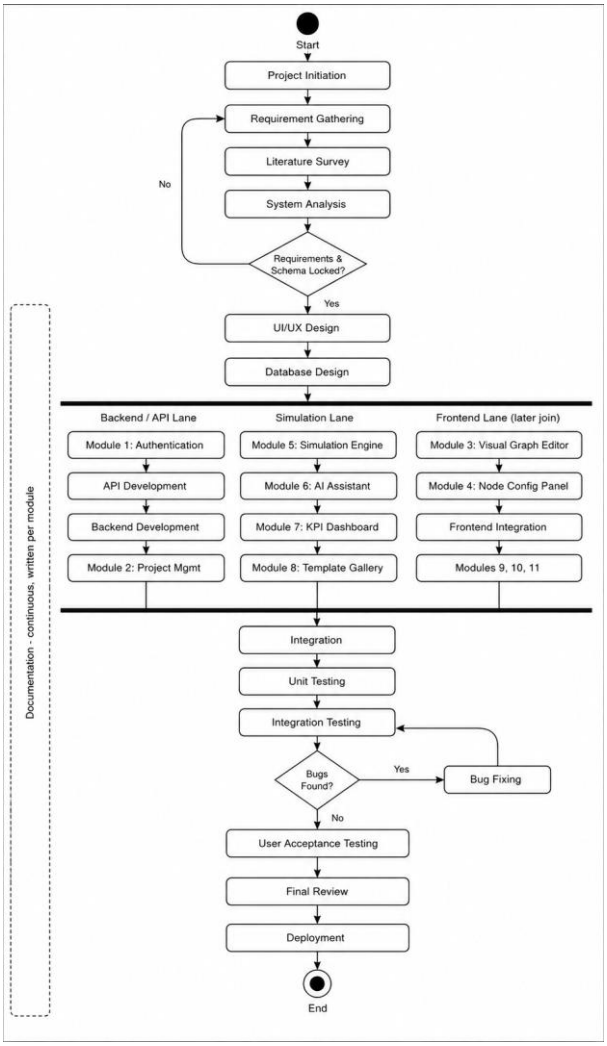


Figure 3.2: Activity Network Diagram of the JustCmul8 Development Lifecycle

The flow starts with project initiation, requirement gathering, literature review and system analysis. A decision node then asks whether the requirements and the database schema are locked. If not, we go back to requirements and design only begins once both are confirmed. Development splits into three parallel lanes. The backend and API lane has authentication, API development, backend development and project management. The simulation lane has the engine followed by the AI assistant, KPI dashboard and template gallery. The frontend lane has the graph editor, configuration panel, frontend integration, export, profile and collaboration. A join bar brings the three lanes back together for system integration. After that come unit tests and integration tests and then another decision node checks for defects. If there are any, the flow goes back to bug fixing and integration testing and it keeps looping until the build is clean. Only then do we move to user acceptance and the final review and deployment. Documentation runs alongside every phase.

3.3.3 Critical Path

The critical path is the longest chain of tasks, the one that decides the earliest date the project can finish. For us it runs through Requirement Gathering, System Analysis, Database Design, API and Backend Development, Integration, Integration Testing, Bug Fixing, UAT, Final Review and Deployment. If anything on that chain slips, the delivery date slips with it. Tasks off the chain, like the Template Gallery and Profile and Settings, have some float and can absorb a delay. Together the two diagrams give the calendar and the working method behind JustCmul8.

3.4 Resource Planning

Resource planning means working out which people, equipment, software and services we need and when. Most of it was about people's time, since there are only two of us. For everything else we used an open-source stack and free-tier cloud services.

3.4.1 Human Resources

Table 3.4: Human Resources and Responsibilities

Resource	Role	Responsibilities
Ms. Jinal Gujar	Project Guide	Review of requirements, design and drafts; validation of milestones; academic guidance
Khushi Singh (261713)	Developer, Analyst and Documenter	Requirements and design documents; Platform and Data Layer; configuration panel; AI assistant; KPI dashboard; digital twin; report and share; testing support
Mohit Gupta (261715)	Developer, Architect and Tester	System architecture; graph editor; simulation engine; AI optimiser; Monte Carlo; templates; real-time collaboration; verification harness; deployment
Both members	Team	Requirement analysis, database design, integration, testing, user acceptance and the project report

3.4.2 Hardware, Software and Service Resources

Table 3.5: Hardware, Software and Service Resources

Category	Resource	Cost basis
Hardware	Two development laptops (8–16 GB RAM, SSD); college laboratory machines for cross-machine collaboration testing	Owned / provided by the college
Development software	VS Code, Git, GitHub, Node.js, Postman, Python and SimPy, Chrome DevTools	Free
Frameworks and libraries	Next.js, React, TypeScript, Tailwind CSS, React Flow, Zustand, Recharts, Pixi.js, Pyodide	Open source
Backend services	Supabase (database, authentication, realtime)	Free tier
AI service	Google Gemini API	Free tier / metered usage
Hosting	Vercel (Next.js hosting and preview deployments)	Free tier
Project management	Jira (Scrum board, backlog and bug tracking), linked with GitHub for code and pull requests	Free plan

3.4.3 Effort Allocation

One person-month is one person working for one month, so two people over six months gives a nominal 12 person-months. Table 3.6 splits that effort across the phases, following the schedule in Section 3.2.

Table 3.6: Planned Effort Allocation by Phase

Phase	Weeks	Share of effort	Principal activity
Initiation and Analysis	4	10%	Requirements, literature survey, technology assessment
Design	3	10%	Interface design, database design, architecture
Development	18	45%	Twelve modules, API and backend, integration
Testing	9	20%	Unit, integration, security, acceptance testing
Documentation	continuous	10%	Module documents and this report
Deployment	4	5%	Final review, configuration, release

3.5 Proposed Software Development Model

We followed an iterative and incremental model, which is a mix of waterfall and agile. Two things led us there. First, the requirements were fairly steady from the start. The problem area is well understood and the tools we reviewed in Section 1.3 already showed most of what a simulator needs to do. Pure agile makes sense when requirements keep changing during development and that was not our case. Waterfall's heavy paperwork also didn't fit an academic project of this size. Second, there was one big technical risk. At the start we didn't know if SimPy would run in a browser through Pyodide and a lot of other modules depended on the answer. A strict waterfall would have made us wait until implementation to find out, by which time a wrong assumption would already be baked into the design. So we built an early prototype of the engine to test that assumption before building the modules that needed it.

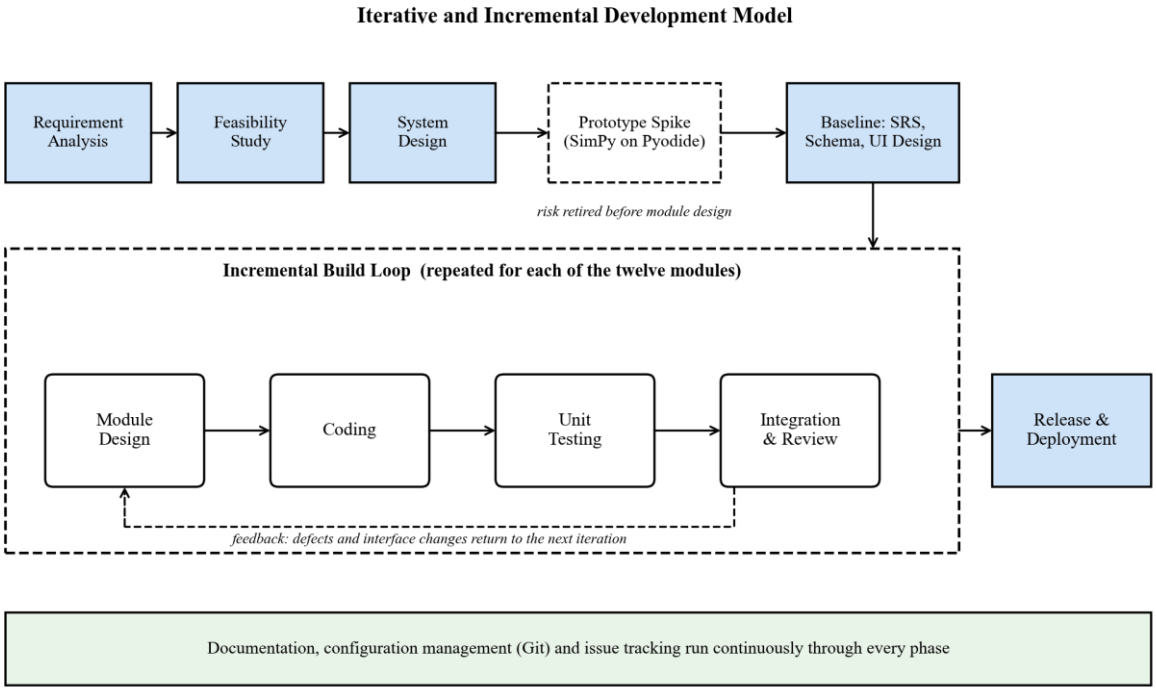


Figure 3.3: Iterative and Incremental Development Model adopted for JustCmul8

In practice, analysis and design went the waterfall way. The requirement specification and the schema were finished before any workflow code was written. Implementation then went module by module and each module went through design, coding, testing and integration before the next one started. Testing ran the whole time and documentation was written for each module as we went.

Table 3.7: Phases of the Adopted Life-Cycle Model

Phase	Activities performed	Outcome
Requirement analysis	Collected the needs, studied similar tools, listed the functional and non-functional requirements	Approved requirement specification (Section 2.1)
Feasibility study	Checked technical, economic, operational, schedule, legal and ethical viability	Confirmed that the browser-hosted approach could work
System design	Architecture, database schema, interface layout and UML modelling	Design documents and diagrams (Chapter 5)
Prototyping (iterative)	A spike of the Pyodide and SimPy execution path, checked against analytical results	Biggest technical risk cleared at milestone M4
Implementation (incremental)	Module-by-module build across three parallel streams, each with its own design, code and test cycle	Twelve delivered modules (Chapter 4)
Testing	Unit, structural, analytical, integration, security and acceptance testing alongside development	Verified system (Chapter 7)
Deployment	Final review, environment setup, database policies and release	Live application on the hosting platform
Documentation	Writing down analysis, design, implementation and test results as we went	This report and the module documents

Table 3.8: Why the Iterative and Incremental Model Was Preferred

Model	Strength	Weakness for this project
Waterfall	Clear stages and documentation	Finds out about the engine risk only late in implementation
Pure Agile / Scrum	Copes well with changing requirements	Our requirements were stable and we needed formal documentation
Spiral	Risk analysis in every loop	Too heavy for a two-person team
Iterative and Incremental (adopted)	Risk cleared early, testable increments, documentation kept in order	Needs stable interfaces between modules, which the worker boundary gives us

One thing about this model is worth recording. Some modules use the output of the engine, so the format of that output changed as those modules came along. Monte Carlo needed statistics for each run and the digital twin needed a journey trace for each entity. Both were cheap to add because the worker boundary keeps the engine separate from the rest, which is exactly why we used a layered design.

3.6 Project Management Tool: Jira

For tracking the work we used Jira. Two people can keep a small project in their heads, but 27 activities and twelve modules spread over three parallel streams is a lot and we wanted one place where everything was written down. Jira gave us that on its free plan. We also connected it to our GitHub repository, so commits and pull requests could be tied back to the task they belong to.

3.6.1 How We Set It Up

- **Project and board:** one Scrum project for JustCmul8, with a board of four columns: To Do, In Progress, In Review and Done.
- **Epics:** every one of the twelve modules is an epic. The smaller pieces of work sit inside the epic as stories and tasks, for example “Build node palette” under the Visual Graph Editor epic.
- **Sprints:** we worked in two-week sprints, which gives roughly twelve sprints over the twenty-four weeks. At the start of a sprint we picked what we could finish from the backlog and split it between the two of us.
- **Issue types and priority:** Story, Task and Bug, each with a priority from Highest to Lowest, so the important things were never buried under small ones.
- **Reviews:** a ticket only moved to Done after the other member had looked at the work, which matches the code review rule in Section 6.1.
- **Bugs:** every defect found while testing went in as a Bug ticket with the steps to reproduce it. It was fixed, tested again and linked to the commit that closed it. This is the defect handling described in Section 7.4.
- **Tracking progress:** the roadmap view gave us a timeline of the epics that matched the Gantt chart and the sprint burndown chart told us early on whether we were falling behind.
- **Meetings:** a short check-in on the board twice a week and a longer review at the end of every sprint, which we also used to update our guide.

3.6.2 Why Jira

We picked Jira for three reasons. It is free for small teams, it suits the sprint-by-sprint way of working from Section 3.5 and it is what most IT companies actually use, so working with it gave us a feel for how a real software team runs its projects.

4. SOFTWARE MODULES

JustCmul8 has twelve software modules. Each one has a single job, talks to the others through a defined interface and was built and tested on its own before we joined it to the rest. This chapter goes through them one at a time: what the module is, how we built it, which requirements from Section 2.1 it covers and who wrote it. At the end there is a short note on hardware, since the simulations run on the user's own machine.

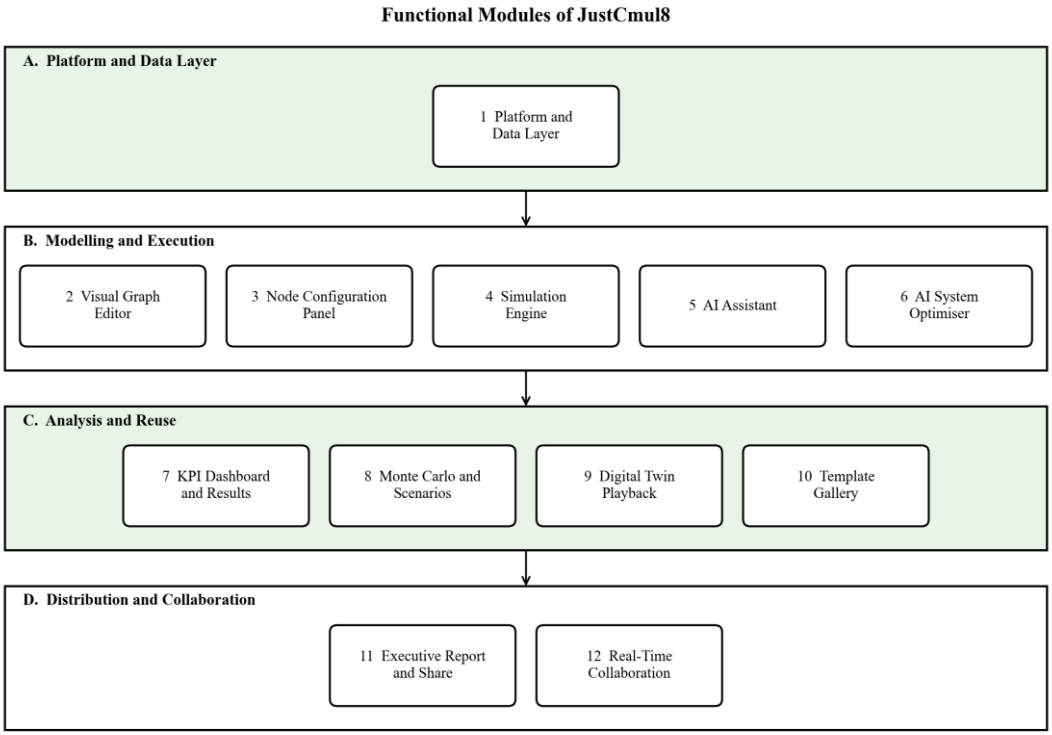


Figure 4.1: Module Structure of JustCmul8

Table 4.1: Module Overview and Requirement Coverage

No.	Module	Primary responsibility	Requirements
1	Platform and Data Layer	Authentication, project lifecycle, cloud persistence, profile	FR-01 to FR-06
2	Visual Graph Editor	Canvas-based construction of the simulation graph	FR-07 to FR-09
3	Node Configuration Panel	Parameter and cost entry with per-type validation	FR-10, FR-11
4	Simulation Engine	Code generation and in-browser SimPy execution	FR-12 to FR-15
5	AI Assistant	Natural-language generation of candidate models	FR-16, FR-17

No.	Module	Primary responsibility	Requirements
6	AI System Optimiser	Bottleneck diagnosis and one-click corrective fixes	FR-18
7	KPI Dashboard and Results	Metric computation, verification, visualisation, cost and ROI	FR-19 to FR-21
8	Monte Carlo and Scenario Comparison	Replicated runs, confidence intervals, baseline comparison	FR-22
9	Digital Twin Animated Playback	Animated 2D entity flow with transport controls	FR-23
10	Simulation Template Gallery	Reusable starter models across six domains	FR-24
11	Executive Report and Share	PDF, CSV, JSON, notebook export and share links	FR-25, FR-26
12	Real-Time Collaboration	Live presence, cursors and graph-operation broadcast	FR-27

4.1 Module 1: Platform and Data Layer

Purpose: This module decides who the user is, what they own and where their work gets saved. Everything else depends on it, because a project can't be created, shared or saved without somebody to own it. It sits on top of Supabase and handles login, project management, cloud storage and account settings in one place.

Functional responsibilities.

- Sign up and log in with email and password, or in one click with Google or GitHub.
- The login session carries over between visits and middleware blocks protected routes before any project data is fetched.
- Password reset through an emailed link, password change while signed in, secure sign-out and account deletion that removes everything the user owns.
- Creating a project in any of the six domains, optionally from a template and renaming or deleting projects.
- The graph saves to cloud storage by itself while the user edits, so there is no Save button to remember.
- A dashboard listing the projects with their domain and a settings page for display name, avatar and default domain.

Implementation. Supabase Auth handles sign-in, so the application never stores a plain-text password. Next.js middleware refreshes the session and guards the protected routes. One rule we stuck to is that authorisation doesn't live in the client. Every table has a PostgreSQL Row-Level Security policy that checks whether the logged-in user is the one asking. If not, nothing comes back, even if someone writes the request by hand. A profile row is created when a new user signs up and every child table cascades on delete, so deleting an account deletes the data with it.

Requirements covered	Principal source files	Developed by
FR-01 to FR-06	app/login, signup, forgot-password, update-password, auth/callback, dashboard, settings; middleware.ts; lib/supabase; api/account/delete; supabase/schema.sql	Khushi Singh (261713)

4.2 Module 2: Visual Graph Editor

Purpose. This is where the modelling happens and it is where most of the no-code promise is kept. The user draws a process as a directed graph by moving things around directly on a canvas.

Functional responsibilities.

- A canvas with drag &drop nodes and a palette to pick them from.
- Fifteen node types across six domains, each with its own icon, colour and connection handles.
- Directed edges can be created and deleted between node handles and they re-route on their own when a node is moved.
- Zoom, pan and fit to view, plus selecting and deleting nodes and edges. Before a run the graph is checked structurally, so a graph that can't run is stopped and the problem node is named.
- Automatic layout for generated and template graphs, using a strategy chosen for each domain.

Implementation. So React Flow will gives us the viewport transform, node placement, connections and edge routing. Writing that ourselves would have eaten a big part of the schedule. Each of the fifteen types has its own custom node component. Labels come from a domain registry, so someone modelling a bank sees "Teller" and someone modelling a factory sees "Machine" though both are the same resource node underneath. A single generic layout gave diagrams that were correct but hard to read, so the automatic layout uses Dagre with a strategy for each domain.

Requirements covered	Principal source files	Developed by
FR-07 to FR-09	components/workspace/NodeCanvas.tsx, NodePalette.tsx; lib/simulation/layoutEngine.ts, simTypeRegistry.ts	Mohit Gupta (261715)

Table 4.2: The Fifteen Node Types Supported by the Simulation Engine

Node type	Behaviour	Typical domains
source	Creates entities at gaps drawn from the chosen distribution	All six

Node type	Behaviour	Typical domains
queue	A waiting line with limited or unlimited room, in FIFO, LIFO or priority order	All six
resource	A server with limited capacity; entities take it, hold it for a sampled service time and then let it go	All six
service	One processing step that takes its own time	Manufacturing, logistics
decision	Sends each entity down one of several outgoing edges according to set probabilities	All six
sink	Ends an entity's journey and records its completion figures	All six
container	Holds a level of bulk material or liquid, sampled at event boundaries	Liquid / material flow
store	A buffer for separate items (SimPy Store, with priority and filter versions)	Manufacturing, logistics
event_trigger	Watches for a condition and fires an event when it becomes true	Network, manufacturing
priority_resource	A preemptive server: a higher-priority arrival pushes out a lower one	Human queues, healthcare
channel	A transmission medium that adds propagation delay	Network and signal flow
broadcaster	Copies an entity onto every outgoing edge (one-to-many)	Network and signal flow
any_of	Waits for several events at once and carries on when the first one finishes (SimPy AnyOf)	Network, manufacturing
all_of	Waits until every one of the events has finished (SimPy AllOf)	Manufacturing, logistics
interrupter	Cuts into a process that is running, to model breakdowns and people walking away	Manufacturing, human queues

The first six node types are the standard discrete event vocabulary and most models only need those. The other nine cover things the basic set can't express: continuous levels, typed buffers, preemption, propagation delay, fan-out and event synchronisation. The two synchronisers map straight onto AnyOf and AllOf in SimPy, so a generated script can wait on several events without any scheduling code from the user.

Table 4.3: The Six Simulation Domains and Their Vocabularies

Domain	Typical scenarios	Example node labels
Human Queues	Banks, clinics, ticket counters, reception desks	Teller, Counter, Waiting Area, Customer Arrival

Domain	Typical scenarios	Example node labels
Vehicle Traffic	Junctions, toll plazas, fuel stations, car washes	Approach, Signal, Toll Booth, Pump
Liquid / Material Flow	Tanks, pipelines, mixing and bottling plants	Reservoir, Pipe, Valve, Mixer
Manufacturing Lines	Assembly lines, machining cells, inspection bays	Machine, Buffer, Inspection, Assembly
Logistics / Warehousing	Depots, sorting hubs, order fulfilment	Inbound Dock, Picking, Packing, Dispatch
Network / Signal	Packet routing, message buses, sensor networks	Transmitter, Router, Link, Receiver

4.3 Module 3: Node Configuration Panel

Purpose. When a node is dropped on the canvas, it says what the node is. How it acts is determined by configuring it. This module collects the parameters that turn a bare diagram into a running model, without any scripts.

Functional responsibilities.

- A left side panel that opens to display the selected node and only lists the fields appropriate to that node type.
- Fields for arrival rate, service-time distribution and parameters, capacity, queue discipline, routing probabilities and propagation delay.
- The user can select from the 5 probability distributions and adjust the parameters accordingly.
- Cost fields (standardised hourly resource rate, holding cost and waiting penalty cost) used for the Cost and ROI analysis.
- All the values are validated against the type of each node and each modification is made directly in the associated model.

Implementation. A node's settings are represented by a JSON object where each key is a part of its settings and its value is the key's corresponding setting. In that way, when we add a new node type, the schema remains unchanged and we can validate each type in a diffierent manner. Validation runs when the value is typed in and again before a run. The messages name the node and specify what to do, because the idea was to provide a "no code" option, which means we wouldn't want to show the user a raw engine error.

Requirements covered	Principal source files	Developed by
FR-10, FR-11	components/workspace/NodePropertiesPanel.tsx; lib/simulation/types.ts	Khushi Singh (261713)

4.4 Module 4: Simulation Engine

Purpose. The engine is integral to the project, its technical core. It transforms a visual graph into discrete events and runs this event simulation in the browser on a real Python engine.

Functional responsibilities.

- Compile the node and edge specification into a Python SimPy script that matches the parameters set.
- Then execute it on the client with the help of the Pyodide WebAssembly runtime, without any simulation work done on the server.
- Run on a Web Worker to keep the interface responsive.
- Take the simulated duration and the time unit (seconds, minutes, hours or days) before a run and stream state snapshots and log events to the interface during the run.
- A real Python code inspector which displays the generated code and saves it as a Jupyter notebook.

Implementation. There are 4 stages of a run. First the graph is checked on a structural level. It is then traversed by the code generator, which generates a SimPy script, in which all the nodes are resources, stores and processes and all the edges are routing decisions. The script is passed to the worker where Pyodide runs it and the event trace is returned to the main thread for analysis. All the resources are matched by the corresponding SimPy primitive: Resource, PriorityResource, Store, PriorityStore, FilterStore or Container.

Requirements covered	Principal source files	Developed by
FR-12 to FR-15	lib/simulation/codeGenerator.ts, pyodideEngine.ts, pyodideWorker.ts, clientEngine.ts, jupyterExport.ts; components/workspace/CodeInspectorPanel.tsx	Mohit Gupta (261715)

4.5 Module 5: AI Assistant

Purpose. This module is not about putting technical obstacles in your way, it is about removing a mental one. If a person doesn't know how to draw a process model, they can just write it down in normal English and get back a first version of the model.

Functional responsibilities.

- Transform a narrative situation into a simple graph of nodes and edges with parameters.
- Before presenting it, check the generated graph (valid node types, no dangling edges).
- Show a preview and wait for an explicit accept, so that the live project does not change without the user agreeing.
- Respond to questions about simulation concepts and about the user's own model in basic language and store the chat history by project.
- Switch to another AI model automatically if the preferred AI model is down.

Implementation. The request is processed by the Next.js API routes, which then route to Google Gemini, keeping the API key on the Next.js server and not in the client bundle. The system prompt lists the node types with their parameter schemas and includes the current graph. Keeping it in the prompt in this way holds the model to graphs that the engine can run. A model router maintains a priority list of models, classifies each failure as "unavailable", "rate-limited" or "empty response", marks a failed model as unhealthy for a period of time and subsequently bypasses it on later requests. The graph is validated against the graph rules before it is displayed and arranged by the layout engine.

Requirements covered	Principal source files	Developed by
FR-16, FR-17	components/workspace/AIChatPanel.tsx; app/api/ai/chat and generate; lib/ai/systemPrompt.ts, graphOps.ts, modelRouter.ts, textFormat.ts	Khushi Singh (261713)

4.6 Module 6: AI System Optimiser

Purpose. In Module 5, the user can get help creating a model. This one helps them to do something with it. Once a run has produced the computed metrics, it explains which node is slowing the system down and why and suggests one change that can be applied in a single action.

Functional responsibilities.

- After running, examine the utilisation, waiting time and flow of each node.
- Identify in words the constraining node, in a way a non-specialist can act on.
- Propose one concrete change, for example more capacity at a resource, rerouting at a decision node or a larger buffer.
- A one-click Apply Fix, performed as an ordinary graph edit.
- Keep the graph from before the change, so that its effect can be compared in Module 8.

Implementation. The optimiser is based on the results of the analytics and not on the raw event trace, so its recommendations are based on computed utilisation and queue numbers rather than guesses. First, a flow-analysis step determines which node it actually makes sense to add capacity to, since capacity added to a node that is not limiting the flow has no effect. Only that candidate is passed to the language model, which explains it. The proposal is always presented for approval and the optimiser never makes decisions on its own.

Requirements covered	Principal source files	Developed by
FR-18	app/api/ai/optimize/route.ts; lib/ai/flowAnalysis.ts, graphOps.ts	Mohit Gupta (261715)

4.7 Module 7: KPI Dashboard and Results

Purpose. An event trace is not an answer. This module converts the raw results of a run into the numbers a decision must rely on and presents them so that the node that sets the constraint stands out.

Functional responsibilities.

- Find and display the utilisation, average and percentile waiting times, throughput and queue length for each node.
- Locate the bottleneck node automatically and mark it clearly.
- A Flow and Balance Check using Little's Law to confirm that the run is consistent and a composite system health score from 0 to 100.
- Interactive tabs: Executive Summary, Flow and Balance Check, Traffic Heatmap, Line Buildup Timeline, Station Counters, Person / Item Journey, Live Activity Log and Cost and ROI.
- A Cost and ROI panel that translates the run into money terms, allowing an evaluation of whether a proposed change is cost-effective.

Implementation. The analytics are TypeScript modules that have no React in them, so they can be unit-tested individually. The health score is a weighted average based on completion rate (35%), utilisation balance (30%), queue stability derived from Little's Law (20%) and tail latency (15%). The resource or queue node with the longest average waiting time during the run is the bottleneck. Percentiles come from the real per-entity samples and are not estimated and work in progress is consistent with arrivals minus completions, which is verified by a conservation check.

Requirements covered	Principal source files	Developed by
FR-19 to FR-21	lib/simulation/analyticsEngine.ts, resolveKpiMetrics.ts; components/workspace/SimResultsPanel.tsx, AdvancedResultsDashboard.tsx, results-dashboard/*	Khushi Singh (261713)

Table 4.4: Key Performance Indicators and Their Computation

Metric	Definition	Input data
Utilisation	Share of the run for which a resource was busy	Accumulated busy-seconds per resource
Average waiting time	Average gap between joining a queue and starting service	Queue entry and service-start times per entity

Metric	Definition	Input data
Percentile waiting time	50th, 90th, 95th and 99th percentile of the waiting times	Real per-entity waiting samples
Throughput	Entities that finish the model per unit of simulated time	Sink completion counts and run duration
Average queue length	Time-weighted average number of entities waiting	Periodic depth snapshots
Little's Law check	Compares L with λW and reports the gap and whether the run is stable	Arrivals, completions, work-in-progress, mean time in system
System health score	0–100 mix: completion 35, utilisation balance 30, stability 20, tail latency 15	Outputs of the above
Total system cost	Resource, holding and waiting-penalty cost over the run	Cost parameters from Module 3

4.8 Module 8: Monte Carlo and Scenario Comparison

Purpose. One random run is a single observation and not an estimate. This module repeats the run and reports the range in which the real behaviour is likely to lie. It then allows two scenarios to be compared in light of that uncertainty.

Functional responsibilities.

- Run a model for 10, 20, 50 or 100 replications, each one independent.
- Work out the mean, standard deviation and 95% confidence interval for each metric across the replications.
- Represent the intervals graphically as bands, so that what is out of the interval can be seen as well as what is within it.
- Enter a baseline scenario next to a modified scenario and display the change in each metric.
- Highlight progress during a lengthy analysis, so that the user always gets feedback.

Implementation. All replications are run through the same engine as a single run and are combined using the standard-error formula from Section 1.1.2. There is also a scenario-comparison function that compares the baseline data recorded before a fix is applied with the current data. The aggregation is plain TypeScript code, so it is easy to test and independent of the interface.

Requirements covered	Principal source files	Developed by
FR-22	lib/simulation/monteCarlo.ts, monteCarloRunner.ts; components/workspace/MonteCarloPanel.tsx	Mohit Gupta (261715)

4.9 Module 9: Digital Twin Animated Playback

Purpose. Numbers describe congestion and animation shows it. Once a run is complete, this module replays it as a 2-D animation of entities moving through the model, so that problems in the system are visually apparent at once.

Functional responsibilities.

- Animated motion of entities through the nodes and edges of the model.
- Highlighting of queue congestion, so you can see lines emerge as they grow.
- Video-style controls: play, pause, step and speed up to 10×.
- Scrubbing to any point in the run without executing the model again.

Implementation. The playback itself is drawn with Pixi.js running on WebGL and not with DOM elements, because using the DOM to render several hundred moving entities is too slow. The animation is a playback of the recorded journeys and does not re-run the model. Therefore scrubbing and speed changes happen instantly and what is displayed always matches the run whose metrics are shown.

Requirements covered	Principal source files	Developed by
FR-23	components/workspace/DigitalTwinCanvas.tsx, PlaybackControls.tsx; lib/simulation/timelineSelectors.ts	Khushi Singh (261713)

4.10 Module 10: Simulation Template Gallery

Purpose. A blank canvas is a turn-off for a new user. This module provides working models for all six domains, which means users can begin with a working model and modify it. That is easier than starting from scratch!

Functional responsibilities.

- A gallery of starter graph examples, each with a brief description, grouped by domain.
- When you create a new instance of a template, its nodes, edges and default parameter values are added to your project.
- Each instance is a completely independent copy and never changes back into the master template.
- All parameters can be set to anything after you create your instance.

Implementation. Templates are declared in the simulation type registry, next to each domain's node palette and layout strategy. Therefore a domain is defined once and adding another doesn't mean changing a lot of pieces. The copies are independent because they are instantiated copies of the nodes and edges of the template.

Requirements covered	Principal source files	Developed by
FR-24	components/workspace/TemplateGallery.tsx; lib/simulation/simTypeRegistry.ts	Mohit Gupta (261715)

4.11 Module 11: Executive Report and Share

Purpose. Work that doesn't move off the platform isn't worth much. This module allows models and results to be taken anywhere else, whether to be backed up, analysed elsewhere or presented to someone who will not log on.

Functional responsibilities.

- One-click PDF executive report with a written summary, a canvas snapshot and KPI tables.
- Export of the project graph and of the results of the run in JSON and CSV formats.
- Export of the generated Python script as a Jupyter notebook, which others can use to reproduce the run.
- No edit access through public shared links: each shared link is created as a read-only share link.
- A share link can be taken away (revoked) and after that an explanatory notice appears on the URL.

Implementation. The canvas is captured with html-to-image and the PDF is generated on the client with jsPDF. This keeps server compute at zero, with no document-rendering service required. Links are purposefully opaque random characters. There are two very limited public functions in the database. One returns the name, domain and graph of a single shared project and the other says whether a token is active, revoked or invalid. Anonymous visitors never physically hit a table and the read-only nature is enforced not by the interface but by the database.

Requirements covered	Principal source files	Developed by
FR-25, FR-26	lib/simulation/pdfReportGenerator.ts, reportGenerator.ts, jupyterExport.ts; components/workspace/ShareExportModal.tsx, ShareViewer.tsx, ShareLinkUnavailable.tsx; app/api/projects/[id]/share and export; app/share/[token]	Khushi Singh (261713)

4.12 Module 12: Real-Time Collaboration

Purpose. Teams can expect a modern design tool to work with several people at once and this module provides just that. You can see what others are doing in the workspace (similar to Canva or Figma).

Functional responsibilities.

- **Live presence:** users see a row of avatars showing who is currently within the project.
- **Live cursors:** each collaborator's pointer is drawn on the canvas in their own colour, with their name.
- Broadcasting graph operations (node and edge changes and node-data edits) to the other connected clients.
- Selection and editing awareness (who is selecting or editing which node).
- Authorised channel (project level), which can be accessed only by users with access.

Implementation. Supabase Realtime channels use WebSockets when the application utilises collaboration. Each project comes with its own private channel that is identified by the project's identifier. Before the client joins, it ensures that it is able to read the project using Row-Level Security and a realtime policy authorises the channel at the database before the join. The "presence" status is carried by the channel, while cursor positions and graph operations are sent out as broadcast messages. The cursor position is not stored and is only temporary, as it has no meaning at the end of a session.

Requirements covered	Principal source files	Developed by
FR-27	lib/realtime/usePresence.ts; components/workspace/LiveCursor.tsx	Mohit Gupta (261715)

4.13 Hardware Modules and Considerations

JustCmul8 doesn't need any custom hardware. Still, running simulations on the client instead of the server gives the user's own machine a job it wouldn't otherwise have and we want to say that clearly.

Table 4.5: Hardware Roles in the Deployed System

Element	Role in the system	Consequence of the design
Client CPU	Runs the WebAssembly runtime and the whole simulation	Decides how long a run takes; big models are slower on weaker machines
Client memory	Holds the Pyodide runtime, the graph, the event trace and the rendered canvas	Sets the practical limit on model size and replication count
Client GPU	Draws the digital-twin playback through WebGL	Lets many entities animate smoothly at the same time
Network link	Carries only saving, collaboration and AI requests	Simulations keep running even if the connection is patchy
Application server	Serves the app and a small number of API routes	Does no simulation work, so hosting cost doesn't depend on how much people use it
Database server	Managed PostgreSQL holding projects, runs, chats and shares	Sized by how much data there is and not by how much computing is needed

The last two rows repeat the economic argument from Section 2.2.2 in hardware terms. Neither server takes part in the computation, so the infrastructure for a thousand users running simulations is about what a thousand users viewing a static page would need. That is what keeps the platform sustainable without a licence fee.

4.14 Individual Work Justification

The project was done by two people. Requirements analysis, architecture, database design, integration, testing and this report were shared work. The twelve modules were split equally, six each. The owner of a module designed it, implemented it, unit-tested it and documented it and the other member reviewed it. The split is in the table below.

Table 4.6: Division of Work between Team Members

Module	Owner	Justification of contribution
1 Platform and Data Layer	Khushi Singh (261713)	Authentication flows, session middleware, dashboard, settings, database schema and Row-Level Security policies
2 Visual Graph Editor	Mohit Gupta (261715)	React Flow canvas, custom node components, palette, layout engine and structural validation
3 Node Configuration Panel	Khushi Singh (261713)	Type-specific forms, distribution controls, cost parameters and value validation
4 Simulation Engine	Mohit Gupta (261715)	SimPy code generator, Pyodide worker, execution pipeline and the verification harness
5 AI Assistant	Khushi Singh (261713)	Chat interface, prompt design, graph generation, model router and history
6 AI System Optimiser	Mohit Gupta (261715)	Flow analysis, bottleneck diagnosis, one-click fix and cost-aware recommendation
7 KPI Dashboard and Results	Khushi Singh (261713)	Analytics engine, Little's Law check, health score, dashboard tabs and Cost and ROI
8 Monte Carlo and Scenario Comparison	Mohit Gupta (261715)	Replication runner, confidence intervals and scenario comparison
9 Digital Twin Playback	Khushi Singh (261713)	Pixi.js renderer, entity journeys, transport controls and timeline
10 Template Gallery	Mohit Gupta (261715)	Domain registry, six-domain template library and instantiation
11 Executive Report and Share	Khushi Singh (261713)	PDF report generation, JSON/CSV/notebook export, share links and read-only viewer
12 Real-Time Collaboration	Mohit Gupta (261715)	Presence channel, live cursors and graph-operation broadcast

We both worked on the specification, the design diagrams, the test cases and the report. Reviewing each other's modules is what let us put the pieces together with few defects.

5. SYSTEM ANALYSIS AND DESIGN

This chapter focuses on analysing and designing JustCmul8 from the outside to the inside of the system. The boundary is drawn in the context diagram. Data flow diagrams depict the processes within it. The entity relationship diagram and class diagram represent the data and objects and the behavioural diagrams represent how the objects interact when the system runs. The chapter concludes with two physical topics: the database tables and the deployment setup.

5.1 Context Diagram

The boundary of the system is fixed by the context diagram (DFD level 0). JustCmul8 is drawn as a single process with the external entities that exchange data with it around it; nothing inside is broken down. Its sole duty is to decide what is and what isn't part of the system before covering any internals.

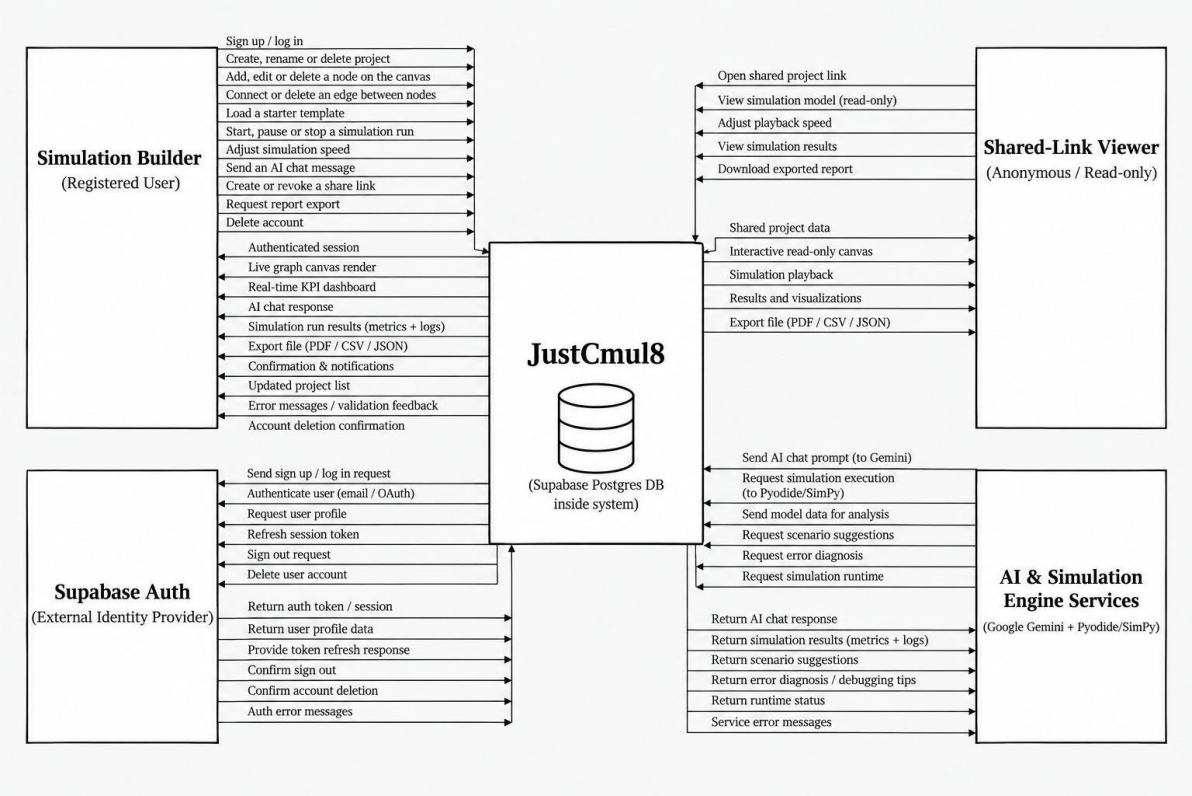


Figure 5.1: Context Diagram (DFD Level 0) for JustCmul8

There are four entities outside the system. The Simulation Builder is the registered user who creates and runs models. This entity sends in flows for login and registration, creating, renaming and deleting projects, adding, editing and deleting nodes and edges, loading starter templates, running and stopping a run, adjusting the simulation speed, sending an AI chat

message, creating and revoking a share link, asking for report exports and deleting an account. The flows returning include the authenticated session, the real-time KPI dashboard, AI replies, run results, metrics, logs, export files, confirmations, the new project list and validation feedback.

The Shared-Link Viewer is an anonymous person who can be reached with a URL to view a shared page. This entity opens a shared project, views the model read-only, changes playback speed, looks at the results and downloads an exported report. The flow is not equal for the two user entities: nothing the Shared-Link Viewer does affects the stored state.

Supabase Auth is the external identity provider used. The system sends it requests for sign-up and sign-in, requests for profile information, refreshes of sessions, sign-out requests and instructions for deletion of the account and receives back tokens, sessions, profile information and confirmations. This also demonstrates that JustCmul8 does not build authentication itself and passes it out of the boundary. The AI and Simulation Engine Services entity stands for the Google Gemini API and for the runtime in the form of Pyodide and SimPy delivered over a CDN. The PostgreSQL database is placed within the boundary because the application has the ability to set it up and control it and the database is not merely used as an independent third-party application.

5.2 DFD, ERD, Class Diagram and State Transition Diagram

5.2.1 Data Flow Diagram, Level 1

The single process from the context diagram has been broken down into seven functional processes in the level-1 diagram and the data stores that they read and write have been added.

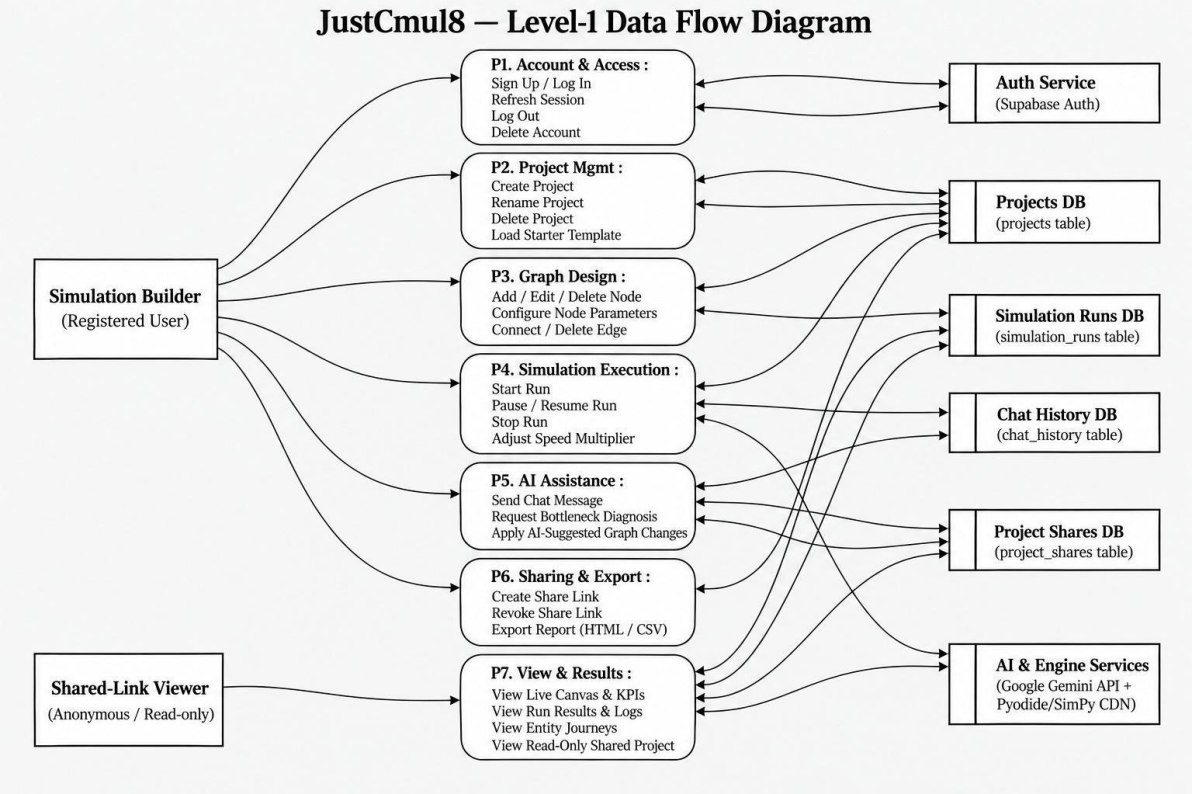


Figure 5.2: Level-1 Data Flow Diagram for JustCmul8

P1 Account and Access, deals with signing up and logging in to the system, refreshing sessions, logging out and deleting accounts and communicates with the external authentication service. P2 which is Project Management it is used for creating, re naming and also deleting projects and loading starter templates and reads from and writes to the Projects store. P3 which is Graph Design which is used adds, modifies and removes nodes that sets their parameters and connects or removes edges between nodes and saves each change to the same store.

P4 Simulation Execution does the actual running of the simulation and can start or stop runs and alter the speed multiplier. It reads the graph from the Projects store, drives the external engine services and writes run records to the Simulation Runs store. P5 AI Assistance sends a chat message, requests a bottleneck diagnosis and applies the graph changes suggested by the AI. It communicates with the Gemini service and writes the exchange to the Chat History store. P6 Sharing and Export generates and terminates share links and generates exports, writing to

the Project Shares store. P7, View and Results, hosts the real-time canvas and KPIs, run results and logs, entity journeys and the read-only shared view. The read-only guarantee is due not to convention but to the fact that it is the only process that the anonymous Shared-Link Viewer can access.

5.2.3 Entity Relationship Diagram

The ER diagram displays the organisation of the data stored in the database and the relationships between the entities. It is in Chen notation: entities are drawn as rectangles, relationships as diamonds and attributes as ellipses and the cardinality is shown.

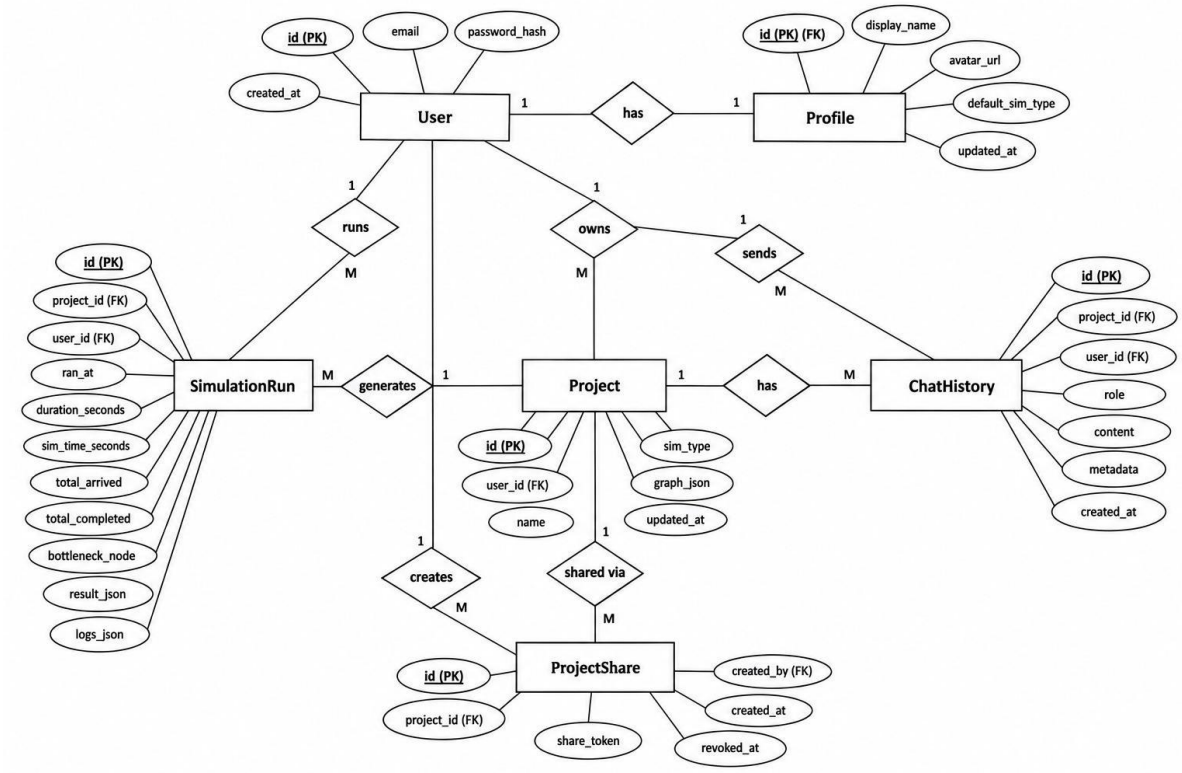


Figure 5.4: Entity Relationship Diagram for the JustCmul8 Database

The User is in the centre and has a one-to-one relationship with Profile, which contains the display name, avatar URL, default simulation type and update timestamp. The profile was separated from the credential record to keep the presentation data apart from the authentication data. A User owns a number of Projects, each of which has an identifier, an owner, a name, a type of simulation, the graph and an update date (in the form of a timestamp). Many SimulationRuns, ChatHistory messages and ProjectShares are also created by a User. Each Project produces many runs, has many chat entries and is shared through many share records.

The SimulationRun entity stores the run timestamp, wall-clock duration, simulated duration, totals arrived and completed, the identified bottleneck node and the result and log payloads. The bottleneck is stored as part of the run and is not recalculated when the run is recalled, so an old run preserves the diagnosis made at that time. ProjectShare holds an opaque share token with creation and revocation timestamps. The row does not get deleted on revocation but is left as it was, with a revocation timestamp.

5.2.4 Class Diagram

The class diagram depicts the object model as used in TypeScript, along with class names, attributes with their types, operations and associations.

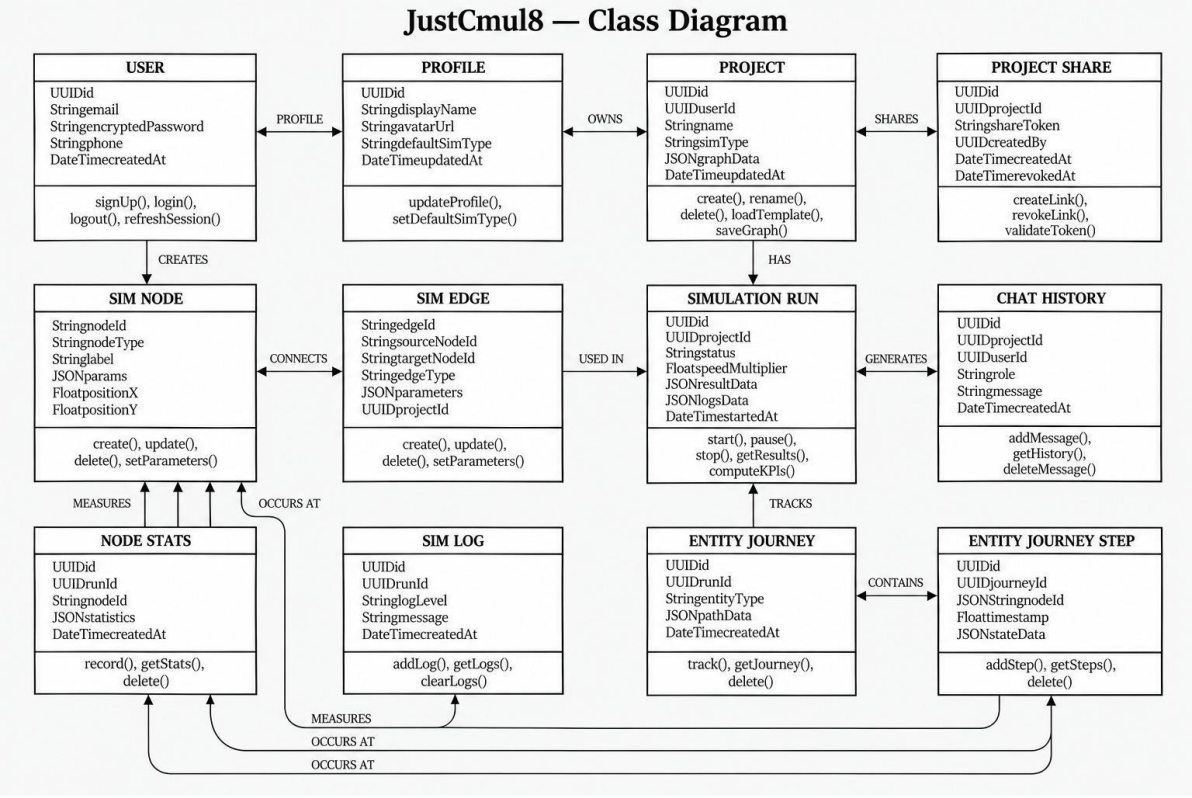


Figure 5.5: Class Diagram of the JustCmul8 Object Model

User, Profile, Project and ProjectShare are the account and project classes in the top row. The associations read “User has a Profile”, “User owns Projects” and “Project is shared through ProjectShare”. The graph and execution classes are in the middle row. SimNode has the node identifier, type, label, a JSON parameter object and canvas coordinates. SimEdge joins two nodes. SimulationRun has the status, speed multiplier and result and log payloads. Each AI conversation is saved in ChatHistory. There is no sparse schema: fifteen node types with quite

different parameters can be stored in the same class, because the parameters are kept as part of the JSON.

The bottom row has the measurement classes: NodeStats, SimLog, EntityJourney and EntityJourneyStep. The journey and step classes exist for the digital-twin playback. Every step has a timestamp and a node reference, which allows the animation to be rebuilt and scrubbed forwards and backwards without having to run the simulation again.

5.2.5 State Transition Diagram

The state transition diagram traces the state of a node as a model is executed. Each node can be in only one of these states at any instant and the changes are made by the event scheduler.

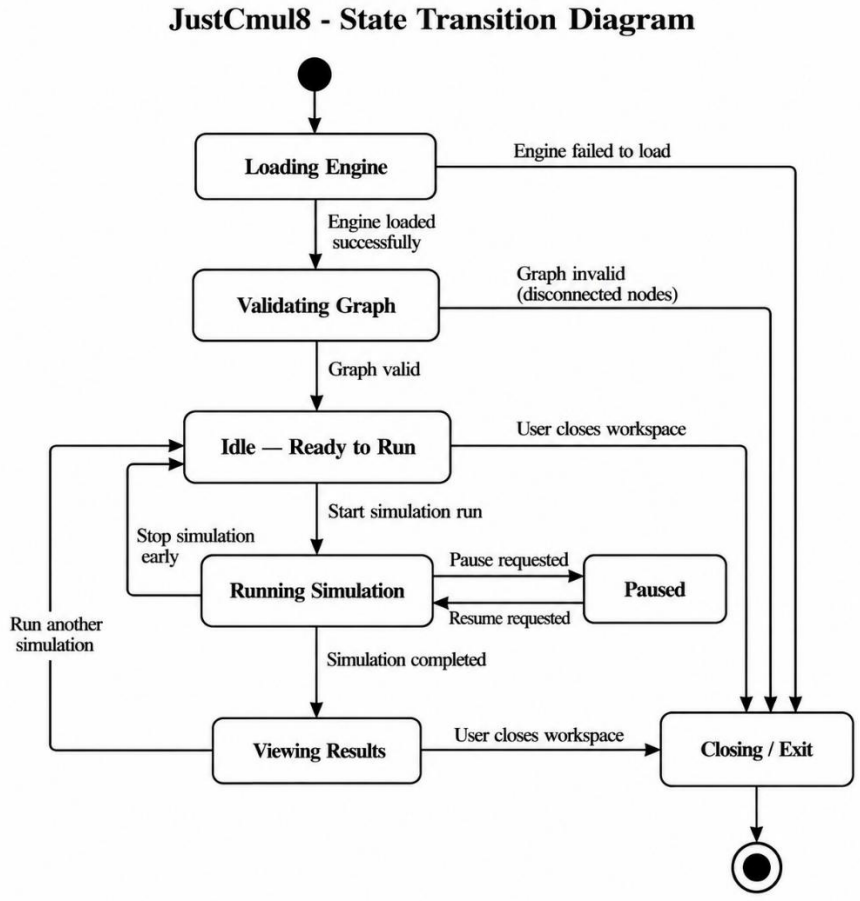


Figure 5.6: State Transition Diagram for a Node During Simulation Execution

A node has nothing to process in IDLE when it is initiated. When an entity arrives, the node goes to QUEUED, where the entity waits for capacity. When the resource is available, it goes into PROCESSING and holds the entity for a length of time drawn from the distribution defined for the service time. Once this is finished, the node moves to COMPLETED and the entity is forwarded on its path. If no other entity is waiting, the node state becomes IDLE.

Two unusual transitions leave PROCESSING. If there is no node ready to accept the entity downstream, the node is placed in the BLOCKED state and remains BLOCKED until there is capacity. Blocking is how congestion moves backwards through a model and giving it a state of its own lets the analytics tell a node that is slow in itself from one that is only held up by the node after it. The other transition is to FAILED, which occurs when an interrupter node interrupts a process, when a process is preempted by a priority resource or when there is a runtime error. The state model maps directly to SimPy: QUEUED is an outstanding resource request, PROCESSING is a timeout while the resource is held, BLOCKED is a pending put on the downstream store and FAILED is a raised Interrupt.

5.3 Use Case Diagram

The use case diagram gives the actors and the services the system provides to them. It is a description of the system from the outside, as it is used and not from the inside, as it is performed.

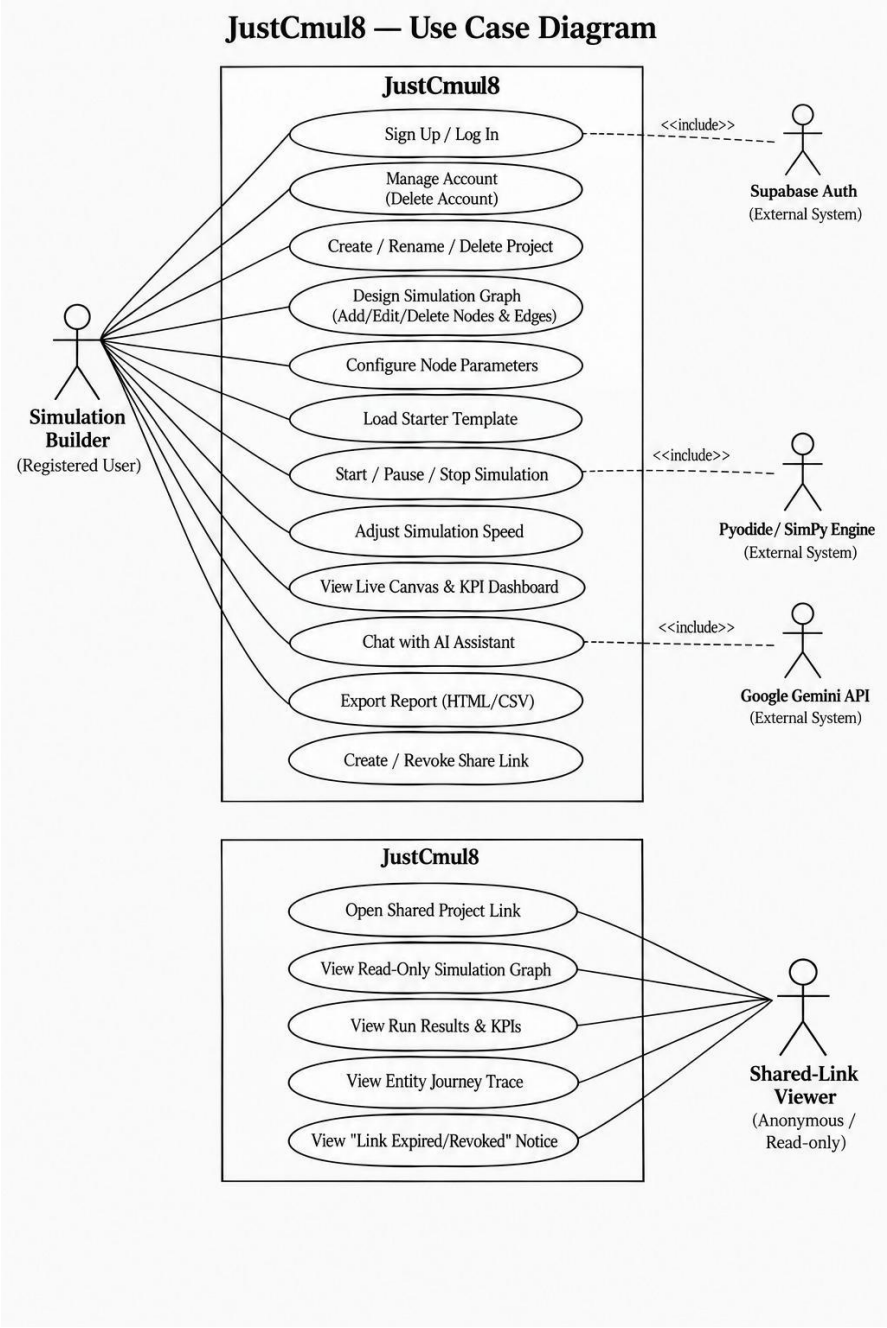


Figure 5.7: Use Case Diagram for JustCmul8

There are two actors you can play. The Simulation Builder, a registered user, can sign up or log in, manage the account (including deleting it), create, rename and delete projects, design the simulation graph, set node parameters, load a starter template, start and stop a simulation,

change the speed, watch the live canvas and KPI dashboard, chat with the AI assistant, export a report and create or revoke a share link. The Shared-Link Viewer is an anonymous actor whose use cases are in a separate boundary: viewing a shared project link, viewing the read-only graph, viewing results and KPIs, viewing the entity journey trace and viewing the notice shown when a link is revoked or expired. That last one is really easy to miss and leaving it out leads to a poor experience, so we modelled it on purpose and did not treat it as an error path.

Three external systems take part through include relationships. Supabase Auth is included by sign-up and log-in, the Pyodide and SimPy engine by starting and stopping a simulation and the Google Gemini API by talking to the AI assistant. We went with include and not extend because in each case the interaction with the outside application has to take place for the use case to conclude.

5.3.1 Use Case Specifications

Table 5.1: Use Case Specification: Run Simulation

Field	Description
Use case ID	UC-01
Use case name	Run Simulation
Primary actor	Simulation Builder
Secondary actor	Pyodide / SimPy engine
Precondition	The user is logged in, a project is open and the graph has at least one source and one sink
Main flow	1. The user sets the duration and time unit. 2. The user starts the run. 3. The system checks the graph structure. 4. The system generates the SimPy script. 5. The worker runs the script through Pyodide. 6. Progress and log events stream to the interface. 7. The system computes the KPIs and ranks the bottleneck. 8. The dashboard shows the results and the run is saved.
Alternative flow	3a. Validation fails: the system names the disconnected node, explains the problem and stops before running. 5a. The runtime throws an error: the error is shown along with the related log entries. 5b. When The user will stop the run the execution is halted.
Postcondition	If the run succeeds, a run record with KPI results and logs is saved
Requirements	FR-12, FR-13, FR-14, FR-15, FR-19, FR-20

Table 5.2: Use Case Specification: Generate Model via AI

Field	Description
Use case ID	UC-02
Use case name	Generate Simulation Model from Natural Language
Primary actor	Simulation Builder
Secondary actor	Google Gemini API
Precondition	The user is logged in and a project is open
Main flow	1. Firstly user will describe the scenario or the flow in English to the AI Assistant. 2. Then the assistant will build prompt from the description and the node registry and also the current graph. 3. Then the request will go through the server to Gemini. 4. Then the reply gets parsed into candidate nodes and edges. 5. After that candidate's graph then gets validated. 6. The graph is then created and shown as a preview. 7. When user accepts only the graph is finalised into the project.
Alternative flow	5a. Validation will fail on a unknown node type or dangling edge then the candidate is rejected and the user is asked to rephrase it. 7a. User discards the preview then the project stays as it was. 3a. When the preferred model is down the router falls back to the next model. If all of them fails a message explains it and the user can still build by hand.
Postcondition	Either the project has the accepted graph or it is untouched
Requirements	FR-16, FR-17

Table 5.3: Use Case Specification: Share Project Read-Only

Field	Description
Use case ID	UC-03
Use case name	Share Project through a Public Link
Primary actor	Project Owner
Secondary actor	Shared-Link Viewer (anonymous)
Precondition	The user owns the project
Main flow	1. When the owner asks for a share link. 2. Then the system generates an opaque token and later on stores it against the project. 3. The link is shown for copying and sharing. 4. A viewer can open the link without logging in. 5. Software then validates the token and it shows the project in read only.
Alternative flow	5a. When token has been revoked the viewer sees an explainatory notice. 5b.Whhe token isn't recognised access is refused. 2a. The owner revokes the link later then the timestamp is set and later requests fall under 5a.
Postcondition	The viewer can read that project only and can't change anything

Field	Description
Requirements	FR-26

Table 5.4: Use Case Specification: Collaborate in Real Time

Field	Description
Use case ID	UC-04
Use case name	Live Collaboration in a Project
Primary actor	Collaborator
Secondary actor	Supabase Realtime, other collaborators
Precondition	Two or more logged-in users with access have the same project open
Main flow	1. Each client will check its access to the project and then joins the project's private channel. 2. Both the Presence state and cursor positions are broadcast all the time. 3. When a user changes an node or edge. 4. The change is then broadcast as a graph operation. 5. Every other user applies the operation and redraws.
Alternative flow	1a. The user has no access to the project: the channel isn't joined. 5a. A client disconnects: its presence indicator is removed.
Postcondition	Connected clients show the same graph and can see each other's presence
Requirements	FR-27

5.4 Architecture Design

JustCmul8 is built in a layered fashion: each layer relies only on the one below it. The unusual part is the execution layer. In a normal web application the computation takes place on the server, but here it takes place on the client and nearly all the consequences discussed in this report come from that switch.

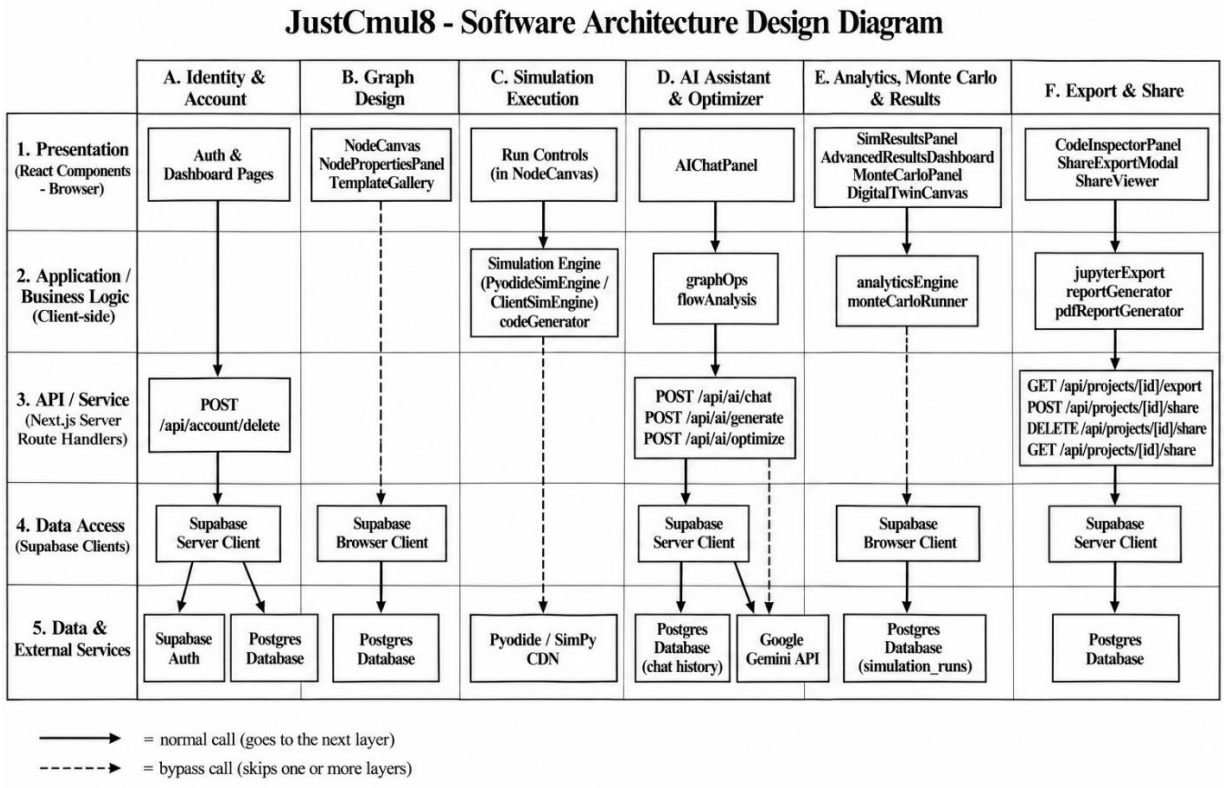


Figure 5.8: Architecture Design of JustCmul8

The presentation layer uses Next.js 16, React 19 and Tailwind CSS. It has the landing and authentication pages, the dashboard, the workspace shell that holds the canvas and panels and the read-only share viewer. It doesn't contain any business logic. It shows the state and passes on what the user wants to do.

The Zustand slices are held in the application and state layer, which includes the graph store, the simulation store, the interface store and the presence store. They are independent slices, so a change in the playback position does not cause components whose only job is to watch the graph to be rendered again. When there are hundreds of nodes on the canvas, that matters.

The following TypeScript modules are in the domain and service layer: the code generator (conversion of a graph to SimPy), the analytics engine (computation of KPIs), the Monte Carlo

runner and the report and export modules. They are plain modules with no React in them, so each one can be tested as a standalone unit.

The execution layer runs in a Web Worker, off the main thread. It includes the Pyodide WebAssembly runtime and the SimPy engine, with the event priority queue used to order the scheduled events. It is isolated and never “freezes out” the interface, which was one of our requirements.

The persistence and external services layer is made up of Supabase Auth, PostgreSQL with Row-Level Security, Supabase Realtime and the Gemini API. The authorisation is not in the presentation layer, because that is where it would be easiest to bypass; it sits in the bottom layer. Any request that reaches the database is made in the user’s name and is filtered there, whatever the client believes it is allowed to do.

5.5 Activity, Component and Collaboration Diagrams

5.5.1 Activity Diagram: End-to-End User Workflow

In Chapter 3 you were given an activity network of the development lifecycle. This is a different flow: here a user moves from logging in to exporting in a running system. Swimlanes separate the activities of the Simulation Builder, the system, Supabase Auth, the Pyodide and SimPy engine and the Gemini API, making it clear who is doing what.

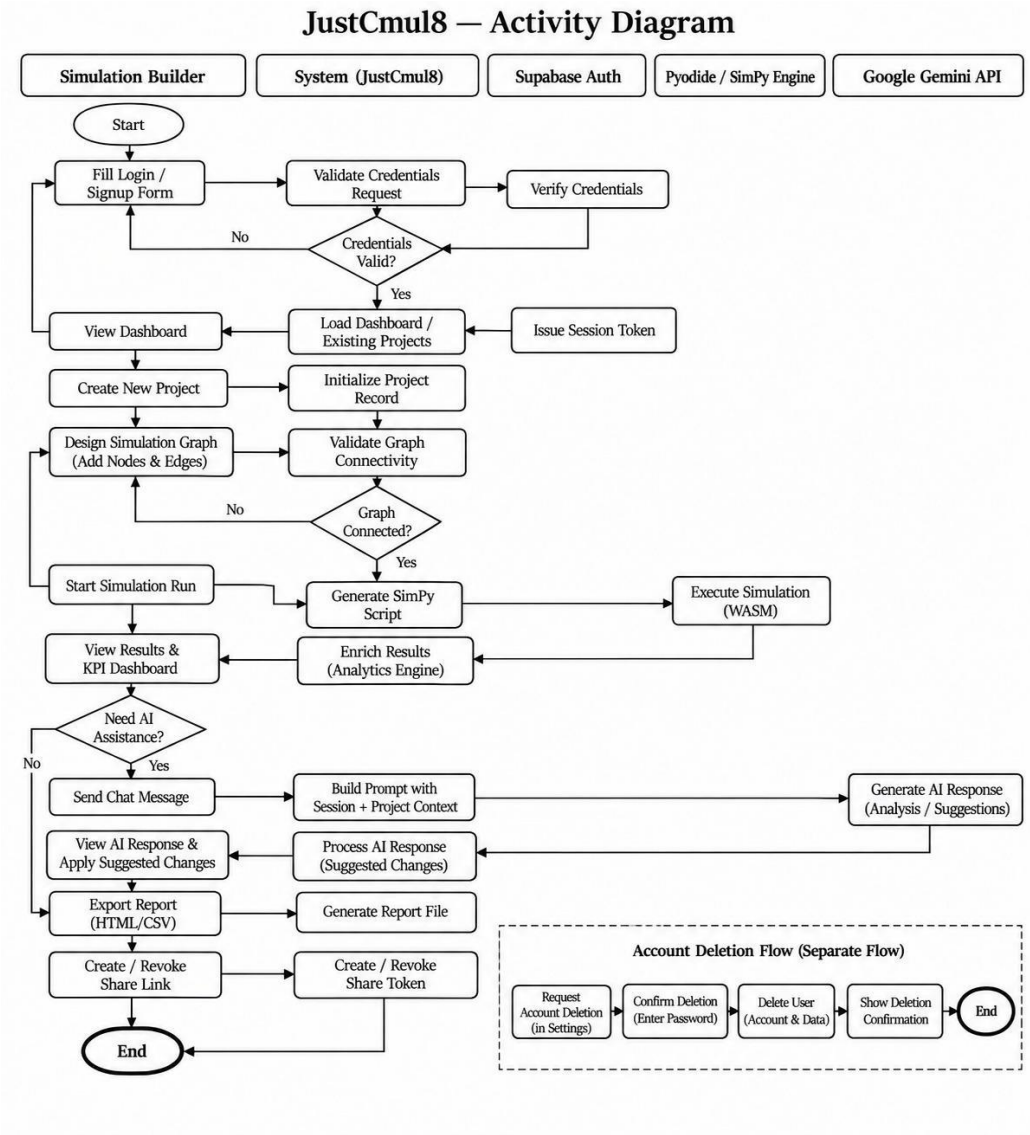


Figure 5.9: Activity Diagram of the End-to-End User Workflow

The flow begins with the user filling in the login or sign-up form. The system asks Supabase Auth to verify the credentials and a decision node checks whether they are valid. If not, control returns to the form. If they are, a session is issued and the dashboard loads the user's projects. The user creates a project and designs a graph, which the system validates. If the graph is not

connected, a second decision node returns control to the design activity, which keeps an unrunnable model from reaching the engine.

Once the graph is valid the user starts the run. The program will then generates the SimPy script and then it goes to the engine model which then executes it as WebAssembly. Then the result is returned to the software where the analytics engine draws them and the user reviews the KPI dashboard. When A third decision node asks whether the user wants help from the Assistant. The system will build a prompt from the project context Gemini provide analysis or suggestions and then the user can apply them which returns control to the design activity. If not the flow continues with exporting a report and creating or revoking a share link before it reaches the end node. Account deletion is then drawn as its own flow as it is not part of the modelling logic and should not be there by chance from inside the engine.

5.5.2 Component Diagram

The component diagram will then show model of the software and how they depend on each other and also grouped by where each one runs.

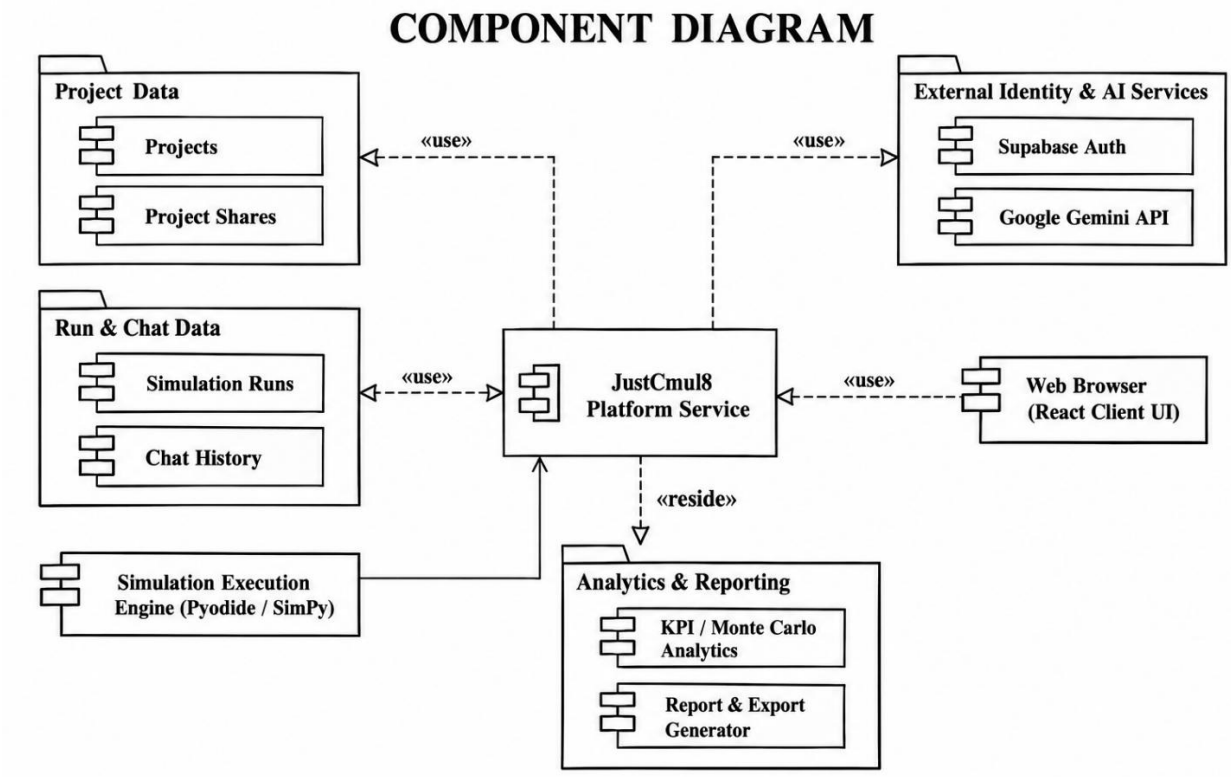


Figure 5.10: Component Diagram of the JustCmul8 Implementation

Then web browser client runtime has the interrupt the components (the graph editor on React Flow, configuration panel KPI dashboard using Recharts and Pixi.js digital twin) the Zustand saves the state and the service section for code generation and analytics and the export engine. Interface components depends on the state and not on each other, so a block can be replaced or moved without touching the rest. The simulation thread is a separate kind of collection as it runs in a Web Worker.

5.5.3 Collaboration Diagram

The collaboration diagram shows the objects involved in one simulation run and the messages they pass, numbered in the order they happen.

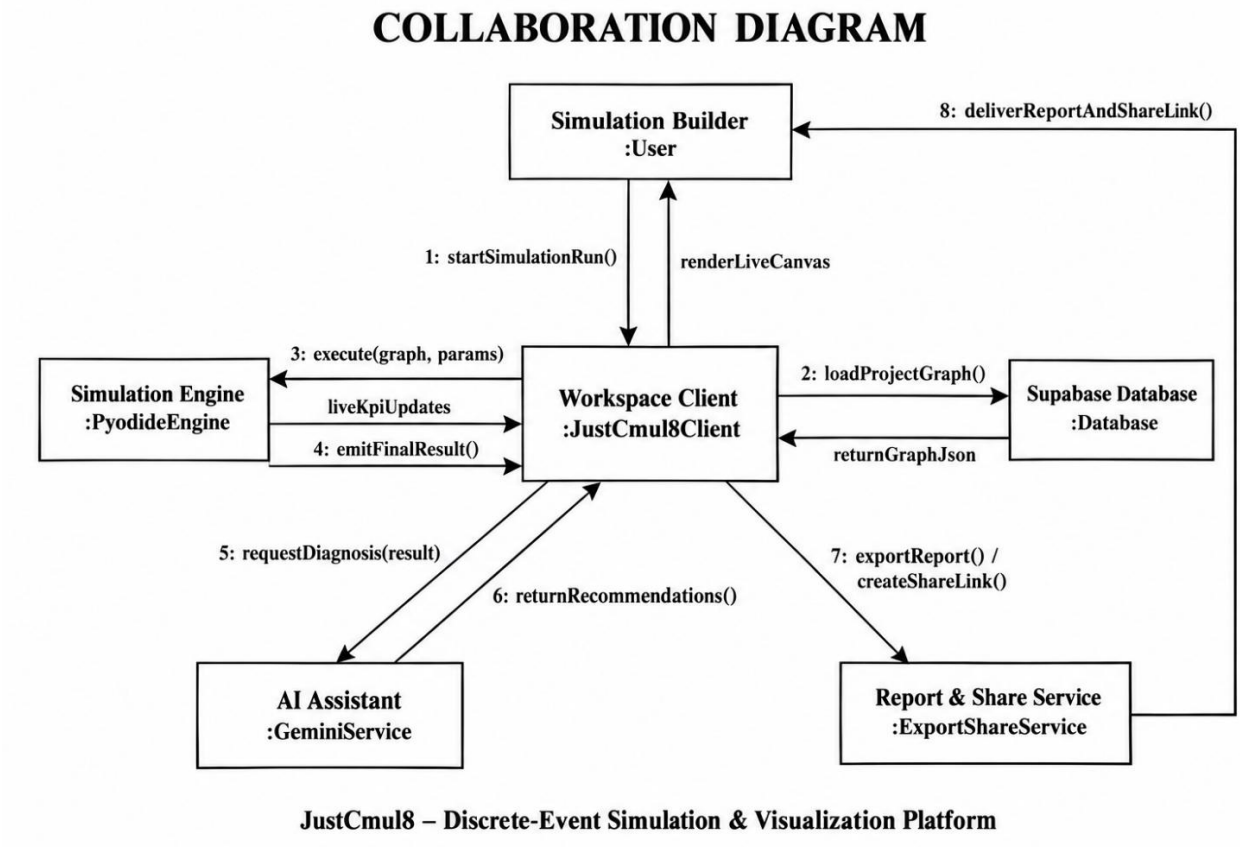


Figure 5.11: Collaboration Diagram for the Simulation Run Interaction

The interaction will start only when the project state will call startRun on the store with the run config (1). The store then will inovke buildScript on the code generator and then it passes the current graph (2). Then the generated script is forwarded to the simulation worker (3) which crosses the thread limit. Inside the worker runPython is called on the Pyodide runtime (4), which gives the result (5). Then the trace is forwarded to the main thread where the analytics engine receives it (6), renders KPIs on the dashboard (7) and writes the computed results into

the store (8). The state will saves the execution through the Supabase (9) and the dashboard shows the identified bottleneck (10). Two things stand out. The store sits between the interface and the services, so any component that watches it gets a consistent view of the run. And saving happens after analysis, so a run is stored with its computed metrics in a single write.

5.6 Table Design

The database is on PostgreSQL. Every primary key has a UUID generated by the database automatically. Foreign keys use cascade when delete wherever the child can't exist without the parent and every application table has a Row-Level Security inherited in it. Anything whose shape depends on type is saved as JSONB then is stored the relational schema small and stable and still runnable. The nodes with their parameters and positions is saved as one JSONB document on the project which then makes loading and export faster.

5.6.1 Tables

Table 5.5: Table: auth.users (managed by Supabase Auth)

Column	Data type	Constraints	Description
id	uuid	PRIMARY KEY	Unique user identifier, referenced by every application table
email	text	UNIQUE	Login email address
encrypted_password	text	None	Salted hash of the password; never stored in plain text
raw_user_meta_data	jsonb	None	Registration metadata, including the full name
created_at	timestamptz	DEFAULT now()	Account creation timestamp

Table 5.6: Table: public.profiles

Column	Data type	Constraints	Description
id	uuid	PRIMARY KEY, FK → auth.users(id) ON DELETE CASCADE	Shares the user's identifier (one-to-one)
display_name	text	None	Name shown in the interface and on collaborator cursors
avatar_url	text	None	Profile image location
default_sim_type	text	DEFAULT 'human_queue'	Domain preselected for new projects
updated_at	timestamptz	DEFAULT now()	Last modification timestamp

Table 5.7: Table: public.projects

Column	Data type	Constraints	Description
id	uuid	PRIMARY KEY, DEFAULT gen_random_uuid()	Unique project identifier

Column	Data type	Constraints	Description
user_id	uuid	NOT NULL, FK → auth.users(id) ON DELETE CASCADE	Owning user
name	text	NOT NULL	Display name
sim_type	text	NOT NULL, DEFAULT 'human_queue'	One of the six simulation domains
graph_json	jsonb	DEFAULT {nodes: [], edges: []}	The graph: nodes, parameters, positions and edges
updated_at	timestamptz	DEFAULT now()	Last modification timestamp

Table 5.8: Table: public.simulation_runs

Column	Data type	Constraints	Description
id	uuid	PRIMARY KEY	Unique run identifier
project_id	uuid	NOT NULL, FK → projects(id) ON DELETE CASCADE	Project that was simulated
user_id	uuid	NOT NULL, FK → auth.users(id) ON DELETE CASCADE	User who initiated the run
ran_at	timestamptz	DEFAULT now()	Run timestamp
duration_seconds	integer	None	Wall-clock execution time
sim_time_seconds	numeric	None	Simulated duration modelled
total_arrived	integer	None	Entities generated
total_completed	integer	None	Entities that reached a sink
bottleneck_node	text	None	Node identified as the constraint in this run
result_json	jsonb	None	Per-node metrics and aggregate KPIs
logs_json	jsonb	None	Sampled log events

Table 5.9: Table: public.chat_history

Column	Data type	Constraints	Description
id	uuid	PRIMARY KEY	Unique message identifier
project_id	uuid	NOT NULL, FK → projects(id) ON DELETE CASCADE	Project the exchange concerns
user_id	uuid	FK → auth.users(id) ON DELETE CASCADE	User who sent the message
role	text	NOT NULL, CHECK IN ('user', 'assistant')	Originator of the message

Column	Data type	Constraints	Description
content	text	NOT NULL	Message text
metadata	jsonb	None	Model identifier and any graph payload the reply produced
created_at	timestamptz	DEFAULT now()	Message timestamp

Table 5.10: Table: public.project_shares

Column	Data type	Constraints	Description
id	uuid	PRIMARY KEY	Unique share record
project_id	uuid	NOT NULL, FK → projects(id) ON DELETE CASCADE	Project being shared
share_token	text	NOT NULL, UNIQUE, random 128-bit hex default	Opaque token embedded in the public URL
created_by	uuid	NOT NULL, FK → auth.users(id) ON DELETE CASCADE	Owner who created the link
created_at	timestamptz	DEFAULT now()	Creation timestamp
revoked_at	timestamptz	NULL while active	Revocation timestamp

5.6.2 Access Policies, Functions and Indexes

Table 5.11: Row-Level Security Policies

Table	Operations	Policy condition
projects	SELECT, INSERT, UPDATE, DELETE	auth.uid() = user_id
simulation_runs	SELECT, INSERT, DELETE	auth.uid() = user_id
chat_history	SELECT, INSERT, DELETE	auth.uid() = user_id
project_shares	ALL	auth.uid() = created_by
profiles	SELECT, INSERT, UPDATE	auth.uid() = id
realtime.messages	SELECT	Channel topic must be “project:” followed by a project the user owns

Besides tables and policies, the schema has a few objects that carry behaviour. get_shared_project(token) is a security-definer function that returns the name, domain and graph of one project when the token is active. get_share_status(token) returns invalid, revoked or active. Anonymous visitors can run these two functions and nothing else and they never get direct access to any table. handle_new_user() is a trigger function that inserts a profile row

every time a new authentication user is created. Indexes on projects(user_id), projects(updated_at DESC), chat_history(project_id), chat_history(user_id) and project_shares(share_token) keep dashboard and share lookups fast.

5.6.3 Referential Integrity

- profiles.id → auth.users.id (one-to-one, cascading)
- projects.user_id → auth.users.id (cascading)
- simulation_runs.project_id → projects.id (cascading); simulation_runs.user_id → auth.users.id (cascading)
- chat_history.project_id → projects.id (cascading); chat_history.user_id → auth.users.id (cascading)
- project_shares.project_id → projects.id (cascading); project_shares.created_by → auth.users.id (cascading)

We used cascading delete wherever a child row means nothing without its parent. Because of that, deleting an account is one operation that really removes the user's data and leaves no orphans behind.

5.7 Architecture / Deployment Diagram

The deployment diagram shows the physical nodes that the system runs on and the protocols connecting them.

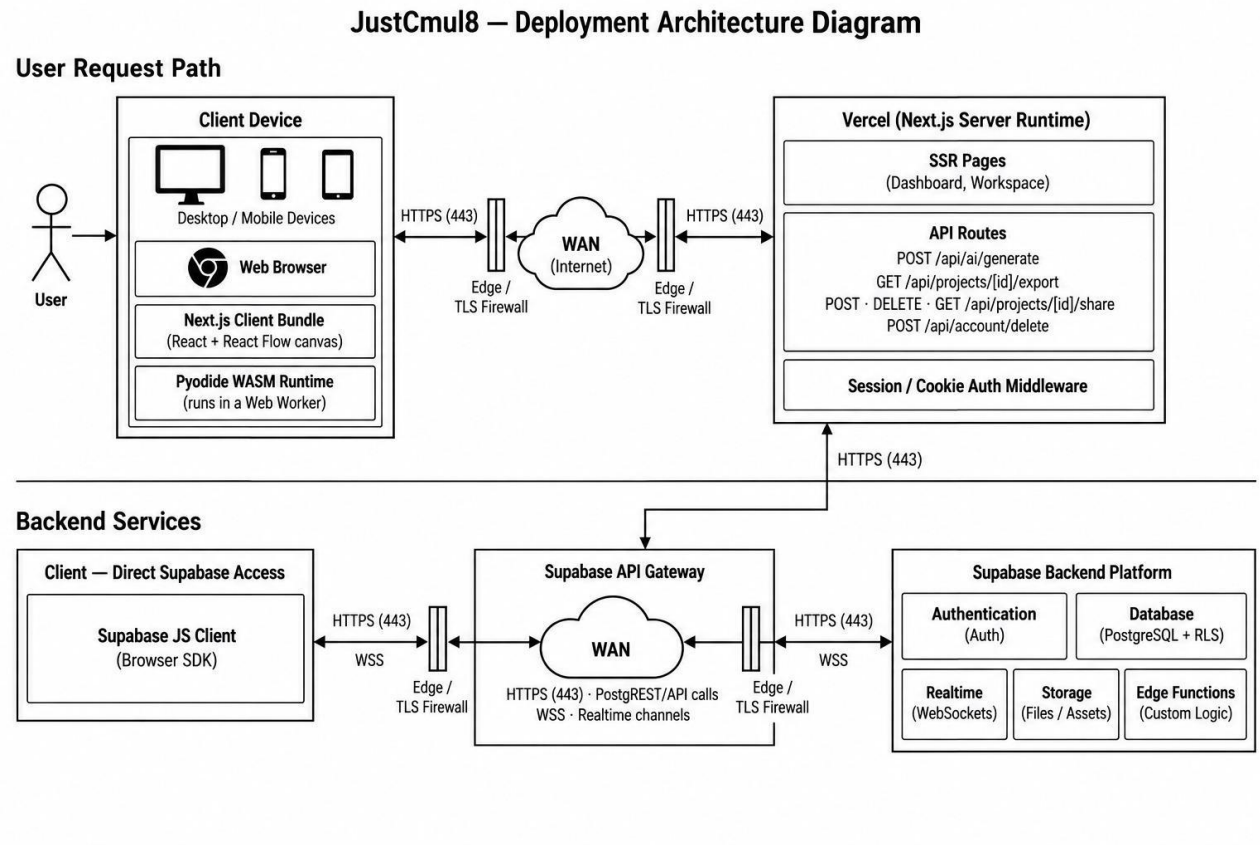


Figure 5.12: Deployment Architecture of JustCmul8

The client device has a web browser running the Next.js client bundle, which holds the React application and the React Flow canvas and next to it the Pyodide WebAssembly runtime running in a Web Worker. The diagram shows the main architectur point clear which then simulation runtime is deployed on the user's own device and not in any data centre.

Requests goes through over HTTPS through edge TLS1.7 protection and the public internet to the Vercel, which runs the Next.js server at the run time. That run time shows the dashboard and workspace pages and also the API routes. It will also apply the session at the middleware. The backend services has the Supabase API gateway which is connected to both from the Next.js server and directly from the browser through the Supabase. Behind the gateway are authentication and PostgreSQL with RLS and the real time WebSocket service.

Letting the browser reach the database directly, instead of proxying every read through the application server, is only safe because Row-Level Security is enforced inside PostgreSQL.

The server is kept for operations that really need a secret, mostly holding the Gemini API key. That keeps the deployment small and, along with client-side simulation, makes the running cost of the platform almost independent of how heavily it is used.

6. SYSTEM IMPLEMENTATION AND METHODOLOGY

This chapter details the process of getting the design from Chapter 5 to software reality. It presents the methodology used, the technologies implemented and the important algorithms, how each individual module and the database were handled, how the user interface was designed and the code fragments that illustrate the system's qualities best.

6.1 Development Methodology

We followed the iterative and incremental model from Section 3.5. The engine was developed first as a prototype and verified against analytical queueing results before any module that depends on it was built. Each module was then designed, coded, unit tested and integrated.

- **Version control:** Git with a GitHub remote. We worked in small steps with clear commit messages, so the history can be considered a record of the project.
- **Code review:** if someone didn't write a module, he reviewed it prior to its being merged.
- **Type safety:** all functions and variables are typed, so a mismatch between a node's parameters and the code using them is detected at compile time.
- **Separation of concerns:** analytics, code generation, Monte Carlo and export are simple TypeScript modules with no interface dependencies, so they can be tested separately.
- **Verification first:** a separate native harness verifies the simulation engine against closed-form queueing results before interface work is based on the engine's output.
- **Consecutive documentation:** module documents were written alongside the code and merged into this report later.

6.2 Technologies Used

The technology stack and the reasons for each choice, were given in Section 2.3.3. This section describes how each technology is used in the implementation.

- **Next.js 16 and the React** shows rendering of the landing and authentication page and the API routes that shows the AI service and generate share tokens. The workspace itself is a client rendered SPA.
- **TypeScript** is used for all application and service code. The SimPy program is produced as Python text by a TypeScript module.
- **React Flow** implements the canvas, node handles, edges, viewport and selection and **Dagre** arranges the generated and template graphs.
- **Pyodide** will load CPython which compiles to Web Assembly in an Web Worker. The script is ran with run PythonAsync and the results are returned through callback functions registered through the Python global name.
- **SimPy** provides the event scheduler, resources, stores, containers and interrupts on which the generated model is based.
- **Zustand** stores the app state in parts and **Recharts** renders the dashboard charts and **Pixi.js** renders the digital twin. **Framer Motion** and **Lenis** will show the transitions and smooth scrolling on the landing page.
- **Supabase** provides authentication (email and OAuth), PostgreSQL with Row-Level Security (RLS) and Realtime channels for presence and broadcast.
- **Google Gemini** is accessed through a model router that keeps an ordered list of models and a short-term health record for each.
- **jsPDF and html-to-image** generate the executive report and capture the canvas in the browser.
- **Tailwind CSS v4** styles the interface consistently through utility classes and design tokens.

6.3 Algorithms and Techniques Used

6.3.1 Graph Representation and Validation

The simulation model is represented as a directed graph and stored in memory as an **adjacency list**, which maps each node identifier to the list of its outgoing neighbours. Simulation graphs are sparse, since a node typically has only one or two successors. An adjacency matrix would waste space, because it grows as n × n, where n is the number of nodes, however few edges there are. The list also matches how the engine works: routing an entity needs the successors of one node and the list hands them over directly. Given an interface with nodes and a graph, it walks from the nodes pre-run, using a set of visited nodes and reports any node which could not be reached. If that is the case, it is rejected with an error message and also will not fail in the engine.

Table 6.1: Complexity of Graph Operations on the Adjacency List

Operation	Complexity	Where it is used
Add node / add edge	O(1)	Placing and connecting nodes on the canvas
Retrieve successors	O(deg v)	Routing an entity onward
Graph traversal	O(V + E)	Connectivity validation before a run
Space occupied	O(V + E)	Whole-graph representation

6.3.2 Event Scheduling

The **event queue** is the basic structure of a discrete event simulator. It holds all the events scheduled for future simulated times and the engine keeps pulling the earliest event and processing it. It is built as a **binary min-heap** of events keyed by event time, so both insertion and extraction run in O(log n), where n is the number of events in the heap. A sorted list would give constant-time extraction but linear-time insertion and an unsorted list the reverse; the heap gives the best balance. Ties in event time are broken by a counter that only increases, so events scheduled for the same time are handled in registration order. JustCmul8 gets all of this from SimPy, which is an appropriate case of reuse in the architecture.

6.3.3 Random Variate Generation

Variability comes from sampling inter-arrival and service times from the five distributions in Table 1.2. In the generated script, the exponential distribution uses random.expovariate(1/mean). The normal distribution takees the same cut-off; service

durations cannot be negative. The deterministic distribution simply returns the mean and the uniform distribution ranges by ± 50 per cent around the mean. Routing at decision nodes uses **cumulative-probability sampling**: a uniform random number is drawn and compared with the running sum of the configured route probabilities. Source nodes can also spread arrivals across several successors in round-robin order.

6.3.4 Metric Computation

Utilisation is found by adding up busy-seconds. Whenever the number of busy servers changes, the time since the last change is multiplied by the previous busy count and added to a running total. Utilisation is that total divided by capacity times run duration. Waiting time is the gap between an entity joining a queue and starting service and the percentiles come from the real per-entity samples. To keep memory bounded on long runs, entity logs are reservoir-sampled. We keep a uniform random sample of whole entity journeys and not just the early part of the run, which would bias the tail percentiles.

The Little's Law check compares the time-averaged number of entities in the system with the arrival rate multiplied by the mean time in the system. It reports the percentage gap and classifies the run as steady-state, transient or accumulating backlog. The bottleneck is the resource or queue node with the highest average waiting time. The system health score is a weighted mix of four parts, as set out in Table 4.4:

Health = Completion (35%) + Utilisation balance (30%) + Stability (20%) + Tail latency (15%)

A maximum utilisation above 88 per cent and again above 95 per cent, lowers the utilisation part step by step. An unstable or inconsistent Little's Law result lowers the stability part and a 99th-percentile wait more than five times the median lowers the tail part.

6.3.5 Monte Carlo Confidence Intervals

For each metric we calculate the mean and the sample standard deviation across the replications and the 95 per cent confidence interval is:

$$CI = \bar{x} \pm 1.96 \times (s / \sqrt{n})$$

where $\bar{x}$ is the sample mean, s the sample standard deviation and n the number of replications. The half-width shrinks with the square root of n, so four times as many replications cut the interval in half. That is why the interface offers presets of 10, 20, 50 and 100 and not a free-entry field.

6.3.6 AI Model Routing

The AI features go through a **model router** and not a hard-coded model. The router keeps a priority list of Gemini models and sorts every failure into unavailable, rate-limited or empty response. A failing model is marked unhealthy for a short time-to-live, so later requests skip it and go to the next candidate. The assistant therefore fails over on its own without the user doing anything and a model that the provider retires does not take the feature down. Generated graphs are validated before they are shown and every recommendation is put forward for approval.

6.3.7 Presence and Broadcast

Collaboration relies on two channel mechanisms. Presence tracks who is in the project: each client publishes a small payload (identifier, name, colour, selected and edited node) and receives the merged state of everyone else. Broadcast carries short-lived messages, namely cursor positions and graph operations, to the other clients, which ignore the messages they sent themselves. Cursor colours come from the user identifier, so the same person always shows up in the same colour.

6.4 Module-wise Implementation

The table relates each module to the size of its implementation. Line counts cover the module's source files and come from the delivered code base.

Table 6.2: Module-wise Implementation Summary

Module	Principal implementation	Approx. lines
1 Platform and Data Layer	Auth pages, dashboard, settings, middleware, Supabase clients, account deletion route	≈ 3,850 (app/)
2 Visual Graph Editor	NodeCanvas (951), NodePalette, layoutEngine, simTypeRegistry (715)	≈ 1,900
3 Node Configuration Panel	NodePropertiesPanel (1,109), shared types (514)	≈ 1,600
4 Simulation Engine	codeGenerator (783), pyodideEngine/Worker, clientEngine, CodeInspectorPanel	≈ 1,300
5 AI Assistant	AIChatPanel (962), chat and generate routes, systemPrompt, graphOps (739), modelRouter (370)	≈ 2,750
6 AI System Optimiser	optimize route (182), flowAnalysis (268)	≈ 450
7 KPI Dashboard and Results	analyticsEngine (806), resolveKpiMetrics, SimResultsPanel (1,047), AdvancedResultsDashboard, eight result tabs	≈ 4,400
8 Monte Carlo and Scenarios	monteCarlo (132), monteCarloRunner, MonteCarloPanel (778)	≈ 970
9 Digital Twin Playback	DigitalTwinCanvas (366), PlaybackControls, timelineSelectors	≈ 510
10 Template Gallery	TemplateGallery (135), templates in simTypeRegistry	≈ 850
11 Executive Report and Share	pdfReportGenerator (235), reportGenerator, jupyterExport, share modal, viewer, share/export routes	≈ 700 (excl. API routes)
12 Real-Time Collaboration	usePresence (143), LiveCursor	≈ 160

6.5 Database Implementation

The schema is in one SQL script that can be run repeatedly because every statement uses IF NOT EXISTS or a guarded block. Tables will come first and then Row-Level Security is switched on for each policies are added in guarded blocks and indexes and functions and the profile trigger finish the script. The excerpt shows the projects table and its policies. The other tables follow the same pattern.

Listing 6.1: Projects table and Row-Level Security policies

```
CREATE TABLE IF NOT EXISTS public.projects (
  id uuid primary key default gen_random_uuid(),
  user_id uuid not null references auth.users(id) on delete cascade,
  name text not null,
  sim_type text not null default 'human_queue',
  graph_json jsonb default '{"nodes": [], "edges": []}'::jsonb,
  updated_at timestamptz default now()
);

ALTER TABLE public.projects ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Users can view their own projects"
  ON public.projects FOR SELECT USING (auth.uid() = user_id);
CREATE POLICY "Users can update their own projects"
  ON public.projects FOR UPDATE
  USING (auth.uid() = user_id) WITH CHECK (auth.uid() = user_id);
```

Public sharing never gives anonymous users access to a table. A share token is looked up inside a security-definer function, which returns only the fields needed to draw the project:

Listing 6.2: Read-only public access through a security-definer function

```
CREATE OR REPLACE FUNCTION public.get_shared_project(token text)
RETURNS TABLE (name text, sim_type text, graph_json jsonb) AS $$
  SELECT p.name, p.sim_type, p.graph_json
  FROM public.projects p
  JOIN public.project_shares s ON s.project_id = p.id
  WHERE s.share_token = token AND s.revoked_at IS NULL
  LIMIT 1;
$$ LANGUAGE sql SECURITY DEFINER STABLE;

GRANT EXECUTE ON FUNCTION public.get_shared_project(text)
  TO anon, authenticated;
```

6.6 User Interface Design

The interface rests on three ideas: plain language, low mental load and familiar interaction. Words come from the user's field and not from simulation theory. A bank model offers a "Teller" and a "Waiting Line", a factory model offers a "Machine" and a "Buffer" and the result tabs are named after what they show: Executive Summary, Traffic Heatmap, Line Buildup Timeline, Station Counters, Person / Item Journey, Live Activity Log and Cost and ROI. The workspace has three regions. A node palette is on the left, the canvas is in the middle and on the right there is a contextual panel that switches between node configuration, AI chat, results and Monte Carlo analysis depending on what the user is doing.

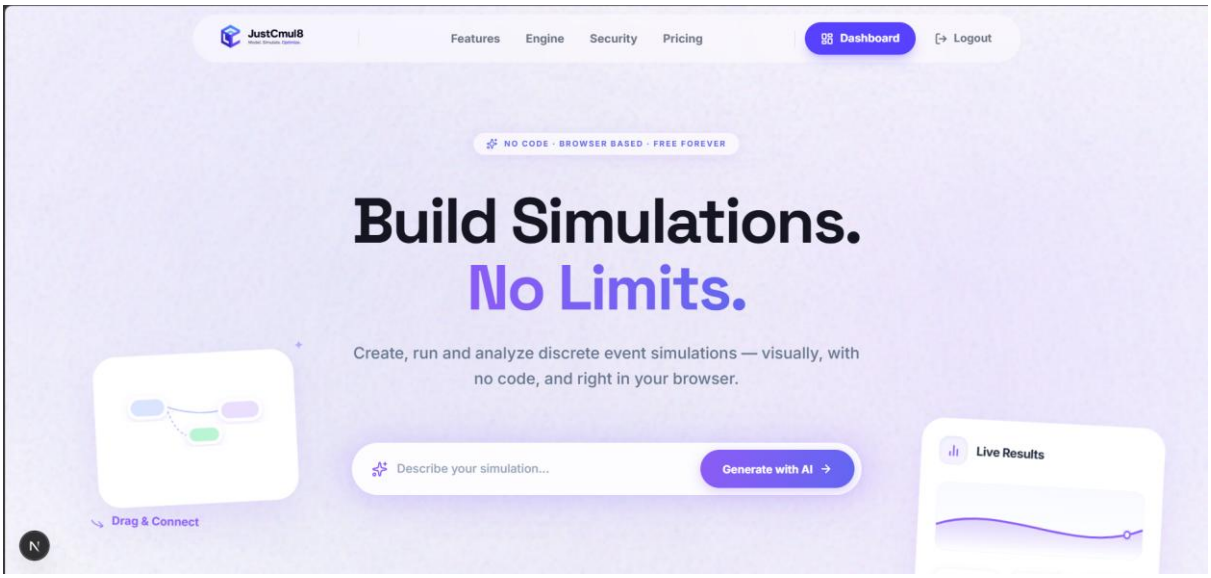


Figure 6.1: Landing Page

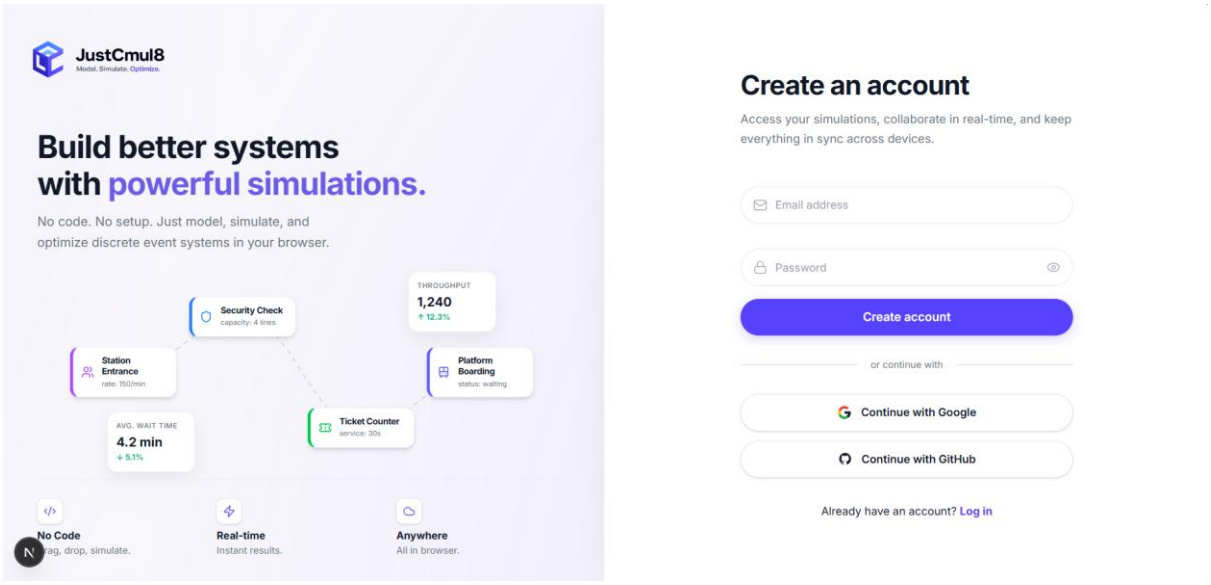


Figure 6.2: Registration and Login Screens

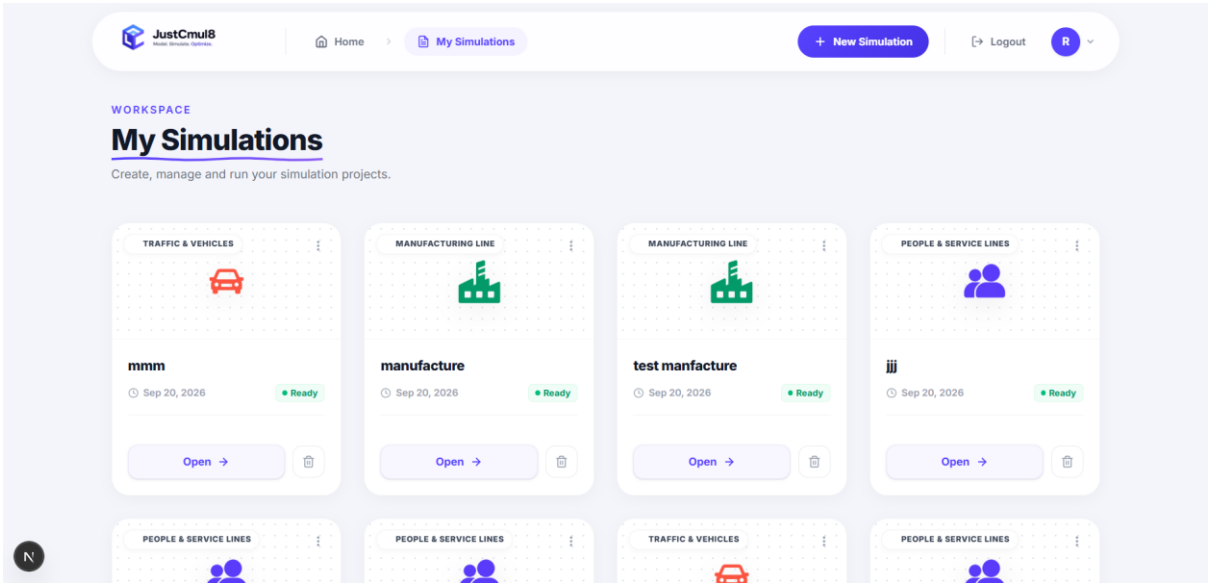


Figure 6.3: Project Dashboard

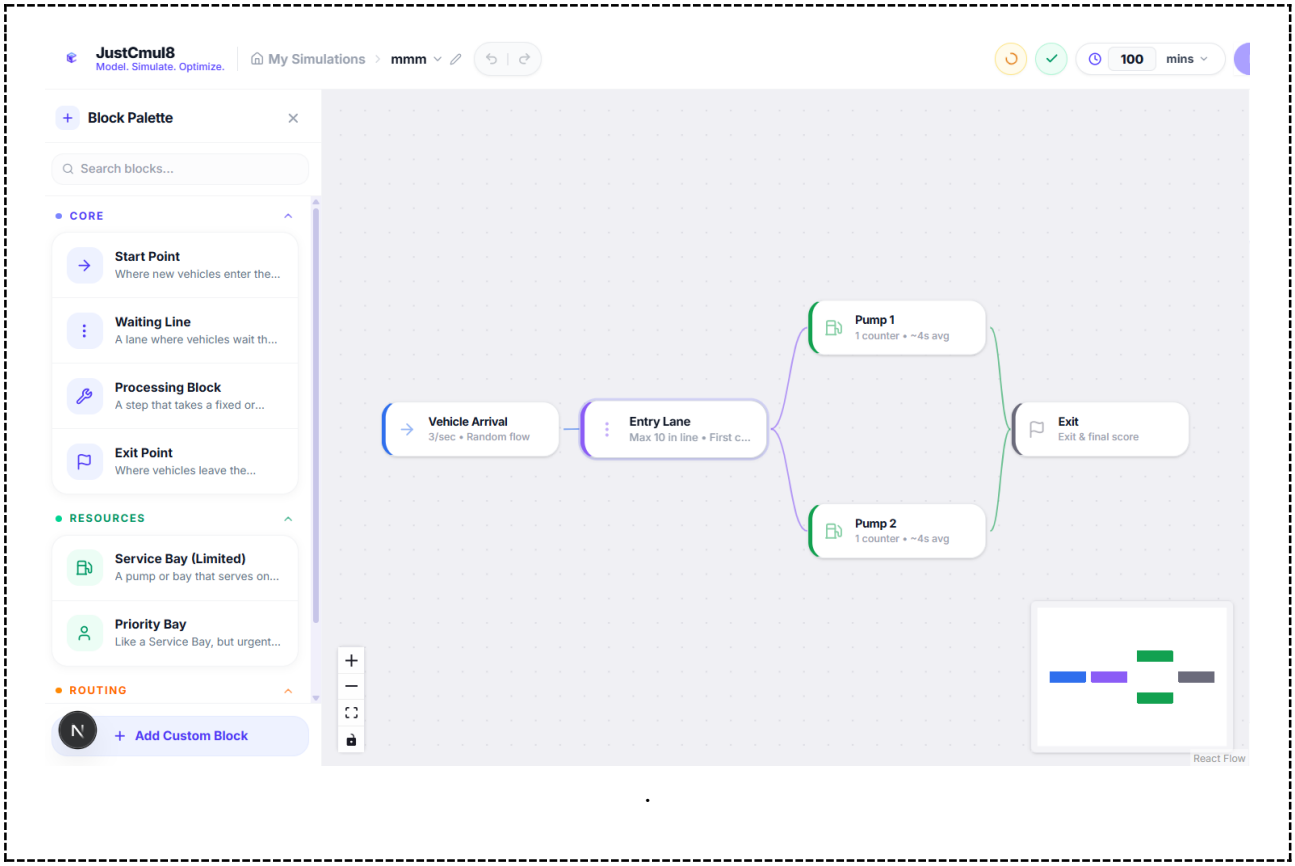


Figure 6.4: Workspace with the Visual Graph Editor

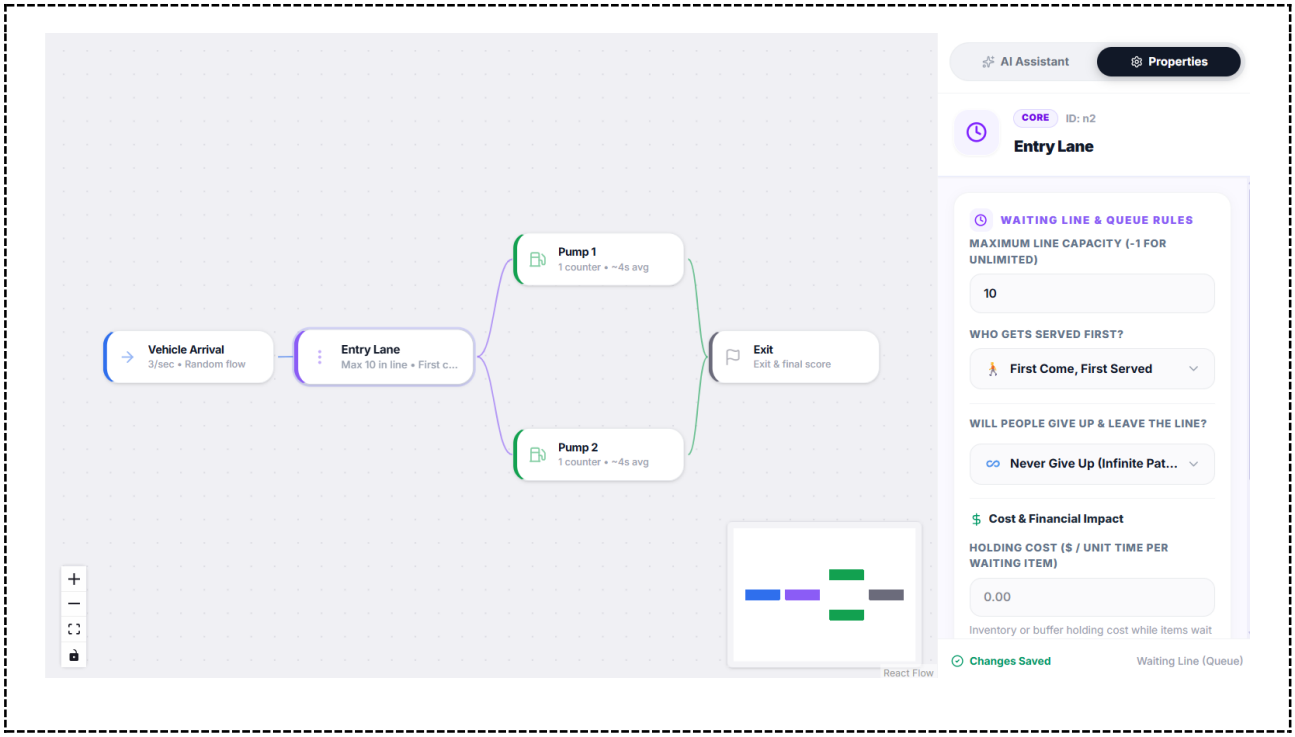


Figure 6.5: Node Configuration Panel

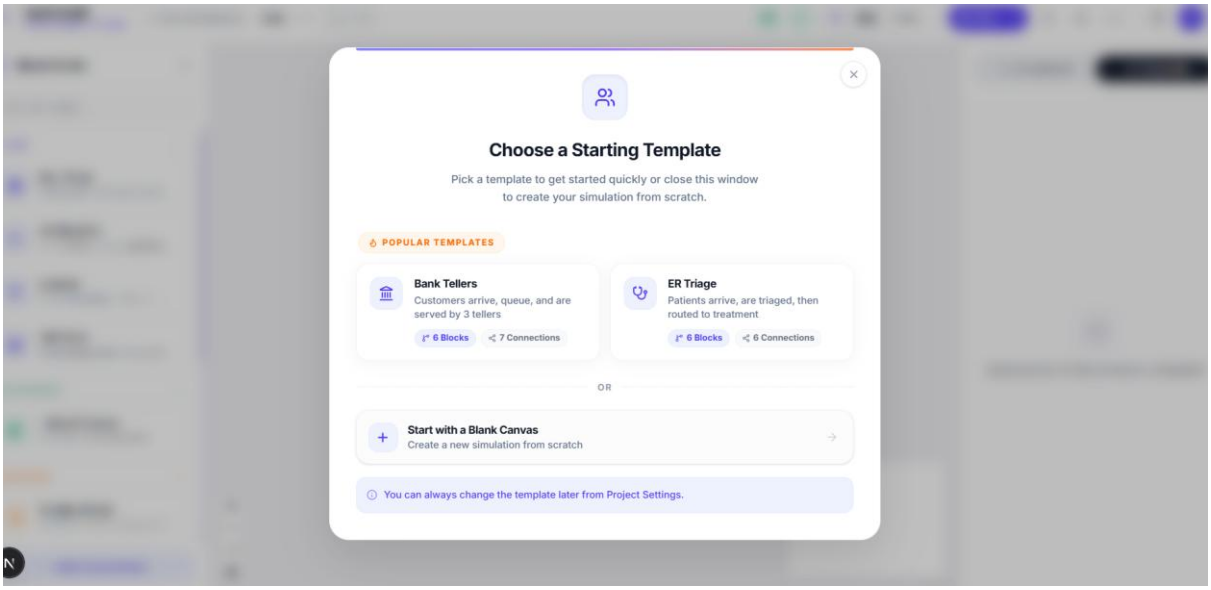


Figure 6.6: Template Gallery

6.7 Important Code Snippets

Here we reproduce the pieces of code that best show how the system works. Each one comes straight from the delivered source and has a short explanation.

6.7.1 Distribution Sampling in the Generated SimPy Script

The generated program has one sampling function that every source and resource calls. It shows how the five distributions of Table 1.2 are done, including cutting the normal distribution off at zero.

Listing 6.3: Distribution sampler in the generated script (codeGenerator.ts)

```
def sample(dist_type, mean, std=None):
    if dist_type == "exponential":
        return random.expovariate(1.0 / mean) if mean > 0 else 0
    elif dist_type == "uniform":
        return random.uniform(mean * 0.5, mean * 1.5)
    elif dist_type == "normal":
        s = std if std is not None else mean * 0.3
        return max(0, random.gauss(mean, s))
    elif dist_type == "deterministic":
        return max(0, mean)
    elif dist_type == "poisson":
        return random.expovariate(1.0 / mean) if mean > 0 else 0
```

6.7.2 Resource Service with Busy-Time Accounting

This part is the heart of the model: an entity will wait , takes a server, is served for a sampled time and lets the server goes . Busy time is added up whenever the busy count which makes the reported utilisation exact and not sampled. The comment records a correction we made to the design. The next hop is returned to the caller so that the server is released before the entity moves on, which stops the server being held for the rest of the entity’s journey.

Listing 6.4: Resource service and utilisation accounting (excerpt)

```
def do_resource_service(env, eid, node_id, cfg, s, resources,
                        queued_at, queue_stats=None):
    params = cfg["params"]
    wait_time = env.now - queued_at
    s["waits"].append(wait_time)
    if s["busyCount"] > 0:
        s["busy_seconds"] += (env.now - s["last_busy_change"]) * s["busyCount"]
    s["last_busy_change"] = env.now
    s["busyCount"] += 1
    svc_time = sample(params.get("serviceDistribution", "exponential"),
                      params.get("serviceTimeMean", 1))
    yield env.timeout(svc_time)
    s["busy_seconds"] += (env.now - s["last_busy_change"]) * s["busyCount"]
    s["last_busy_change"] = env.now
    s["busyCount"] -= 1
    s["entitiesOut"] += 1
```

```
    return get_next_target(node_id)   # caller releases the server first
```

6.7.3 Running Python in the Browser

The worker gets a start message, generates the script from the graph, registers two callbacks in Python's global namespace and runs the script. The callbacks are ordinary JavaScript functions, so the Python code can stream ticks and hand over its result with no network traffic.

Listing 6.5: Executing the generated SimPy script in the Web Worker (pyodideWorker.ts)

```
case "start": {
  const { python } = generateSimPyScript(
    params.graph, params.durationSeconds, params.tickIntervalSeconds);

  pyodide.globals.set("emit_sim_tick", (tickJson: string) => {
    if (!stopped) emit(JSON.parse(tickJson));
  });
  pyodide.globals.set("emit_sim_result", (resultJson: string) => {
    if (!stopped) emit(JSON.parse(resultJson));
  });

  await pyodide.runPythonAsync(python);
  break;
}
```

6.7.4 Confidence Interval Computation

Listing 6.6: Monte Carlo 95% confidence interval (monteCarlo.ts, abbreviated)

```
export function computeConfidenceInterval(samples: number[]): RunStatistic {
  const n = samples.length;
  const mean = samples.reduce((a, b) => a + b, 0) / n;
  const variance =
    samples.reduce((a, b) => a + (b - mean) ** 2, 0) / (n - 1);
  const stdDev = Math.sqrt(variance);
  const marginOfError = 1.96 * (stdDev / Math.sqrt(n));
  return {
    mean,
}
```

6.7.5 System Health Score

Listing 6.7: Composite system health score (analyticsEngine.ts, abbreviated)

```
const completionScore = Math.min(1, completed / arrived) * 35;

let utilScore = 30;                    // utilisation balance
if (maxUtil > 0.95) utilScore = 15;      // severe bottleneck
else if (maxUtil > 0.88) utilScore = 22;

let stabilityScore = 20;               // Little's Law consistency
```

```
if (!littlesLaw.isStable) stabilityScore = 5;
else if (littlesLaw.discrepancyPercent > 20) stabilityScore = 12;

let tailScore = 15;                     // tail latency
if (waitStats.p99 > waitStats.p50 * 5) tailScore = 8;

return clamp(Math.round(completionScore + utilScore +
                    stabilityScore + tailScore), 10, 100);
```

Put together, the fragments show the shape of the platform: a small, readable SimPy program produced from a diagram, a worker that runs it without touching the network, analytics that turn its output into figures you can check and a database that decides who may see what.

7. SYSTEM TESTING

Even if a simulator is simple to use, it can provide the wrong answerand combination of the two can be especially problematic. Interface tests show whether a button works, but only a comparison with independent mathematics can determine whether the numbers behind the button are right. So we tested in two parts: first, verification of the engine against analytical results and invariants and second, functional testing of the application around the engine. This chapter outlines the strategy, the units under test, the test cases, the test approach and the results.

7.1 Testing Strategy

The tests was not completed after the modules but side by side development, for the reason given in Chapter 3. The strategy is layered: narrow, quick tests at the bottom and broader, slower checks above them.

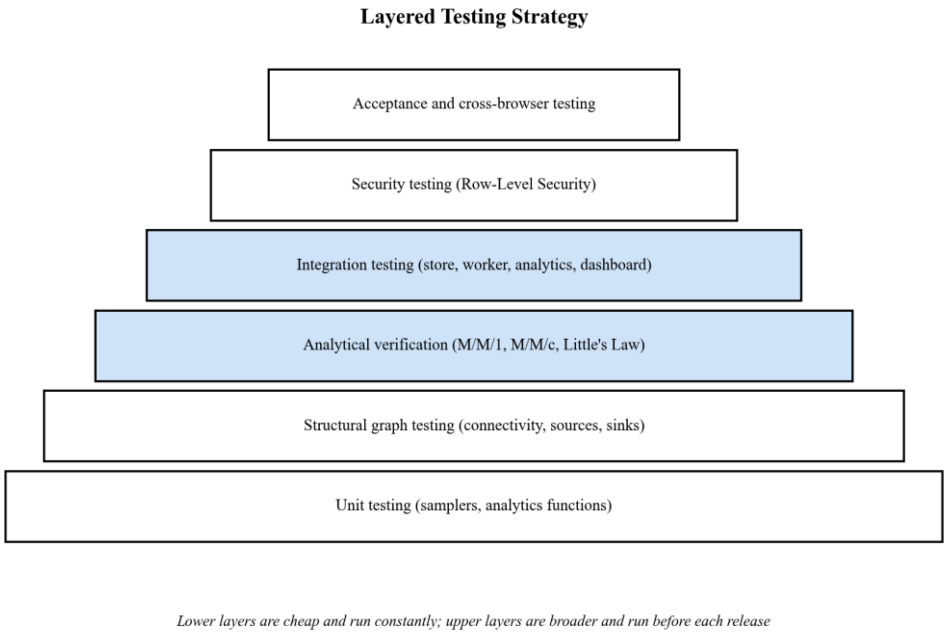


Figure 7.1: Layered Testing Strategy for JustCmul8

- The highest value-added layer is **analytical verification**. The engine's output is evaluated against closed-form results for M/M/1 and M/M/c and against Little's Law.
- **Invariant testing** applies conservation and flow-balance principles to a complex, multi-stage model that has no closed-form solution.
- **Integration testing** covers the paths that cross module boundaries: store, code generator, worker, analytics and dashboard.

- In **security testing**, authorisation checks are made directly against the database rather than through the interface.
- **Cross-browser and acceptance testing** verifies that behaviour is consistent between browsers and that a first-time user gets a result without a tutorial.

7.2 Testable Units

Table 7.1: Testable Units and the Techniques Applied to Them

Unit	Location	What is tested	Technique
Distribution samplers	codeGenerator.ts (generated sample), distributions.ts	Mean, spread and truncation of each of the five distributions	Statistical unit test
Code generator	lib/simulation/codeGenerator.ts	Every node type produces valid, executable SimPy	Execution of generated script
Analytics engine	lib/simulation/analyticsEngine.ts	Utilisation, waits, percentiles, Little's Law, health score	Known-answer test; analytical oracle
Monte Carlo aggregation	lib/simulation/monteCarlo.ts	Mean, standard deviation and 95% interval	Known-answer test
Graph validation	NodeCanvas.tsx (validateGraphConnectivity)	Empty, unconnected and partially connected graphs	Boundary and negative testing
Graph operations	lib/ai/graphOps.ts	Validation of AI-generated graphs	Negative testing with malformed input
Model router	lib/ai/modelRouter.ts	Fail-over between models on error classes	Fault injection
Database policies	supabase/schema.sql	Row-Level Security and share functions	Direct-request security testing
Presence channel	lib/realtime/usePresence.ts	Join authorisation, presence, cursor broadcast	Multi-session integration test
Export modules	pdfReportGenerator.ts, jupyterExport.ts	Content of PDF, CSV, JSON and notebook exports	Output inspection

7.3 Test Case Design with Reference to the Project

We designed two families of test cases. Engine verification cases take their expected results from queueing theory and give numerical outcomes. Functional test cases describe the expected behaviour of the application and trace back to the requirements in Section 2.1.

7.3.1 Engine Verification Cases

The verification suite runs the generated SimPy script as it is, with five independent replications per benchmark over 36,000 simulated seconds and compares the results with theory. B1 is an M/M/1 queue at utilisation 0.8. B1v repeats it without an explicit queue node. B2 is an M/M/3 queue and B3 is a deliberately saturated queue. B4 checks how time units are handled and B5 checks Little's Law. Each deviation is judged against a tolerance that was set in advance.

Table 7.2: Engine Verification Test Cases and Results

ID	Benchmark	Metric	Theory	Simulated	Dev.	Tol.	Result
E-01	B1 M/M/1	Utilisation ρ	0.8000	0.7979	−0.27%	±3.5%	Pass
E-02	B1 M/M/1	Queue wait Wq	4.000	4.005	+0.13%	±3.5%	Pass
E-03	B1 M/M/1	System time W	5.000	5.004	+0.08%	±3.5%	Pass
E-04	B1 M/M/1	Queue length Lq	3.200	3.188	−0.39%	±3.5%	Pass
E-05	B1 M/M/1	Percentiles from real samples (4,000 points)	real	real	n/a	n/a	Pass
E-06	B1 M/M/1	WIP conservation (arrived − completed)	3.8	3.8	n/a	n/a	Pass
E-07	B1 M/M/1	Source node arrivals reported	28,584	28,584	n/a	n/a	Pass
E-08	B1v (no queue)	Utilisation ρ	0.8000	0.7979	−0.27%	±3.5%	Pass
E-09	B1v (no queue)	Queue wait Wq	4.000	4.005	+0.13%	±3.5%	Pass
E-10	B1v (no queue)	System time W	5.000	5.004	+0.08%	±3.5%	Pass
E-11	B1v (no queue)	System WIP L	4.000	3.980	−0.51%	±3.5%	Pass
E-12	B2 M/M/3	Utilisation per server	0.8000	0.7983	−0.21%	±4.0%	Pass
E-13	B2 M/M/3	Queue wait Wq (Erlang-C)	1.079	1.101	+2.09%	±4.0%	Pass
E-14	B2 M/M/3	System time W	2.079	2.100	+1.03%	±4.0%	Pass
E-15	B3 saturated	Total arrivals	1,200	1,181	−1.55%	±5.0%	Pass
E-16	B3 saturated	Total completed	600	603.2	+0.53%	±5.0%	Pass
E-17	B3 saturated	Utilisation ρ	1.000	0.9968	−0.32%	±2.0%	Pass
E-18	B3 saturated	Final WIP within 600 ± 70	530–670	578.2	n/a	2σ	Pass
E-19	B3 saturated	Little's Law verdict (accumulating backlog)	backlog	backlog	n/a	n/a	Pass
E-20	B3 saturated	Health score degrades below 70	< 70	45.8	n/a	n/a	Pass

ID	Benchmark	Metric	Theory	Simulated	Dev.	Tol.	Result
E-21	B4 time units	Run horizon identical in seconds, minutes, hours	3,600	3,600 / 3,600 / 3,600	n/a	n/a	Pass
E-22	B4 time units	Tick interval identical across units	3.6	3.6 / 3.6 / 3.6	n/a	n/a	Pass
E-23	B4 time units	Arrival-count spread across units	< 1%	0.000%	n/a	n/a	Pass
E-24	B4 time units	Tick granularity for a 5 s run	≤ 0.1 s	0.05 s (100 ticks)	n/a	n/a	Pass
E-25	B5 Little's Law	Discrepancy between L and λW	< 5%	2.41%	n/a	n/a	Pass
E-26	B5 Little's Law	Time-averaged true WIP	4.000	3.982	−0.46%	±8.0%	Pass
E-27	B5v (no queue)	Little's Law discrepancy	< 5%	2.41%	n/a	n/a	Pass
E-28	B5v (no queue)	Time-averaged true WIP	4.000	3.982	−0.46%	±8.0%	Pass

7.3.2 Invariant Cases for a Complex Multi-Stage Model

Benchmark B6 is a hospital-flow model with a triage station and a probabilistic decision node that sends 70 per cent of patients to minor treatment and 30 per cent to major treatment, ending in two sinks. We ran it with and without patient abandonment (reneging), at an arrival rate of 1.2 per second over 7,200 simulated seconds. It has no closed-form answer, so the test asserts invariants, which are laws that every correct simulation has to obey.

Table 7.3: Invariants Asserted for the Complex Model (Benchmark B6)

ID	Invariant	Meaning
I1	Conservation residual ≥ 0 and below 5% of arrivals	Arrived = completed + reneged + dropped + stuck; nothing is created or lost
I2	Flow balance at every node	Entities in = entities out + current depth
I3	Decision split equals the configured probability	70/30 routing is honoured (observed 0.700 with abandonment on, 0.687 off)
I4	Total completed equals the sum over all sinks	Sink accounting is complete
I5	Throughput clears offered load (≥ 95%)	Stable stations serve what they are offered
I6	Per-station utilisation approximates ρ	Triage ≈ 0.72, minor ≈ 0.70, major ≈ 0.72

7.3.3 Functional Test Cases

The functional cases below cover every module and each one names the requirement it checks. The Status column is for the tester to fill in as each case is run (Pass or Fail, with the date).

Table 7.4: Functional Test Case Design

ID	Module	Test scenario and input	Expected output	Req.	Status
TC-01	Platform	Register with a valid, unused email and a compliant password	Account created, profile row inserted, session started	FR-01	
TC-02	Platform	Register with an already-registered email	Registration refused with a clear message	FR-01	
TC-03	Platform	Register with a weak password	Submission blocked; requirement shown	FR-01	
TC-04	Platform	Log in with correct details	Session issued; dashboard loads the user's projects	FR-01	
TC-05	Platform	Log in with a wrong password	Access refused without revealing which field was wrong	FR-01	
TC-06	Platform	Sign in via Google and with GitHub OAuth	Account linked; session established	FR-01	
TC-07	Platform	Open a workspace URL while logged out	Redirected to login before any data is fetched	FR-02	
TC-08	Platform	Request a password reset and set a new password	Email link works; new password accepted	FR-03	
TC-09	Platform	Create a project in a chosen domain	Project appears on the dashboard with the domain	FR-04	
TC-10	Platform	Rename and delete a project	Name changes; deletion removes project and its runs and chats	FR-04	
TC-11	Platform	Move a node, then reload the page	Position persisted without an explicit save	FR-05	
TC-12	Platform	Query another user's project by identifier	Row-Level Security returns no rows	NFR-05	
TC-13	Platform	Delete the account after confirmation	User row and all owned data removed	FR-06	
TC-14	Graph Editor	Place each of the fifteen node types	Every node renders with correct icon, handles and defaults	FR-07	
TC-15	Graph Editor	Connect two nodes and move one	Edge created and re-routes live	FR-08	
TC-16	Graph Editor	Delete a mid-graph node	Node and incident edges removed together	FR-07	
TC-17	Graph Editor	Run an empty canvas / an unconnected graph	Run refused; message explains the fault	FR-09	
TC-18	Graph Editor	Run a graph with a node unreachable from the rest	Run refused; the disconnected node is named	FR-09	
TC-19	Node Config	Enter capacity 0 or a negative value	Value rejected with a corrective message	FR-11	
TC-20	Node Config	Switch the service distribution	Parameter fields change to match	FR-10	
TC-21	Node Config	Enter routing probabilities not summing to one	Warning shown before the run	FR-11	
TC-22	Engine	Run a valid single-server model	Run completes; arrivals, completions and KPIs reported	FR-12	
TC-23	Engine	Run the same model with duration in seconds, minutes and hours	Equivalent horizons produce equivalent results	FR-13	

ID	Module	Test scenario and input	Expected output	Req.	Status
TC-24	Engine	Interact with the canvas during a long run	Interface remains responsive	FR-14	
TC-25	Engine	Stop a run part-way; change playback speed	Run halts; speed changes apply	FR-15	
TC-26	Engine	Open the code inspector and export the notebook	Readable SimPy shown; .ipynb downloads and runs	FR-25	
TC-27	AI Assistant	Describe “a clinic with two doctors and a waiting room”	Valid graph proposed with source, queue, resource, sink	FR-16	
TC-28	AI Assistant	Inject a response with an unknown node type	Candidate rejected by validation; not displayed	FR-17	
TC-29	AI Assistant	Generate, then discard the preview	Project remains unmodified	FR-17	
TC-30	AI Assistant	Make the preferred model unavailable	Router falls back to the next model	FR-17	
TC-31	AI Optimiser	Diagnose a deliberately under-provisioned server	Correct node identified with a capacity recommendation	FR-18	
TC-32	AI Optimiser	Apply the suggested fix	Graph updated as an ordinary edit	FR-18	
TC-33	KPI Dashboard	Three-stage line with one slow stage	Slow stage flagged as bottleneck	FR-20	
TC-34	KPI Dashboard	Open the Flow and Balance Check for a stable model	Little’s Law discrepancy shown with a steady-state verdict	FR-20	
TC-35	KPI Dashboard	Enter cost parameters and open Cost and ROI	Total cost and the return of a change are reported	FR-21	
TC-36	Monte Carlo	Run 20 replications	Mean, standard deviation and 95% interval per metric	FR-22	
TC-37	Monte Carlo	Apply a fix, then compare baseline and scenario	Per-metric difference shown with both intervals	FR-22	
TC-38	Digital Twin	Play back a completed run; scrub and change speed	Entities animate; position changes without re-running	FR-23	
TC-39	Templates	Instantiate one template from each domain	Canvas pre-populated; every template runs unmodified	FR-24	
TC-40	Templates	Instantiate, then modify the new project	Source template unchanged	FR-24	
TC-41	Export / Share	Export graph as JSON and results as CSV	Files download with correct content	FR-25	
TC-42	Export / Share	Generate the executive PDF report	PDF contains summary, canvas snapshot and KPI tables	FR-25	
TC-43	Export / Share	Open a share link in a private window	Project renders read-only with no editing controls	FR-26	
TC-44	Export / Share	Attempt a direct write using only a share token	Refused by the database	FR-26	
TC-45	Export / Share	Open a revoked share link	Explanatory notice shown instead of an error	FR-26	
TC-46	Collaboration	Open one project in two sessions	Presence and live cursors appear in both	FR-27	
TC-47	Collaboration	Try to join a project channel without access	Channel not joined	FR-27	

ID	Module	Test scenario and input	Expected output	Req.	Status
TC-48	Cross-browser	Run one model on Chrome, Edge, Firefox and Safari	Consistent metrics; canvas and playback render correctly	NFR-08	
TC-49	Responsiveness	Load the workspace at a phone viewport	Layout adapts; panels remain reachable	NFR-08	
TC-50	Profile	Change display name and default domain	Values persist and apply to new projects	FR-06	

7.4 Test Approach

- **Oracle-based testing.** The closed-form M/M/1 and Erlang-C results, Little's Law and the conservation laws are the expected values for the engine. That way a test does not just record what the code happens to do today.
- **Replication and confidence.** Each analytical benchmark has five independent replications and reports a 95% interval, so a pass indicates how the engine will perform and not that luck prevailed in a single attempt.
- **Tolerance discipline.** We set the tolerances in advance and they differ by benchmark: tighter for utilisation and looser for waiting times, which vary more.
- **Invariant testing.** Where there is no closed-form solution, we make assertions about laws that are always true.
- **Negative and boundary testing.** The test data deliberately included zero and negative parameters, zero duration, empty graphs, unreachable nodes and malformed AI responses.
- **Adversarial security testing.** We tested authorisation by sending requests directly to the database, without using the interface: reading another user's project, writing with only a share token and joining a channel without access.
- **Cross-environment testing.** One model is run on various browsers, the metrics are compared and the layout is verified at phone width.
- **User acceptance testing.** People who match the personas in Section 2.1.7 are asked to build and set up a model with no instructions. The goal is to validate that a first result can be reached without any tutorial, which is the true meaning of no-code.
- **Defect management.** Defects are reported as Bugs in Jira, corrected and retested and then closed with the change, so the bug can be traced back to that change.

Entry and exit criteria. A module enters integration testing when its unit tests pass and it has been reviewed. The build passes testing when all of the engine verification cases pass, there is no open defect of high severity and all of the functional test cases have been performed and logged.

7.5 Test Results and Validation

7.5.1 Engine Verification Results

All 28 of 28 analytical checks passed. In the M/M/1 benchmark the simulated utilisation was 0.7979 against a theoretical 0.8000, the mean queue wait was 4.005 s against 4.000 s and the mean system time was 5.004 s against 5.000 s. The 95% confidence interval of every estimate contained the theoretical value. In the three-server benchmark the simulated queue wait was 1.101 s against an exact Erlang-C value of 1.079 s, a deviation of 2.09 per cent, which is inside the ±4 per cent tolerance. In the hospital-flow model all 22 of 22 invariants held, with and without patient abandonment. No deviation from theory was more than 2.1 per cent.

Two things from this exercise are worth writing down. The exact Erlang-C probability of waiting for three servers at an offered load of 2.4 is 0.6472, which gives an exact waiting time of 1.0787 s, so we scored the benchmark against the exact value and not against a rounded figure from a planning brief. And the saturated benchmark showed that the analytics react properly to overload: the health score fell to 45.8 and the Little's Law verdict became accumulating backlog, so a failing system is reported as failing.

7.5.2 Edge-Case Probes

We also ran a set of edge-case probes to see how the engine behaves at the limits of its inputs.

Table 7.5: Edge-Case Probes and Observed Behaviour

Probe	Observed behaviour
Zero service time	Run completes; 39 completed with utilisation 0.000, as expected
Zero duration	Rejected by the engine with a clear error
Zero arrival rate	Recorded for input-validation hardening: the sampler falls back to a default rate
Recurring schedule at time zero	A guard prevents an unbounded zero-delay loop
Very long run (28,714 entities)	Completed in 3.65 s; log retention capped at 20,000 events using an entity reservoir
Floating-point accumulation over 36,000 ticks	Simulated clock reached exactly 36,000.0
Multiple sinks	Both sinks counted (240 + 240 = 480 completed)
Two resources in series (ρ = 0.5 each)	Backlog of 2 over 1,768 arrivals; the first stage remains seized while the entity is in the second stage (blocking after service), which the model documents

7.5.3 Validation against Requirements

Validation asks whether we built the right system. The traceability table links each group of requirements to the evidence behind it.

Table 7.6: Requirements Traceability

Requirement group	Evidence
FR-01 – FR-06 Platform	Functional cases TC-01 – TC-13, TC-50; database policies and cascade rules in schema.sql
FR-07 – FR-11 Editor and configuration	Cases TC-14 – TC-21; graph validation module
FR-12 – FR-15 Engine	Engine cases E-01 – E-28; functional cases TC-22 – TC-26
FR-16 – FR-18 AI	Cases TC-27 – TC-32; graph validation and model router
FR-19 – FR-21 Analytics	Engine cases E-01 – E-28; invariants I1 – I6; cases TC-33 – TC-35
FR-22 Monte Carlo	Cases TC-36 – TC-37; confidence-interval unit test
FR-23 – FR-24 Playback and templates	Cases TC-38 – TC-40
FR-25 – FR-26 Export and share	Cases TC-41 – TC-45; security-definer share functions
FR-27 Collaboration	Cases TC-46 – TC-47; channel authorisation policy
NFR-03 Correctness	All 28 analytical checks within tolerance; 22 of 22 invariants
NFR-05 Access control	TC-12, TC-44, TC-47

8. RESULTS AND DISCUSSION

This chapter displays some of the products of the system and provides an interpretation of the results. It starts with the outputs a user sees, then analyses the numbers from the verification exercise and ends with an evaluation of performance.

8.1 System Output

The interface is explained step by step, listing the results and screens a user will encounter in a run. Every entry describes what should be displayed on the screen and a placeholder marks where the matching screenshot of the running application should go.

When a run finishes, the **Executive Summary** displays the headline information (arrivals, completions, throughput, the flagged bottleneck and the 0 to 100 system health score) in plain English, with a diagnosis and suggested actions. A non-specialist can understand the answer simply by reading it.

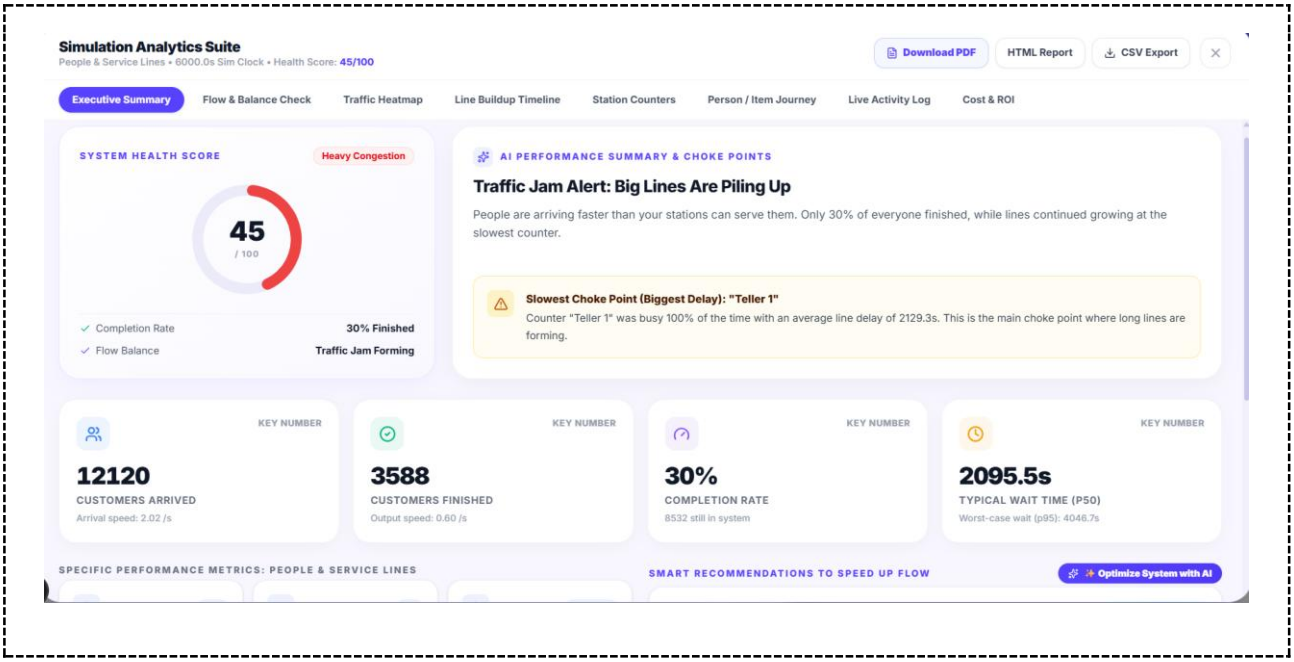


Figure 8.1: Executive Summary Tab of the Results Dashboard

The Flow and Balance Check applies Little's Law and a conservation check to see whether the run agrees with itself. It reports the gap between L and λW and labels the system steady-state, transient or accumulating backlog.

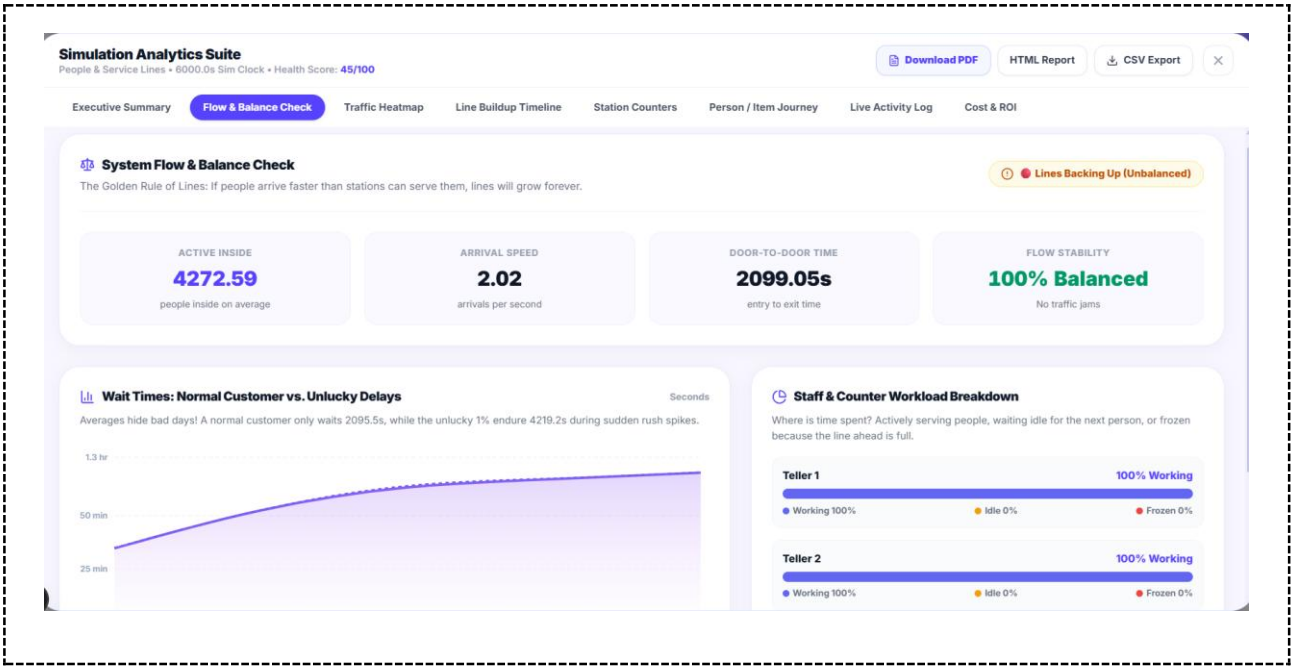


Figure 8.2: Flow and Balance Check (Little's Law Verification)

The Traffic Heatmap lays the load on each node and edge over the model, so congested stations can be spotted at a glance.

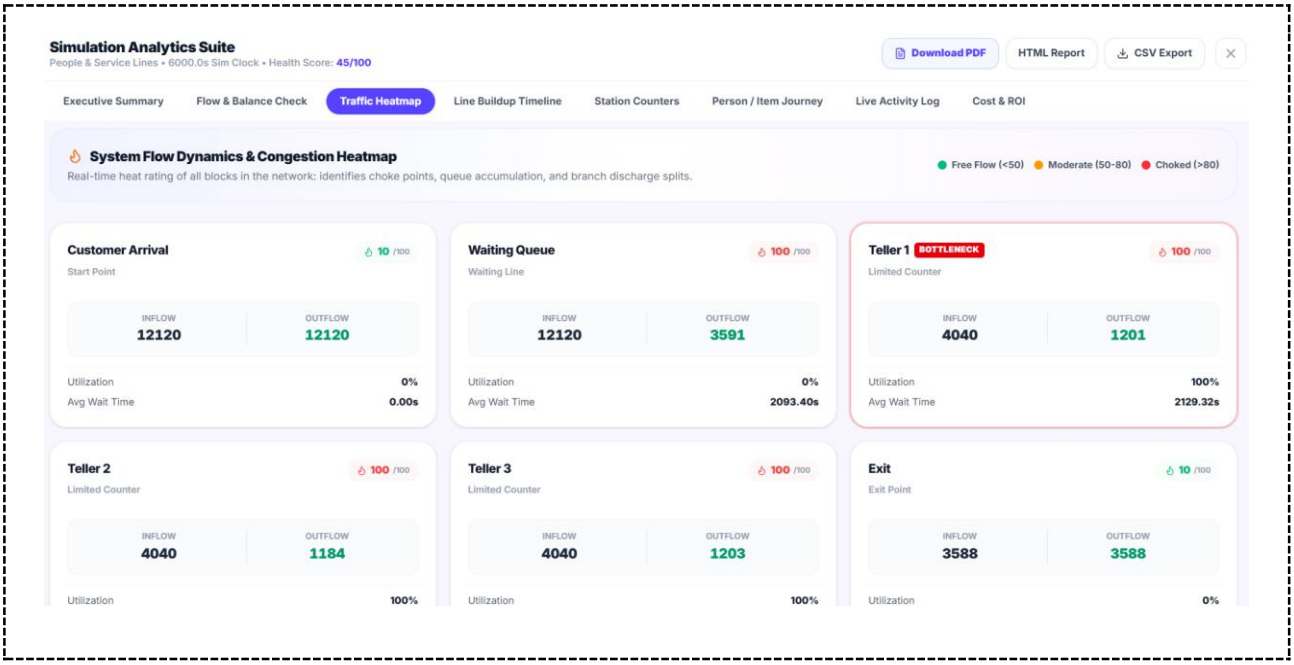


Figure 8.3: Traffic Heatmap

The Line Buildup Timeline plots queue length against simulated time and shows whether a queue is stable, cyclical or growing without limit.

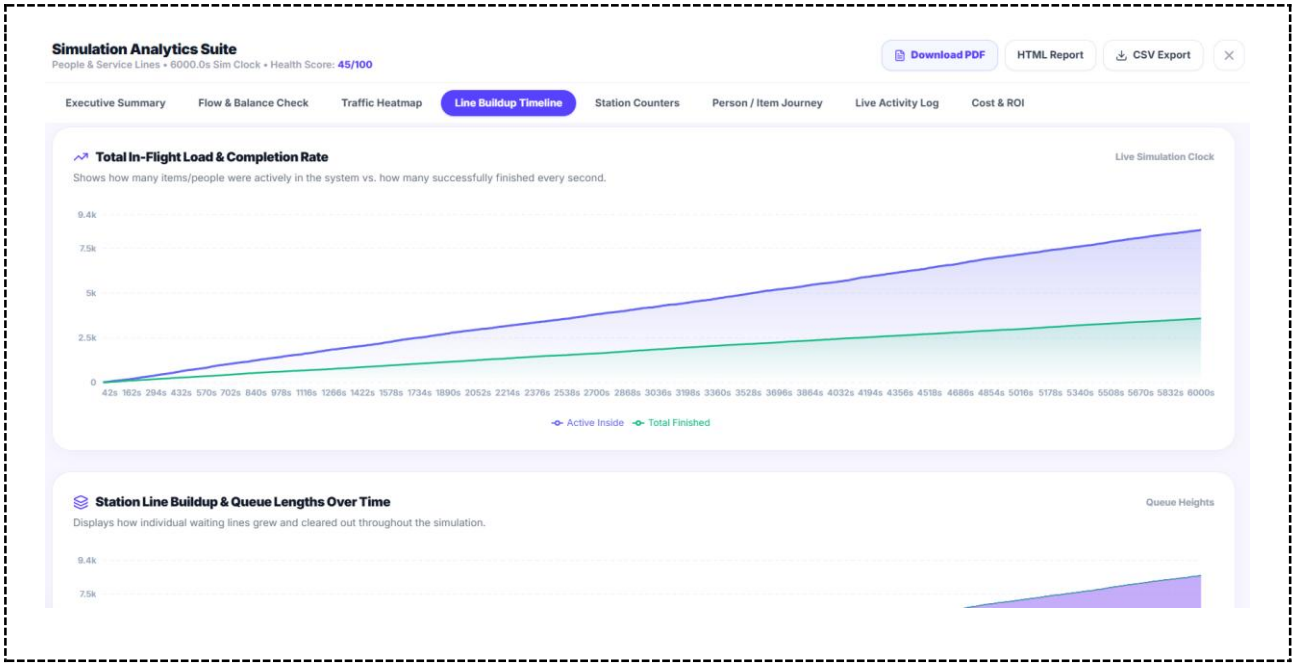


Figure 8.4: Line Buildup Timeline

Station Counters list the utilisation, average and percentile waiting times, throughput and queue depth of every node.

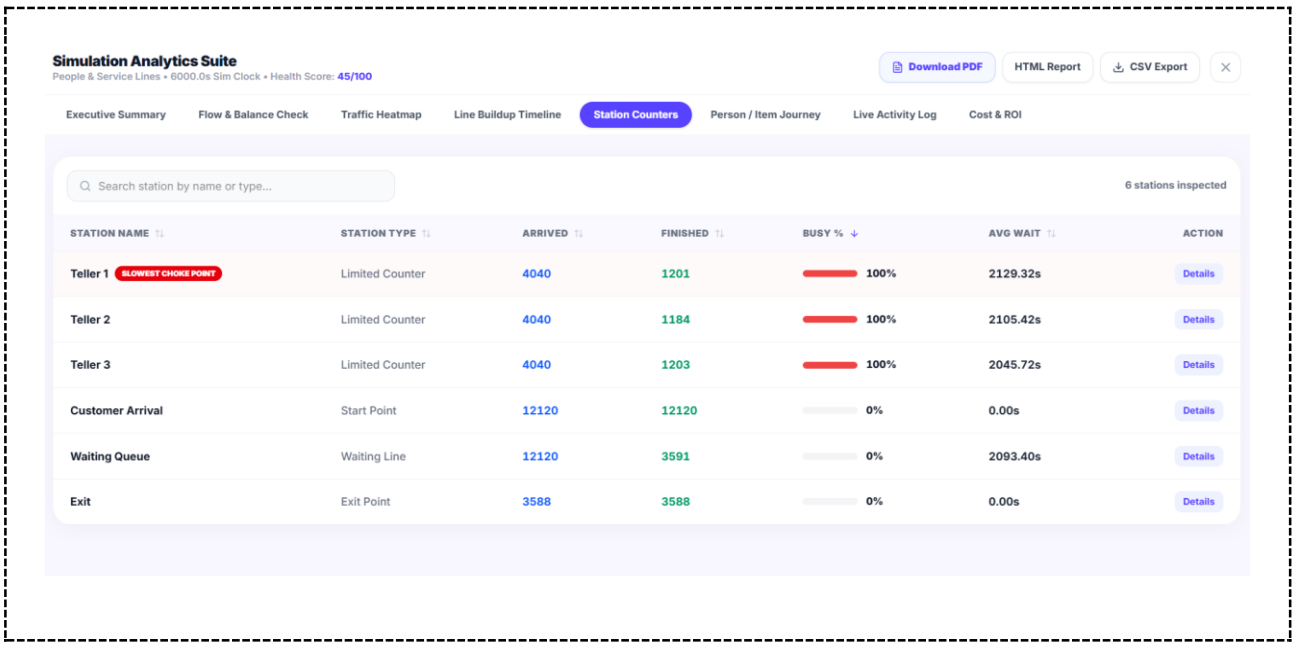


Figure 8.5: Station Counters

The Person / Item Journey tab follows one entity through the model and lists each node it visited with the time it waited and the time it was served. It is the tool for tracing an odd figure back to the events behind it.

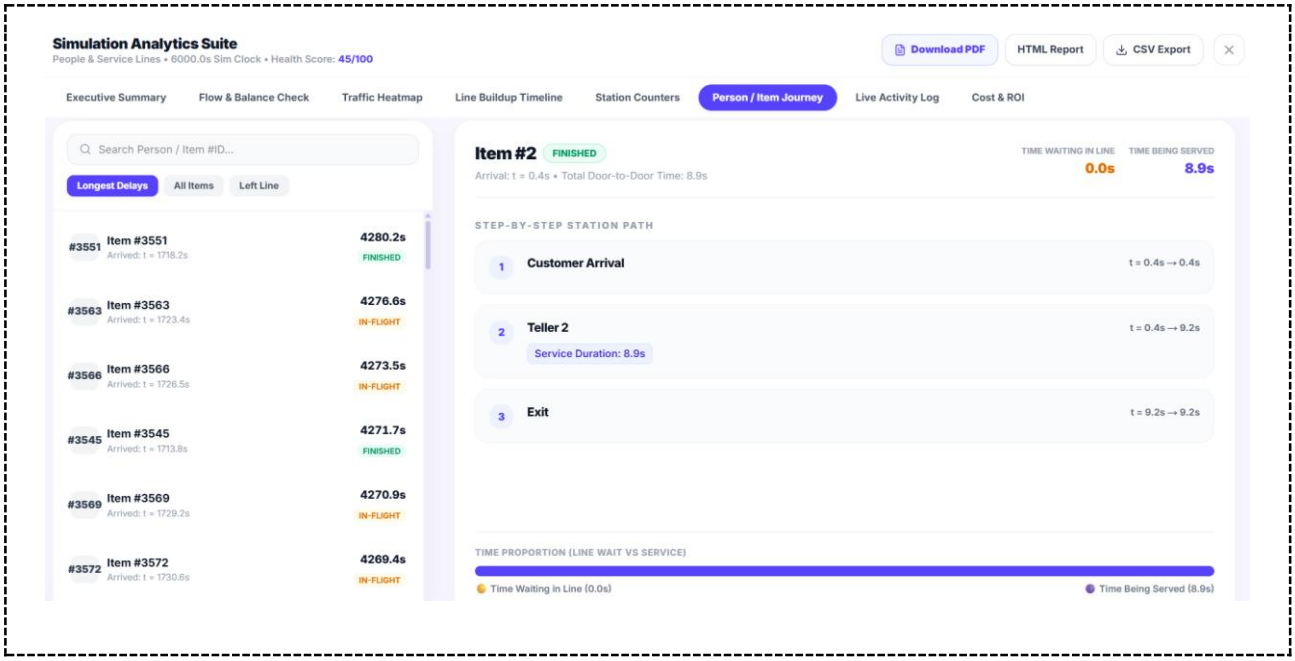


Figure 8.6: Person / Item Journey (Entity Tracer)

The Cost and ROI tab puts the run into money: resource cost over busy time, holding cost weighted by queue length and the waiting penalty. When a change has been proposed, it says whether the saving pays back the cost.

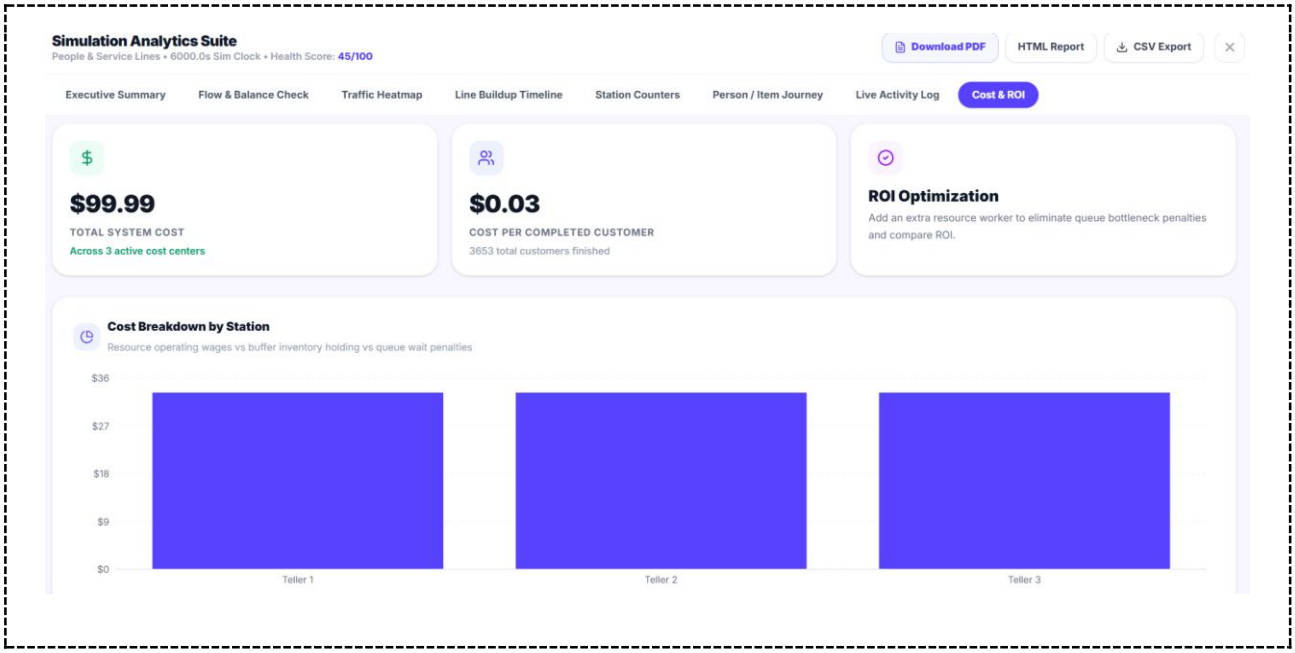


Figure 8.7: Cost and Return-on-Investment Panel

The Monte Carlo panel runs 10, 20, 50 or 100 replications and shows progress as each one finishes. Every metric is drawn as a mean with a shaded 95 per cent confidence band and the baseline-versus-scenario view puts two configurations side by side.

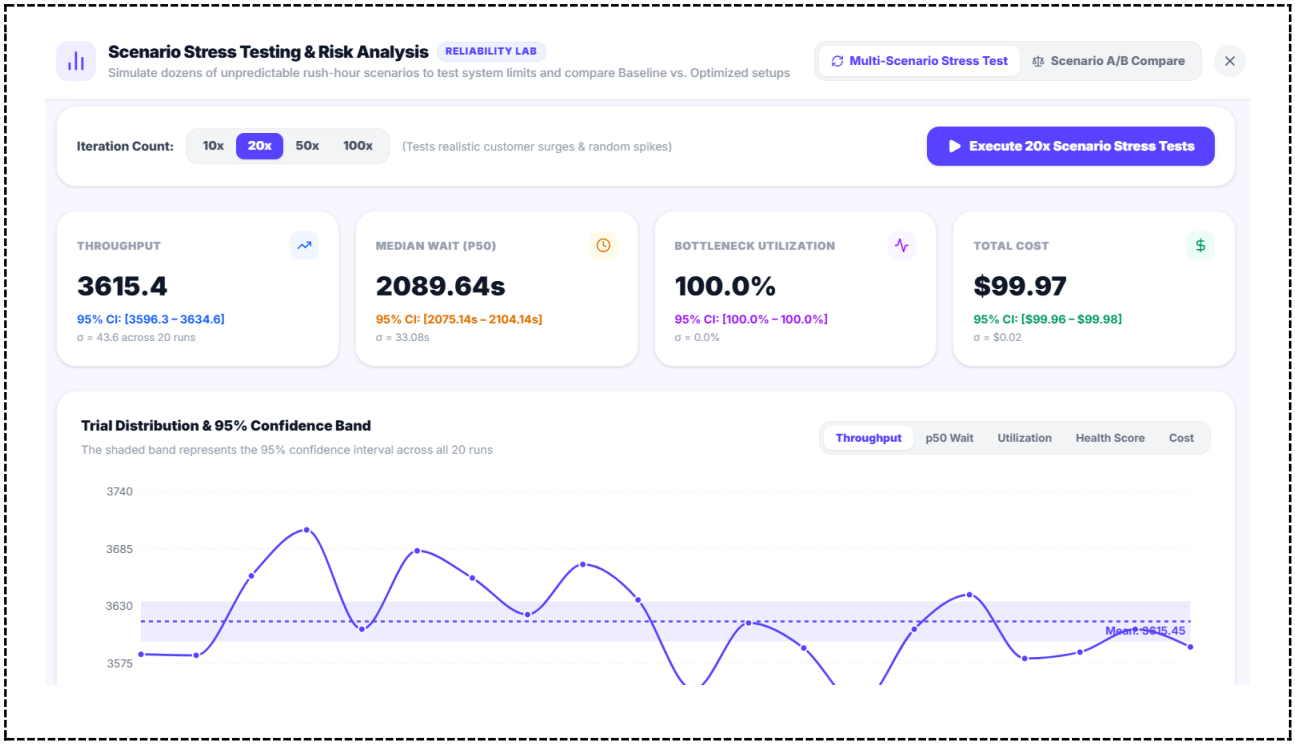


Figure 8.8: Monte Carlo Analysis with 95% Confidence Bands

The Digital Twin replays the run as moving entities on a WebGL canvas. Queue congestion is highlighted and there are controls to play, pause, step, scrub and change the speed.

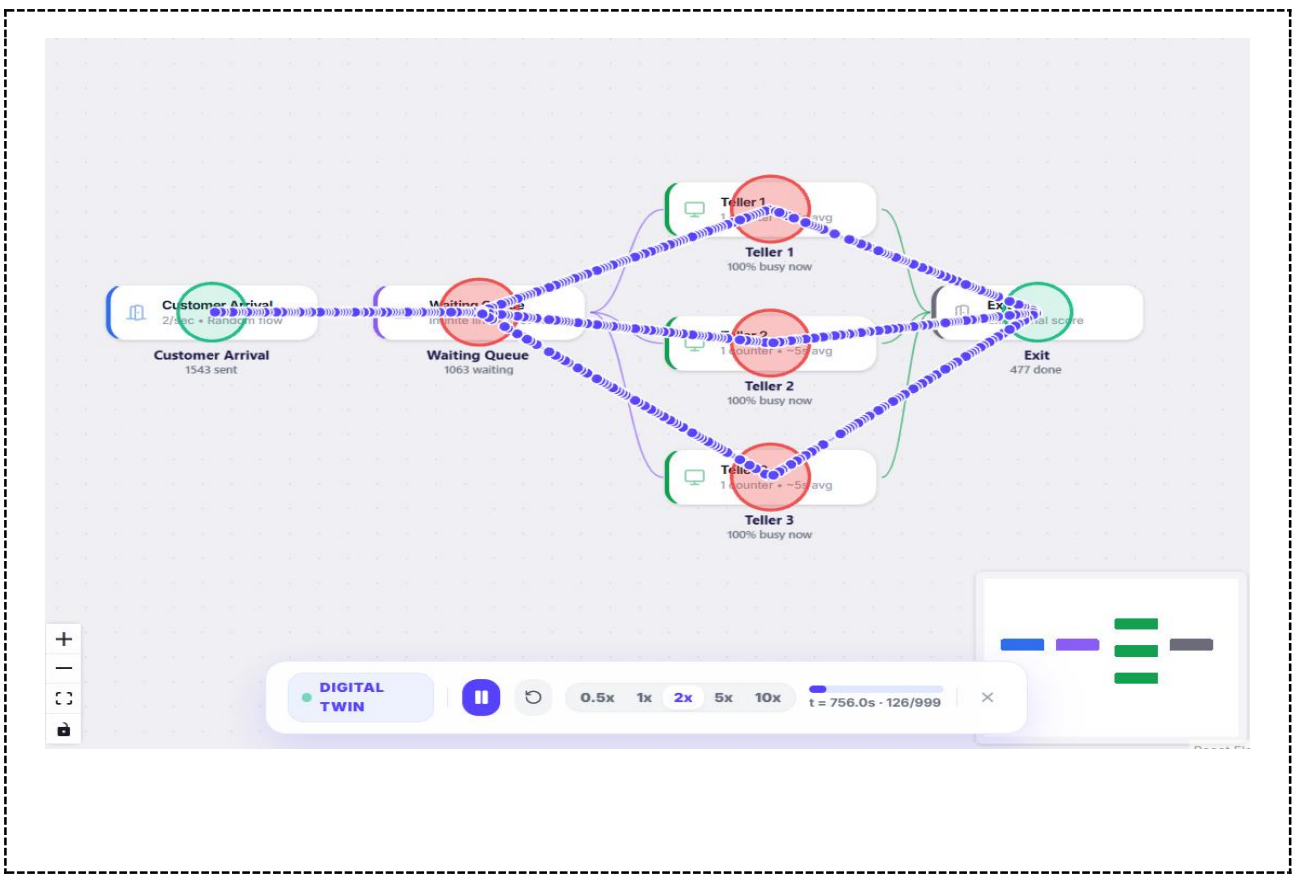


Figure 8.9: Digital Twin Animated Playback

The AI Assistant takes a description in plain English and gives back a candidate graph as a preview. The user has to accept it before the project changes. The same panel answers questions about simulation concepts and about the user's own model.

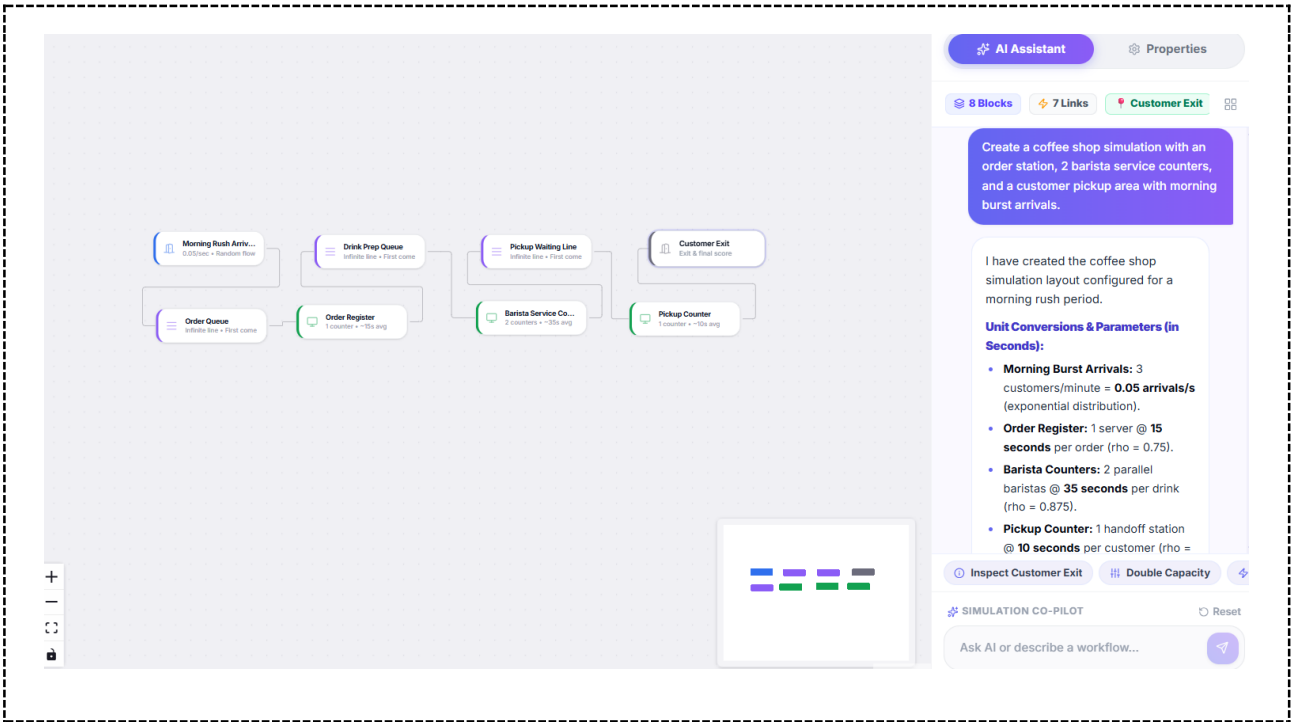


Figure 8.10: AI Assistant Generating a Model from a Description

The AI Optimiser says which node is holding the system back and why, in terms of the computed statistics. Its Apply Fix action carries out the proposed change as an ordinary graph edit.

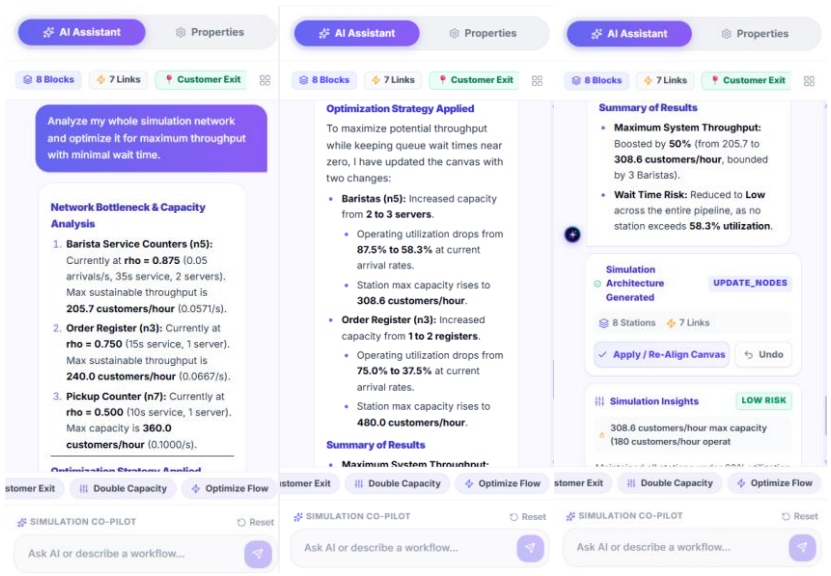


Figure 8.11: AI System Optimiser Recommendation

The Python Code Inspector shows the SimPy script generated from the current graph, so any result can be reproduced independently. One action exports it as a Jupyter notebook.

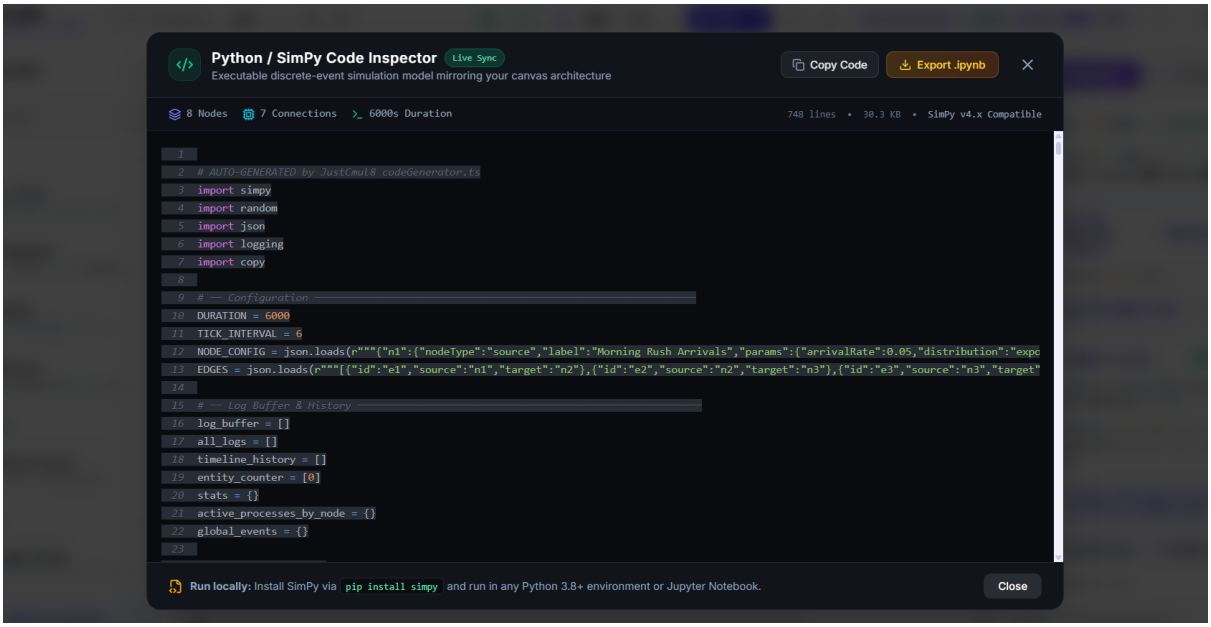


Figure 8.12: Python Code Inspector

In a shared session each collaborator's cursor is drawn in their own colour with their name and a presence row shows who is in the project.

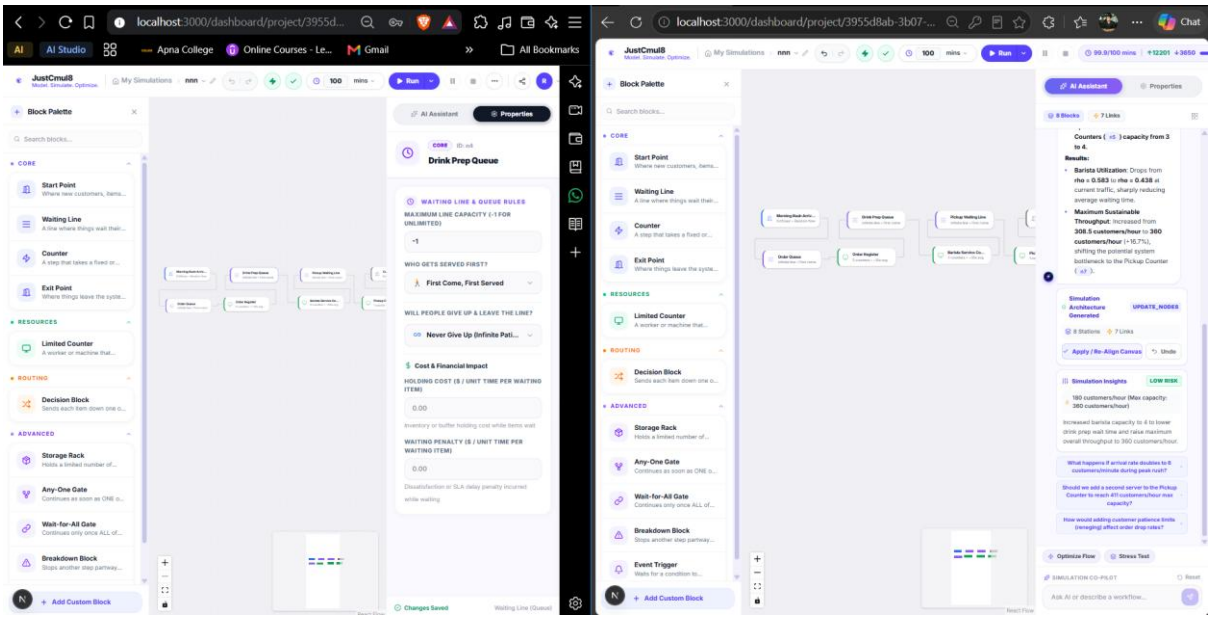


Figure 8.13: Real-Time Presence and Live Cursors

The Export dialog offers the graph as JSON, the results as CSV, the script as a notebook and the full executive report as PDF. It also manages the public share link, with controls to create, copy and revoke it.

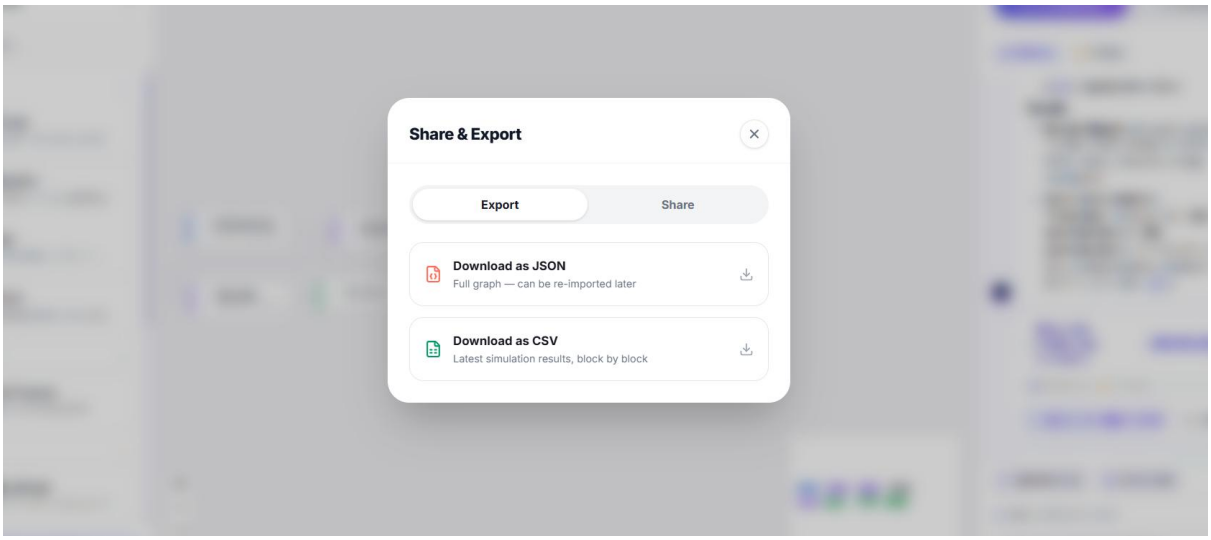


Figure 8.14: Export, Share and Executive PDF Report

A visitor who opens a share link without an account sees the model read-only, with every editing control missing. If the link has been revoked, an explanatory notice replaces the project.

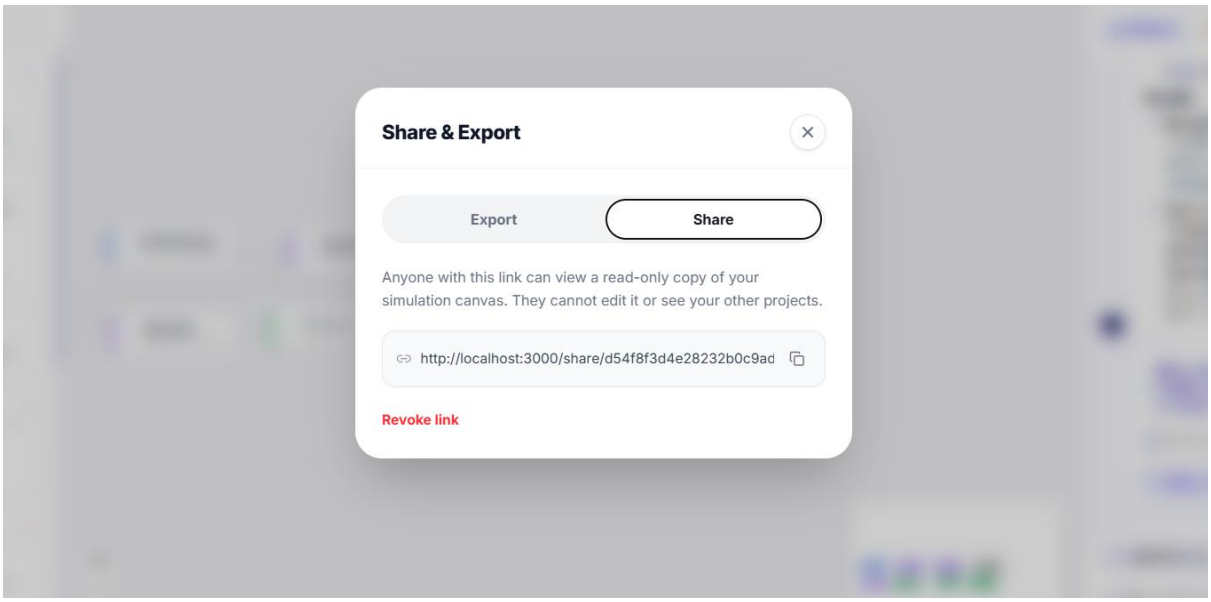


Figure 8.15: Read-Only Shared View

The Settings screen handles the display name and avatar, password change, the default simulation domain for new projects and account deletion, which needs confirmation and cascades through all owned data.

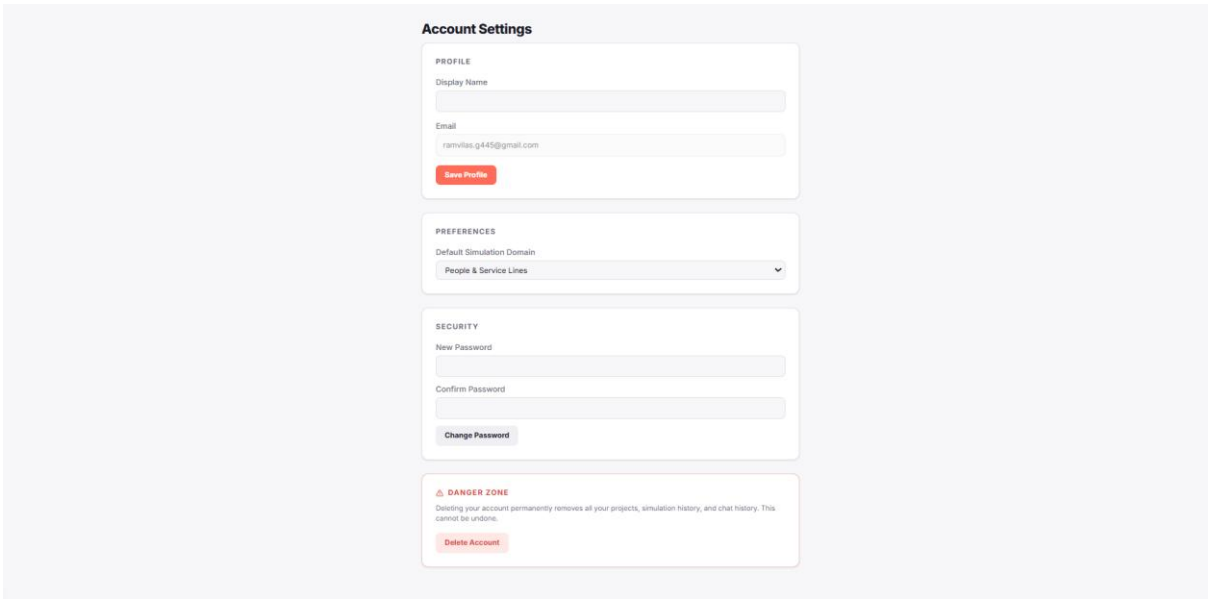


Figure 8.16: Profile and Settings

8.2 Result Analysis

8.2.1 Agreement with Queueing Theory

The key question for any simulator is whether it is right. The figure compares simulated results with the analytical predictions for the M/M/1 and M/M/3 reference queues. In every case the two bars look the same at the scale of the plot.

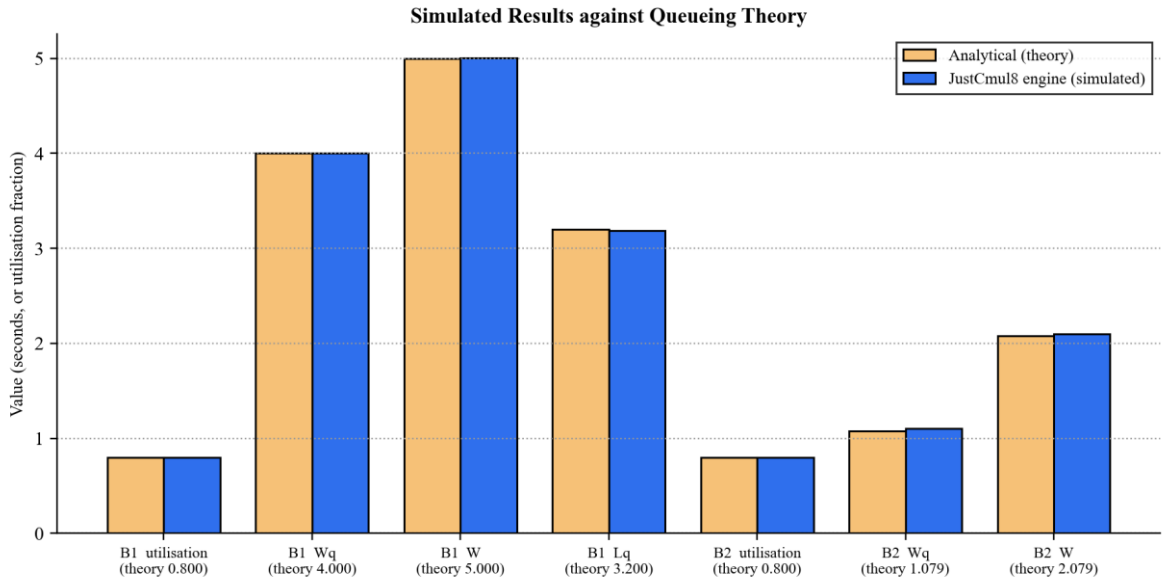


Figure 8.17: Simulated Results against Queueing Theory (M/M/1 and M/M/3 benchmarks)

The second figure shows the deviation of each of the fifteen main checks as a percentage of the theoretical value, next to the tolerance allowed. The biggest deviation, +2.09 per cent for the M/M/3 queue wait, is about half of its tolerance and the M/M/1 deviations are all below 0.6 per cent.

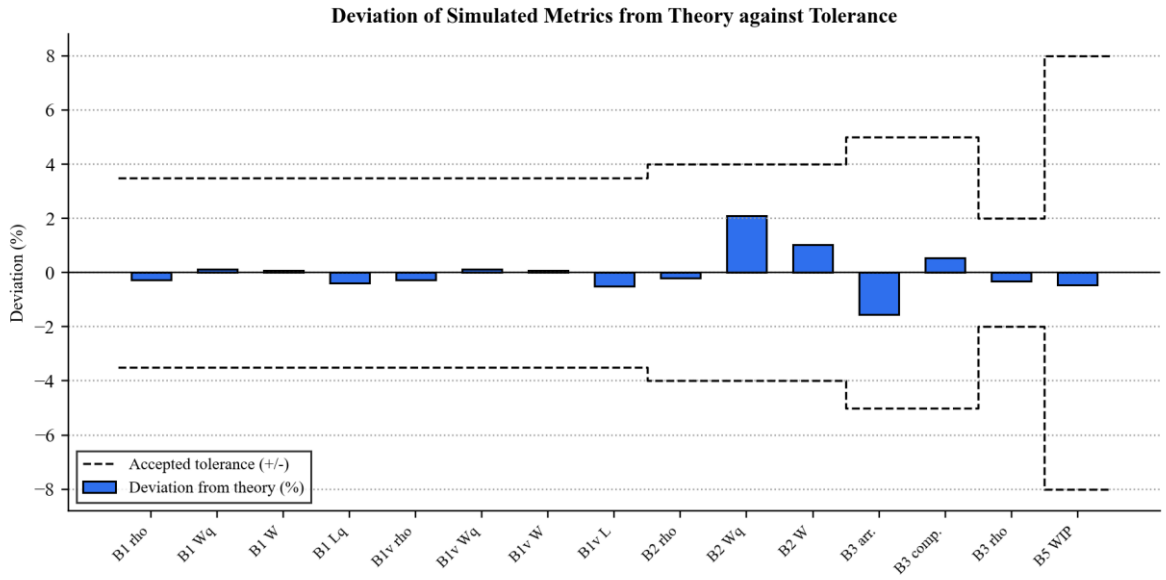


Figure 8.18: Deviation of Simulated Metrics from Theory against Accepted Tolerance

Table 8.1: Worked Example: M/M/1 Queue at ρ = 0.8 (λ = 0.8, μ = 1.0, 36,000 s, 5 replications)

Metric	Formula	Theory	Simulated	95% CI (±)
Utilisation ρ	λ / μ	0.800	0.798	0.0056
Mean wait in queue Wq	ρ / (μ − λ)	4.000 s	4.005 s	0.173
Mean time in system W	1 / (μ − λ)	5.000 s	5.004 s	0.174
Mean queue length Lq	ρ² / (1 − ρ)	3.200	3.188	0.183

8.2.2 A Multi-Stage Model: Hospital Flow

Benchmark B6 puts the platform through a realistic scenario. Patients arrive at 1.2 per second and pass through a triage nurse. A decision node then sends 70 per cent to minor treatment and 30 per cent to major treatment and each station on its own is stable with utilisation near 0.7. The table sums up the results with and without patient abandonment.

Table 8.2: Results for the Multi-Stage Hospital-Flow Model (7,200 s, 3 seeds)

Measure	Abandonment on	Abandonment off
Patients arrived	8,683	8,541
Patients served	8,301	8,531
Patients who abandoned	374	0
Throughput clearance across seeds	0.956, 0.958, 0.954	0.999, 0.999, 1.000
Utilisation: triage / minor / major	0.705 / 0.690 / 0.697	0.708 / 0.682 / 0.726
Decision split to minor treatment	0.700	0.687

Measure	Abandonment on	Abandonment off
Little's Law discrepancy and verdict	3.2%, steady state	0.2%, steady state
System health score	91	93
Invariants satisfied	11 of 11	11 of 11

This is the kind of result a decision-maker needs. With on 4.3 % of patients leave un served yet the software stays stable and the health score of 91 describes an balanced service that is not perfect. With abandonment off the service clears almost all of the offered load. The same model can be run again under a proposed change for eg a third triage nurse and compared through the Monte Carlo scenario view to see whether the gain is bigger than the noise.

8.2.3 The Platform Detects Failure as Well as Success

An analytics layer is only useful if it notices a system in trouble. In the saturated benchmark the arrival rate is twice the service rate. 1,181 entities arrive but only about 603 complete, utilisation is 0.997 and work-in-progress climbs to 578. The Little's Law verdict switches to accumulating backlog and the health score drops to 45.8, well under the alert threshold of 70. So the platform tells a healthy system from a failing one without asking the user to read raw numbers. A separate smoke test also confirmed that a model using the extended node types (broadcaster, channel, store and container) runs without error and conserves flow, with 1,225 entities in and out at every stage.

8.3 Performance Evaluation

We looked at performance on two levels. The first is the speed of the engine, measured by the verification harness, which runs the generated SimPy script in native CPython. The second is how responsive the application is in the browser, measured with the browser's developer tools.

8.3.1 Engine Throughput

Table 8.3: Engine Throughput Measured by the Verification Harness

Measurement	Workload	Observed
Analytical benchmark suite (B1–B5)	Five replications of each benchmark at a 36,000 simulated-second horizon; the M/M/1 benchmark generates ≈ 28,600 entities per replication	159 s wall-clock for the whole suite
Single long run	28,714 entities through one resource	3.65 s wall-clock
Complex model	Hospital flow, 7,200 simulated seconds, 3 seeds, two configurations	Completed within the harness run
Clock accuracy	36,000 accumulated timeouts	Simulated clock exactly 36,000.0

One run with nearly thirty thousand entities finishes in under four seconds and a working day of a typical service model takes far less. A discrete event simulation jumps from event to event, so run time depends on how many events happen and not on the simulated duration and the heap-based event queue keeps the cost per event logarithmic. In the browser the same script runs through WebAssembly, which for CPU-bound Python is usually within a small multiple of native speed.

8.3.2 Application Performance in the Browser

The measurements below are to be taken on the target device with Chrome DevTools and Lighthouse and filled into the table.

Table 8.4: Browser Performance Measurements (to be recorded)

Measurement	Target	Measured
Canvas interaction (drag / connect a node)	Immediate; no dropped frames	
First-run Pyodide download and start-up	Cached thereafter	
Run of a 10-node model, 1,000 simulated seconds	Interactive	

Measurement	Target	Measured
Run of a 100-node model, 1,000 simulated seconds	A few seconds	
Dashboard render after a run	Perceptibly immediate	
Peak browser memory for the 100-node run	Well below device limits	
Lighthouse performance score (landing page)	≥ 90	
Presence / cursor update between two sessions	Near real time	

8.3.3 Discussion

The measurements support the main claim of the project, that a genuine discrete event simulator can run acceptably inside a browser. Verification is the more important result. The engine agrees with queueing theory to within a few per cent, conserves entities across a multi-stage model and classifies both healthy and overloaded systems correctly. So the platform is not just fast enough, it is correct and for a simulator correctness is what counts. Three points follow from the results and feed into the roadmap in Chapter 10. Memory and not processing speed, sets the ceiling on model size. Monte Carlo replications are independent, so they are good candidates for running in parallel. And the first-run download of the runtime is the first thing a new user notices, which makes it the best target for optimisation.

9. COST AND BENEFIT ANALYSIS AND SOFTWARE PARAMETER ESTIMATION

This chapter calculates the effort, time and money the system would take if it were produced commercially by a company and how much it actually cost us as students. We use two independent models: the Constructive Cost Model (COCOMO), based on code size and Function Point Analysis (FPA), based on the functionality delivered. Because we built the system with agentic, AI-assisted development, we also adjust the classic numbers to make them more realistic. The money side is then divided into two views: what a real IT company would spend and what a student team would spend. The benefits of the system are summarised at the end of the chapter.

9.1 Effort Estimation

9.1.1 Basic COCOMO

Basic COCOMO estimates effort from size, using coefficients that differ by the kind of project. The team was in the organic mode: a small team working in an area where they feel at ease, with requirements that are not changing very rapidly and not many hard constraints.

Table 9.1: COCOMO Coefficients by Development Mode

Mode	a	b	c	d	Applicable
Organic	2.4	1.05	2.5	0.38	Yes: small team, known domain, stable requirements
Semi-detached	3.0	1.12	2.5	0.35	No: needs a medium team with mixed experience
Embedded	3.6	1.20	2.5	0.32	No: needs strict hardware and regulatory limits

With the measured size of 22.19 KSLOC (Section 9.2), the formulas give the following:

Effort $E = a \times (KSLOC)^b = 2.4 \times (22.19)^{1.05} = 2.4 \times 25.93 =$ **62.19 person-months**

Duration $D = c \times E^d = 2.5 \times (62.19)^{0.38} = 2.5 \times 4.80 =$ **12.01 months**

Staffing $S = E / D = 62.19 / 12.01 =$ **5.18 persons**

Productivity $P = KSLOC / E = 22.19 / 62.19 =$ **0.357 KSLOC per person-month**

Table 9.2: COCOMO Estimation Results (basic model, written from scratch)

Parameter	Estimated value
Effort	62.19 person-months

Parameter	Estimated value
Development duration	12.01 months
Average staffing	5.18 persons
Productivity	0.357 KSLOC per person-month
Cost at ₹40,000 per person-month	₹24,87,600 (≈ ₹24.88 lakh)

The rupee figure works out to about ₹40,000 per person-month, which is comparable to what an entry-level software engineer costs a company in India once overheads are taken into account. Treat it as a rough guide, because what the model really provides is effort and time. These numbers also assume that people type every line themselves, which is not how the system was developed.

9.1.2 Adjusting COCOMO for Agentic Development

Basic COCOMO was not designed for the way we worked, because people did not write practically every line by hand. Most of the boilerplate, the repetitive interface components and the first drafts of the code came from AI coding agents. Our own time went into deciding what to build, reviewing what the agents wrote, fixing what was wrong and testing the engine against queueing theory. This brings the typing and first-draft effort down a lot, but the design, review and verification effort stays. So we go with a second estimate, taking the effective effort as about 25 per cent of the from-scratch figure. This is not a published constant but our own approximation, so it should be read as a realistic ballpark and not given as an exact number.

Effective effort E = 0.25 × 62.19 = 15.55 person-months

Development time D = 2.5 × (15.55)^0.38 = 2.5 × 2.84 = 7.09 months

Team size S = E / D = 15.55 / 7.09 = 2.19, so 2 to 3 persons

Table 9.3: COCOMO Estimate Adjusted for Agentic Development

Parameter	From scratch (basic COCOMO)	Realistic, with agentic development
Effort	62.19 person-months	about 15.5 person-months
Development time	12.01 months	about 7 months
Team size	5.18 persons	2 to 3 persons
Productivity	0.357 KSLOC per person-month	about 1.43 KSLOC per person-month
Cost at ₹40,000 per person-month	₹24.88 lakh	about ₹6.22 lakh

These figures are close to what we really did: two students, about six months and around 12 person-months of work, against the model's 15.55 person-months and 7.09 months.

9.2 KSLOC Analysis

We measured the size from the delivered source tree, counting the TypeScript, TSX and Python files under the application's source directory and leaving out dependencies, build output and generated files.

Table 9.4: Source Lines of Code by Layer

Layer / directory	Contents	SLOC	Share
src/components	Workspace, landing, layout and shared interface components	11,544	52.0%
src/lib/simulation	Code generator, engine, analytics, Monte Carlo, export, registry	4,905	22.1%
src/app	Pages and API routes (auth, dashboard, workspace, share, AI, export)	3,848	17.3%
src/lib/ai	Model router, prompts, graph operations, flow analysis	1,654	7.5%
src/lib/realtime	Presence and broadcast hook	143	0.6%
src/lib/supabase, hooks	Database clients and helper hooks	59	0.3%
src/middleware.ts	Session refresh and route protection	39	0.2%
Total	91 source files (.ts, .tsx)	22,192	100%

Table 9.5: Measured Project Size

Measure	Value
Source files	91
Total lines of source code	22,192
Non-blank lines	20,286
Size in KSLOC	22.19
Application pages	9 pages and 7 API routes (including the authentication callback)
Node types implemented	15
Simulation domains	6
Delivered modules	12
Database tables (application)	5, plus 2 access functions and 1 trigger

More than half of the code (52 %) is interface code, which shows how much the workspace holds: a canvas editor, eight result tabs, a Monte Carlo panel a WebGL digital twin and a landing page. The engine, analytics and export logic, which decide whether the numbers are

right, take another 22 per cent and the AI layer 7.5 per cent. The database schema is small on purpose, since the graph is stored as a document and Supabase provides authentication and authorisation.

9.3 Function Point Analysis

Function Point Analysis is a technique to detect the functionality given and not the code which makes it useful as cross checked on a size wise estimaion. The IFPUG method counts five kinds of component and gives weight to each one by complexity and adjusts the total with fourteen general system characteristics.

Table 9.6: Standard IFPUG Complexity Weights

Component type	Simple	Average	Complex
External Input (EI)	3	4	6
External Output (EO)	4	5	7
External Inquiry (EQ)	3	4	6
Internal Logical File (ILF)	7	10	15
External Interface File (EIF)	5	7	10

Table 9.7: Function Point Count by Component Type

Component type	Simple	Average	Complex	Count	Weighted
External Inputs: forms, edits and commands	6	8	4	18	74
External Outputs: dashboards, charts and exports	2	4	4	10	56
External Inquiries: listings and retrievals	2	3	2	7	30
Internal Logical Files: application tables	3	0	2	5	51
External Interface Files: Auth, Gemini, Pyodide CDN	1	2	0	3	19
Unadjusted Function Points (UFP)				43	230

The 18 external inputs are the authentication forms, project creation, renaming and deletion, graph edits, parameter configuration, template loading, run initiation, AI prompts, fix application, share link creation & revocation, profile update and account deletion. The ten external outputs include KPI dashboard the bottleneck flag, the Monte Carlo interval charts, the digital-twin playback and the streamed log as well as the entity journey trace and the export formats. The five internal logical files are the database tables. Two of them, projects and runs have complex contents because they contain JSON documents.

Table 9.8: General System Characteristics and Degrees of Influence

No.	General system characteristic	Rating	Justification
1	Data communications	4	Client, server, database and AI service all communicate
2	Distributed data processing	5	Simulation executes on the client, not the server
3	Performance	4	Responsive canvas and immediate dashboard expected
4	Heavily used configuration	2	No constrained deployment hardware
5	Transaction rate	3	Moderate, rising during collaborative editing
6	Online data entry	5	All modelling is interactive and immediate
7	End-user efficiency	5	No-code operation is a primary objective
8	Online update	5	Automatic persistence on every change
9	Complex processing	5	Discrete event simulation, Monte Carlo statistics, AI
10	Reusability	3	Service modules reusable; interface is application-specific
11	Installation ease	5	Zero installation by design
12	Operational ease	4	Managed hosting with automatic deployment
13	Multiple sites	3	Browser-portable across platforms
14	Facilitate change	4	Registry-driven extension of domains and node types
	Total Degree of Influence (TDI)	57	

VAF = 0.65 + (0.01 × TDI) = 0.65 + 0.57 = **1.22**

Function Points FP = UFP × VAF = 230 × 1.22 = **281 function points**

Effort E = FP / productivity = 281 / 13 = **21.6 person-months**

Cost at ₹40,000 per person-month = 21.6 × 40,000 = **₹8,64,000 (≈ ₹8.64 lakh)**

Implied size 22,192 SLOC / 281 FP = **79 SLOC per function point**

For a modern web application like this one, industry productivity is around thirteen function points per person-month. The executed 79 lines per function point is more than the usual 50 to 60 for TypeScript. Table 9.4 shows why there is a large share of the code is presentation and the landing page alone is about 1900 lines of code. That code adds polish and usability but not many function points.

Function points can also be priced directly. The rate per function point depends on who is doing the work, so Table 9.9 applies three rates to the 281 function points of the system.

Table 9.9: FPA-Based Cost Estimate by Rate Tier

Rate tier	₹ per function point	Total cost for 281 FP
College / student team	₹800 – 1,200	₹2.25 lakh – ₹3.37 lakh
Startup / small IT firm	₹1,500 – 2,500	₹4.22 lakh – ₹7.03 lakh
Mid-level IT company	₹3,000 – 5,000	₹8.43 lakh – ₹14.05 lakh

These rates are indicative. The college tier is roughly what a student team would charge for its time and the mid-level tier is close to what an established IT company would quote for the same functionality.

9.4 Comparison of Estimates and Cost–Benefit Analysis

Table 9.10: Comparison of Estimates against Planned Effort

Basis	Effort	Duration	Team	Indicative cost
COCOMO (basic, organic, from scratch)	62.19 PM	12.01 months	5.18 persons	₹24.88 lakh
COCOMO (adjusted for agentic development)	15.55 PM	7.09 months	2 to 3 persons	₹6.22 lakh
Function Point Analysis	21.6 PM	n/a	n/a	₹8.64 lakh
Actual project (planned)	12 PM	6 months	2 students	No salary paid; about ₹1,000 – 19,500 spent

The rows vary markedly and the gap conveys more information than any single number. Neither basic COCOMO nor FPA can take advantage of the head start provided by the modern framework stack the system is built on. The simulation engine is the most obvious case. A discrete event scheduler with resource contention, preemption, container levels and typed stores would take roughly a third of the COCOMO estimate by itself, yet we got all of it from SimPy and only had to build the code generator for it. React Flow gave us the canvas and Supabase gave us access control and the realtime transport. On top of that, the system was built with AI coding agents, which did much of the first-draft typing and left us with design, review and testing. That is why the adjusted COCOMO row, with two to three people and about seven months (7.09 by the model), sits much closer to reality than the from-scratch row. The 22 thousand lines of code that we wrote sit on top of a very large amount of code that we did not write. FPA is also closer to our effort than basic COCOMO is, because it focuses on end-user capabilities and not on code size.

The estimates therefore provide an upper limit on the cost of a system built without such foundations and they show the value of the open-source community and modern tools to a project like this. In the current digital age, a company building the same system with a small team and AI coding agents would probably land somewhere between the adjusted COCOMO figure and the FPA figure and that is precisely what Section 9.4.3 does, with actual rupee numbers.

9.4.1 Role-Based Cost Breakdown

Table 9.11: Role-Based Personnel Cost (Real IT Sector)

Role	Monthly salary	Count	Duration	Subtotal
Full-Stack Developer (Junior)	₹25,000 – 40,000	1	6 months	₹1.50 lakh – ₹2.40 lakh
Full-Stack Developer (Mid-level)	₹50,000 – 75,000	1	6 months	₹3.00 lakh – ₹4.50 lakh
UI/UX Designer (part-time)	₹20,000 – 35,000	1	2 months	₹0.40 lakh – ₹0.70 lakh
QA Tester	₹18,000 – 30,000	1	3 months	₹0.54 lakh – ₹0.90 lakh
Project Manager (part-time)	₹15,000 – 25,000	1	6 months	₹0.90 lakh – ₹1.50 lakh
Total personnel cost				**₹6.34 lakh – ₹10.00 lakh**

Here we assume a company builds the system with a small team that uses AI coding agents. That is why the team is two developers over six months and not the five people and twelve months of basic COCOMO. The salaries are typical monthly figures for junior and mid-level roles in the Indian IT sector and the designer, tester and project manager only join for the part of the project where they are needed.

9.4.2 Infrastructure and Operational Costs

Running the system after it is built has a small monthly cost. The lower end of each range assumes free tiers, which is what we used and the upper end assumes paid plans for a company that wants guaranteed capacity and support.

Table 9.12: Infrastructure and Operational Costs (Monthly and Annual)

Item	Monthly	Annual
Database and authentication (Supabase)	₹0 – 2,100	₹0 – 25,200
Frontend hosting (Vercel)	₹0 – 1,700	₹0 – 20,400
AI inference (Gemini API)	₹200 – 1,500	₹2,400 – 18,000
Domain and SSL	n/a	₹1,000 – 2,500
Email and monitoring	₹0 – 500	₹0 – 6,000
Miscellaneous	₹500	₹6,000
Total infrastructure		**₹9,400 – ₹78,100 per year**

Simulation compute is not in the table because it costs nothing. Every simulation runs in the user's browser, so this bill does not grow with the number of simulations that people run.

9.4.3 Total Project Cost Summary

We look at the cost from two sides, because the answer is very different for a company and for us. The first column shows what a real IT company would spend to build and launch the system. The second shows what we, as students, actually spent. For a company the biggest item by far is salaries. For us that item is zero, because the work was done as part of our course and what remains is a small amount for tools and hosting.

Table 9.13: Total Project Cost, Real IT Sector against Student Project

Category	Real IT sector (company build)	Student project (actual)
Development and personnel (6 months)	₹6,34,000 – ₹10,00,000	Nil (unpaid academic work by two students)
AI coding tools	₹24,000 – ₹96,000	₹0 – ₹12,000 (free and student plans)
Infrastructure (first year)	₹9,400 – ₹78,100	₹1,000 – ₹4,500 (free tiers, domain, small AI usage)
Miscellaneous (printing, binding, internet)	Covered by company overheads	₹0 – ₹3,000
Total	**₹6,67,400 – ₹11,74,100 (about ₹6.7 lakh – ₹11.7 lakh)**	**₹1,000 – ₹19,500**

So the same system that would cost a company somewhere around seven to twelve lakh rupees cost us less than twenty thousand and the difference is almost entirely salaries. A student team also gets many of its tools free and doesn't pay for office space, benefits or management overhead, which a company does.

9.4.4 Summary of Cost Estimates

Table 9.14: Summary of Cost Estimates from All Methods

Method	Estimated cost	Remarks
COCOMO, basic (from scratch)	₹24.88 lakh	Upper bound; hand-written code, 5 people, 12 months
COCOMO, adjusted for agentic development	₹6.22 lakh	25 per cent effort factor, 2 to 3 people, about 7 months
FPA, effort based	₹8.64 lakh	Traditional team at 13 FP per person-month
FPA, priced per function point	₹2.25 lakh – ₹14.05 lakh	College rate up to mid-level IT company rate

Method	Estimated cost	Remarks
Role-based (company, 2 developers)	₹6.34 lakh – ₹10.00 lakh	Salaries only
Real IT sector total	₹6.7 lakh – ₹11.7 lakh	Salaries, AI tools and first-year infrastructure
Student project (actual)	₹1,000 – ₹19,500	Unpaid work, free tiers and student plans

Some cost-saving steps we employed in the system:

- **Open-source stack:** Next.js, React, SimPy, Pyodide and PostgreSQL are free to use, with no cost for licences.
- **Academic scale:** Vercel, Supabase and the Gemini free quota carry the system on free tiers.
- **Client-side simulation:** most of the computation takes place inside the user's web browser, on their own machine, so there is no extra server cost as usage increases.
- **Agentic development:** AI coding agents reduce the team size and timeline to two people and six months.
- **Modular design:** one registry contains the parameters that define the domains and node types, so alterations to this registry are inexpensive.

9.4.5 Benefits

Table 9.15: Benefits of the System

Benefit	Type	Beneficiary
Removes the per-seat licence fees that commercial tools charge	Tangible, recurring	Institutions, small businesses
Removes installation and IT-support effort on every machine	Tangible	Laboratories, IT departments
No server computing cost that grows with the number of simulations	Tangible	The platform operator
Decisions get tested on a computer and not on the real system, which avoids costly mistakes	Tangible, decision-dependent	Small-business owners
Time to first result is minutes and not days of training	Tangible (time)	Students, lecturers
Wider access to a decision-support technique	Intangible	Society, education
A transparent, reproducible engine that helps in teaching SimPy and queueing theory	Intangible	Educators, researchers

The balance between cost and benefit is good. The main cost, computation, moves onto hardware that the user already owns and the main benefit, access to trustworthy simulation, comes with no licence fee. A single commercial seat at a few thousand dollars a year would cost more than the whole annual running cost of the system at academic scale and every extra user adds benefit without adding cost.

10. CONCLUSION AND FUTURE SCOPE

10.1 Achievements of the Project

JustCmul8 made three efforts to bring DES to the majority of users who can benefit from its use: remove commercial license costs; remove some programming skills required for open-source DES; remove installation requirement for desktop software. Below are the outcomes of the project.

- **Complete no-code simulation platform:** twelve interconnected modules, all from the account creation to the simulations, which are modelled in a canvas, analysed, shared and collaborated on, all run through a web browser.
- **A true engine, not an imitation:** compiled to a real (not an imitation) Python SimPy program and then executed by a CPython compiled to WebAssembly from a Web Worker, the results thus come directly from the library used directly by researchers.
- **Verified correctness:** all 28 analytical checks of the correctness of M/M/1, M/M/c and saturated-queue theory passed within tolerance; each of the deviations was below 2.1 per cent; all 22 conservation and flow-balance invariants were satisfied on a multi-stage hospital-flow model.
- **A large application scope:** fifteen different nodes in six application areas such as preemptive resources, typed stores, containers, channels, broadcasters, event synchronisers and interrupters.
- **"Not just numbers":** bottleneck flagging, a Little's Law consistency check, a composite health score, cost/return-on-investment analysis and Monte Carlo confidence intervals with comparison of baseline and scenario.
- **AI-assisted workflow:** plain-English model generation and one-click, cost-aware fixes served via the router that monitors the health of the models and swaps between them.
- **Security built in:** Row-Level Security on each table, opaque revocable share links provided through narrow database functions, secrets stored on the server and cascading deletion of user data.
- **Sustainable economics:** maintaining a low running cost through client-side calculation and an open-source stack enables sustainable economic practicability that is almost independent of usage.

The project related to the academic work in using material from everywhere across the programme rather than just citing it. A few data structure elements are employed in the event scheduler: priority queues and a graph traversal from data structures, which serve as part of the validation pass. The schema and its policies include the concept of normalisation, referential integrity and access control in a DBMS. The software development life cycle is evident in the scheduling decisions of the software project, such as completing the project's riskiest portion first, in Chapter 3. The distribution samplers are also found in the probability and statistics and the check against queueing theory uses probability and statistics.

10.2 Conclusion

The one that had the greatest impact on the result was not to do the engine approximation. Writing a sketch of an event loop in JavaScript would have been much quicker and superficially the platform would have been pretty much the same. Rather, the graph is added to the real SimPy script and the verification outlined in Chapter 7 is used to make the claim become proof. The script that is generated is in agreement with the theory within a couple per cent and thus it is concluded that the statistics is collected correctly and the intended model is expressed.

A set of practical, non-technical items.A collection of functional but not technical products. Within a minute a student can load a template and see a queueing behaviour within that template. The lecturer may show the ‘congested’ situation and provide a share link later. A small-business owner can explain in plain English how to describe a shop and read the confidence interval prior to spending a penny! All this requires no licence and does not require any installation or code and that is what we wanted. JustCmul8 demonstrates that with a Web page and a plain pointing on the path to serious simulation, accessible is easy and affordable without affecting accuracy.

10.3 Future Scope

The architecture has been designed in a way to aid extension. One registry defines each domain, the engine sits behind a worker boundary and the authorisation lives in the database. The decisions made above should not be replaced, but rather expanded on and the roadmap below addresses each of the previously recorded caveats in Section 1.4.

Near-term enhancements

- **Parallel Monte Carlo:** deploying the replications to multiple Web Workers, so a hundred-replication analysis would take about as many times less as there are machine cores, since the replications are independent.
- **Reproducible runs:** a random seed is provided which the user can select and which he/she can then be confident that exact runs of the same seed can be replicated and shared with the seed.
- **Pause and resume:** this allows for checkpoints during a long run, thus stopping the long run and starting again.
- **Streaming result aggregation:** removing the per-entity detail after it is aggregated, but retaining all traces for playback requests, increasing the maximum model size.
- **Role-based collaboration:** e-mail invites with owner, editor and viewer access rights upheld by Row-Level Security; version history and rollback.
- **Convergent conflict resolution:** last-write-wins replaced with mutual merging using OT or CRDTs; concurrent resolution in which two users make changes to the same node and neither one loses any changes.
- **Input-validation hardening:** modifying each boundary condition, such as zero arrival rate and each other condition into a friendly message before a run.
- **Progressive runtime loading:** fetch a smaller Pyodide payload to speed up the start-up during first run.

Medium-term extensions

- **Custom node type plugins:** support custom node type plugins, executed securely within the current node sandbox, for users to create domain models we didn't anticipate.
- **More simulation paradigms:** an agent-based engine and a true continuous solver (subject to the project's needs), in addition to SimPy.

- **Search-based optimisation:** where an optimal configuration is derived from a sweep of the parameter space to achieve the lowest-cost configuration that still meets a service level target.
- **Offline and classroom mode:** a progressive web application that has local storage for labs that aren't guaranteed to be online.
- **Native mobile app** that allows viewing dashboards and playback.
- **A community template marketplace** of publishing and rating models to be used in individual communities.

Longer-term directions

- **Cloud-server execution** will be an option for models too large for memory in the browser, while the execution in the client will remain the default.
- Very large models can be split and distributed among several workers or machines, whose clocks are synchronised, termed **parallel and distributed simulation**.
- **Calibration from real data:** instead of estimates, measured parameters. Fitting of arrival distribution and service distribution are automatically performed based on historical data or sensor data.
- **Live digital-twin integration:** linking a running model to IoT feeds so the simulation follows the physical system it stands for.
- **Institutional features:** single sign-on, organisation management and classroom cohorts for department-wide adoption.

None of these means going back on what we decided in this project. The layered architecture, the worker boundary, the registry-driven definition of domains and node types and the enforcement of authorisation at the database were each chosen to make extension possible and every item above builds on them.

REFERENCES AND BIBLIOGRAPHY

The sources are listed below in plain form: the books and papers for the theory come first, then the standards and finally the documentation of the software that the platform is built on.

1. Simulation Modeling and Analysis, 5th edition, by A. M. Law (McGraw-Hill Education, New York, 2015).

2. Discrete-Event System Simulation, 5th edition, written by J. Banks, J. S. Carson, B. L. Nelson and D. M. Nicol and brought out by Pearson Prentice Hall, Upper Saddle River, in 2010.

3. B. P. Zeigler, H. Praehofer and T. G. Kim are the authors of Theory of Modeling and Simulation (Academic Press, San Diego), 2nd edition, 2000.

4. Simulation: The Practice of Model Development and Use, 2nd edition, by S. Robinson (Palgrave Macmillan, Basingstoke, 2014).

5. The fifth edition of Fundamentals of Queueing Theory by D. Gross, J. F. Shortle, J. M. Thompson and C. M. Harris came out in 2018 from John Wiley & Sons, Hoboken.

6. L. Kleinrock, Queueing Systems, Volume 1: Theory. Wiley-Interscience, New York, 1975.

7. J. D. C. Little published “A proof for the queuing formula: L = λW” in Operations Research, volume 9, issue 3, pages 383–387, in 1961.

8. Introduction to Algorithms, 4th edition, by T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, was published by MIT Press, Cambridge, in 2022.

9. The paper “A note on the generation of random normal deviates”, by G. E. P. Box and M. E. Muller, appeared in The Annals of Mathematical Statistics, volume 29, issue 2, pages 610–611, June 1958.

10. A. B. Owen, Monte Carlo Theory, Methods and Examples (Stanford University, 2013).

11. Software Engineering Economics was written by B. W. Boehm and published by Prentice Hall, Englewood Cliffs, in 1981.

12. In November 1983, A. J. Albrecht and J. E. Gaffney published “Software function, source lines of code and development effort prediction: A software science validation” in IEEE Transactions on Software Engineering, volume SE-9, issue 6, pages 639–648.

13. R. S. Pressman and B. R. Maxim, Software Engineering: A Practitioner’s Approach, 9th edition (McGraw-Hill Education, New York, 2020).

14. Software Engineering, 10th edition, by I. Sommerville (Pearson Education, Harlow, 2015).

15. The requirements standard used is IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications (1998).

16. The Unified Modeling Language User Guide, 2nd edition, by G. Booch, J. Rumbaugh and I. Jacobson, Addison-Wesley, Boston, 2005.

17. Database System Concepts, 7th edition, was written by A. Silberschatz, H. F. Korth and S. Sudarshan and published by McGraw-Hill Education, New York, in 2019.

18. SimPy: Discrete event simulation for Python, documentation by Team SimPy, at https://simpy.readthedocs.io/ (accessed September 2026).

19. The Pyodide Development Team describe Pyodide as Python with the scientific stack, compiled to WebAssembly, at https://pyodide.org/ (accessed September 2026).

20. The WebAssembly Community Group's WebAssembly Core Specification (W3C Recommendation, 2019) is at https://www.w3.org/TR/wasm-core-1/ (accessed September 2026).

21. Vercel Inc., Next.js Documentation: App Router, at https://nextjs.org/docs (accessed September 2026).

22. React Documentation from Meta Open Source is available at https://react.dev/ (accessed September 2026).

23. xyflow maintains the React Flow Documentation at https://reactflow.dev/ (accessed September 2026).

24. Supabase Documentation: Database, Auth, Realtime and Row Level Security, from Supabase Inc., at https://supabase.com/docs (accessed September 2026).

25. The PostgreSQL Global Development Group documents Row Security Policies in the PostgreSQL Documentation at https://www.postgresql.org/docs/ (accessed September 2026).

26. Google's Gemini API Documentation is at https://ai.google.dev/docs (accessed September 2026).

27. TypeScript Handbook, Microsoft Corporation, at https://www.typescriptlang.org/docs/ (accessed September 2026).

28. Using Web Workers, from the Mozilla Developer Network, at https://developer.mozilla.org/en-US/docs/Web/API/Web_Workers_API (accessed September 2026).

29. Function Point Counting Practices Manual, Release 4.3.1, from the International Function Point Users Group (IFPUG, Princeton Junction, 2010).